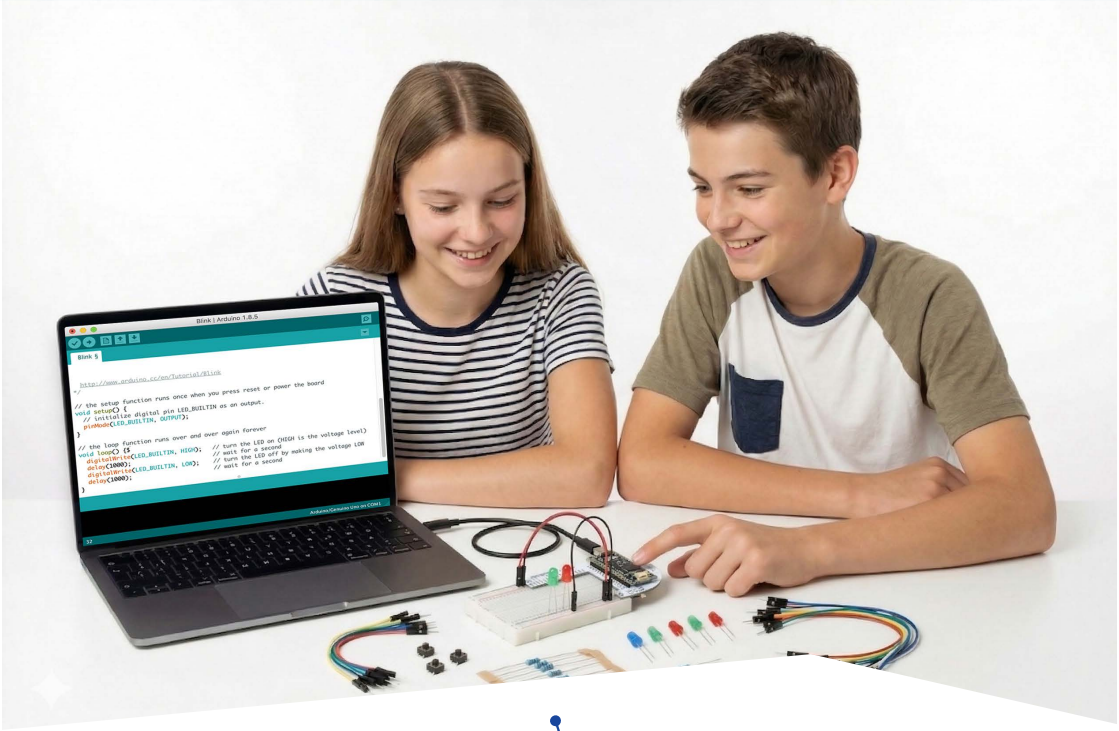
LABORATORIO STEM DI ELETTRONICA

IMPARA E SPERIMENTA

VOLUME 3



ELAB®

© 2025 ELAB. Tutti i diritti riservati.
Nessuna parte di questo libro può essere riprodotta o distribuita in qualsiasi forma
o con qualsiasi mezzo, elettronico o meccanico, senza il permesso scritto dell'autore.

Indice

Capitolo 1 - Un viaggio nella storia della programmazione 05

Capitolo 2 - Cos'è la programmazione? 11

Capitolo 3 - Hardware e Software 17

Capitolo 4 - Introduciamo Arduino 21

Capitolo 5 - Il nostro primo programma (Blink) 29

Capitolo 6 - Il mondo discreto: i pin digitali 43

Capitolo 7 - Il mondo continuo: i pin analogici 79

Capitolo 8 - Visualizzare i dati: Il monitor seriale 113

Capitolo 9 - Il piezo: facciamo musica! 133

Capitolo 10 - Il progetto finale: il gioco Simon 141

Non è finita qui! 171

COMPONENTI DEL KIT

- 1x Breadboard 830 punti
- Cavi di differenti lunghezze
- 1x Clip Batteria intestata con header
- 1x Breakout Nano Elab
- 1x Arduino Nano R4:
- 1x Cavo USB-C
- Resistori 4 bande:
  - 10x 100 Ohm
  - 10x 220 Ohm
  - 10x 330 Ohm
  - 10x 470 Ohm
  - 10x 1 KOhm
  - 10x 10 KOhm
- LED:
  - 5x Blu
  - 5x Rosso
  - 5x Verde
  - 5x Giallo
  - 5x Bianco
- 5x Pulsanti
- 3x Trimmer con alberino 10 kOhm
- 1x Cicalino
- 1x Supporto progetto

UN VIAGGIO NELLA STORIA DELLA PROGRAMMAZIONE

Bentornati!

Oggi scopriamo una cosa davvero importante: **il modo in cui gli esseri umani parlano con le macchine**.

Questo modo si chiama **programmazione**.

Programmare significa dire a un computer cosa deve fare, usando istruzioni chiare e precise, una dopo l'altra.

È un po' come dare indicazioni passo passo, tipo:

- fai questo
- poi fai quello
- infine, fai quest'altro

Ma prima di iniziare a scrivere codice, facciamo un piccolo viaggio nel passato così capiamo **come è iniziato tutto**.

PRIMA DEI COMPUTER: IDEE GENIALI!

Siamo intorno al **1800**. Non esistono ancora i computer, non c'è l'elettricità nelle case e tante invenzioni moderne non sono ancora state create. Eppure succede qualcosa di incredibile.

Una giovane matematica inglese, **Ada Lovelace**, immagina una cosa mai vista prima: una macchina che può seguire istruzioni per fare calcoli difficili.

Ada ha un'idea davvero geniale.

Lei descrive per la prima volta quello che oggi chiamiamo **algoritmo**.

Cos'è un algoritmo?
È una **lista di passi**, in ordine, che servono per fare qualcosa.

Per esempio:

- prendi un numero
- fai un calcolo
- scrivi il risultato

Ada Lovelace è stata la prima a capire che una macchina poteva "seguire istruzioni", proprio come fanno i computer oggi.

Per esempio:

- come muovere i fili
- quale disegno creare sul tessuto

Anche se era solo una macchina meccanica, senza computer, possiamo considerarla uno dei primi esempi di:

- **programma** (istruzioni da seguire)
- **automazione** (la macchina lavora da sola)

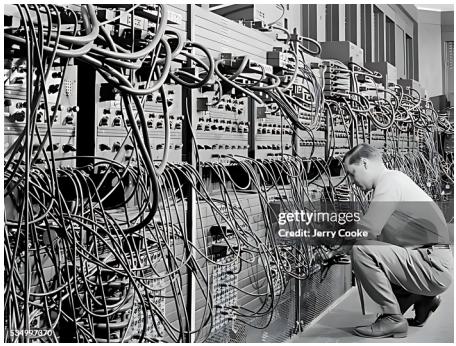
Jacquard aveva già creato una macchina che "seguiva istruzioni" … proprio come fanno i computer oggi!

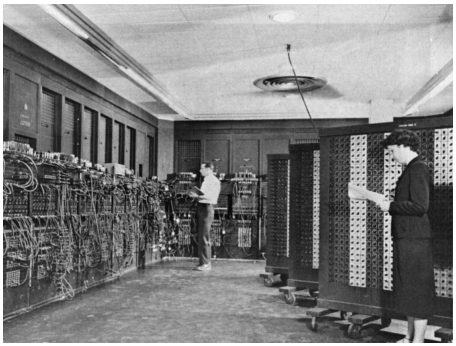
Qualche anno prima, nel **1804**, succede un'altra cosa molto importante.

Un inventore di nome **Joseph Jacquard** costruisce una macchina speciale: un **telaio** (cioè una macchina per fare tessuti).

Questa macchina funzionava in un modo molto intelligente.

Usava delle **schede forate** (cartoncini con dei buchi).
Ogni scheda diceva alla macchina cosa fare.





I PRIMI COMPUTER: GIGANTI LENTI E RUMOROSI

Molto tempo dopo, durante la **Seconda guerra mondiale**, nascono i primi computer elettronici.

Ma attenzione… erano molto diversi da quelli di oggi!

Erano:

- **enormi** (grandissimi)
- **lenti**
- **rumorosi**
- pieni di **valvole** che si scaldavano subito

Uno dei più famosi si chiamava **ENIAC**, costruito nel **1946**.
Era così grande che occupava **un'intera aula scolastica**!

E programmare non era affatto facile…

Non c'era la tastiera
Non c'era lo schermo

Per dare istruzioni al computer bisognava:

- collegare cavi
- spostare interruttori
- cambiare interi pannelli

In poche parole: usare questi computer era complicato e richiedeva tanto lavoro manuale.

IL LINGUAGGIO MACCHINA

Con i computer un po' più avanzati, cambia anche il modo di programmare.

I programmatori iniziano a scrivere istruzioni usando solo numeri: **0 e 1**

Questo si chiama **linguaggio macchina**.

Cos'è?
È il linguaggio che il computer capisce direttamente.

Ma c'è un problema…

! È molto difficile da usare, perché:

- è fatto solo di numeri
- è difficile da leggere
- è facile sbagliare

DA 0 E 1 ALLE PAROLE VERE

Negli anni **'50 e '60** succede qualcosa di rivoluzionario.

I programmatori non devono più usare solo 0 e 1!

Nascono i primi **linguaggi di programmazione leggibili dalle persone**.

Alcuni esempi:

- **FORTRAN** → usato per la matematica
- **COBOL** → usato nelle aziende
- **BASIC** → creato per chi vuole imparare

La grande novità?
Ora si possono scrivere comandi simili all'inglese!

Questo rende la programmazione:

- più facile da leggere
- più semplice da ricordare
- meno piena di errori

In poche parole:
per la prima volta, programmare diventa qualcosa che anche le persone possono capire meglio.

IL BOOM DEL PERSONAL COMPUTER

Ogni scoperta ha portato a nuove invenzioni, che hanno cambiato il mondo e anche il nostro modo di vivere.

Grazie alla programmazione oggi possiamo avere:

- smartphone
- videogiochi
- internet
- robot e intelligenza artificiale

E questo è solo l'inizio!

Chissà quali altre scoperte incredibili ci aspettano nel futuro…

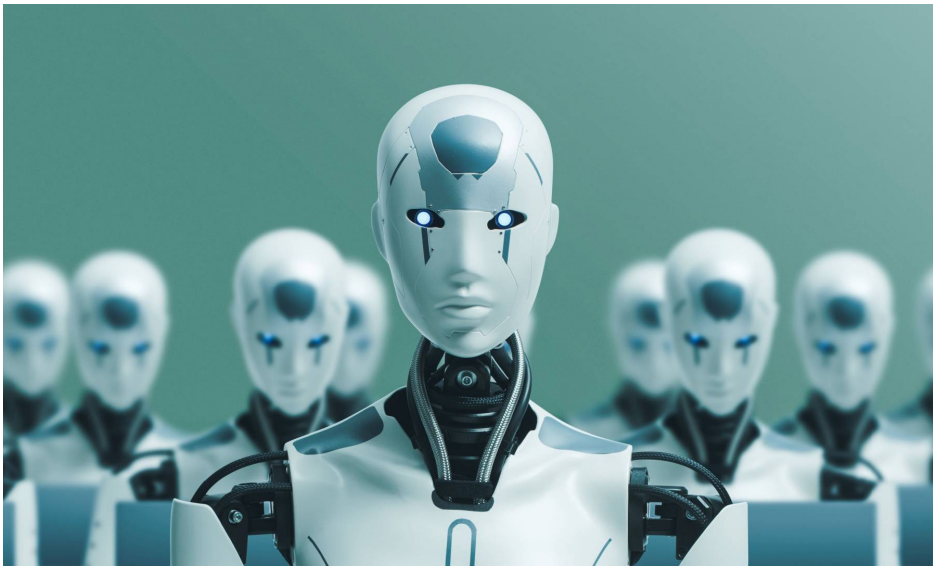
Magari una di queste la inventerai proprio tu!

PROGRAMMARE OGGI: DA ARDUINO ALL'INTELLIGENZA ARTIFICIALE

Oggi la programmazione è ovunque!
Si usa in tantissimi campi:

- a scuola
- nelle aziende
- nella tecnologia di tutti i giorni

Anche con una piccola scheda come **Arduino**, si possono fare cose davvero reali.



Per esempio:

- accendere luci
- controllare dispositivi
- misurare temperatura o distanza
- creare piccoli robot

La cosa importante è questa: le stesse idee usate per accendere un LED sono le stesse che stanno dietro a tecnologie molto più complesse!

Arduino è perfetto per iniziare perché:

- è semplice da usare
- ti fa vedere subito cosa succede
- collega il **codice** al **mondo reale**

Nei prossimi capitoli impareremo passo dopo passo:

- come scrivere il codice
- ma soprattutto **perché funziona**

NOTE

COS'È LA PROGRAMMAZIONE?

La programmazione è il modo in cui **parliamo con i dispositivi elettronici** per dire loro cosa devono fare.

È un po' come scrivere una **ricetta di cucina**.

Immagina di voler fare una torta, hai:

- una lista di ingredienti
- una serie di passaggi da seguire

Devi fare tutto nell'ordine giusto!

Per esempio:

1. prepara gli ingredienti
2. mescola
3. inforna

Se segui bene tutti i passaggi:

- la torta viene buona
- se sbagli, la torta si brucia o non riesce

La programmazione funziona allo stesso modo!

Se diamo istruzioni:

- chiare
- precise
- nell'ordine giusto
- il dispositivo farà esattamente quello che vogliamo.

Programmare significa dare istruzioni per creare cose utili… e anche divertenti!

I LINGUAGGI DI PROGRAMMAZIONE

I dispositivi elettronici, un po' come le persone, usano dei linguaggi per comunicare.

Questi si chiamano **linguaggi di programmazione**.

Proprio come le lingue che parliamo (italiano, inglese…), anche questi linguaggi hanno delle **regole precise**.

Se non rispettiamo le regole:

- la macchina non capisce
- il programma non funziona
- oppure fa cose strane

Ora prova a leggere questa frase:
"Io può volere andare ma se io volere andare come fare per volere andare ad andare?"

È difficile da capire, vero?

Questo succede perché la frase è **scritta male** e non segue le regole dell'italiano.

Con i computer succede la stessa cosa: se scriviamo male le istruzioni, il computer non capisce cosa deve fare.

Per programmare bene, bisogna scrivere istruzioni **chiare e corrette**, proprio come quando si parla o si scrive una lingua.

IL COMPILATORE: UN PROFESSORE SEVERO

Lo stesso problema succede quando si scrive male un programma.

Nel mondo della programmazione c'è una specie di **professore molto severo**: si chiama **compilatore.**

Cosa fa il compilatore?
Controlla se il codice è scritto bene, cioè se rispetta tutte le regole.

- Se è corretto → "ti dà la sufficienza" e il programma parte
- Se ci sono errori → blocca tutto e non ti fa continuare

È molto preciso: non gli sfugge niente!

In programmazione non scriviamo frasi come in italiano, ma **istruzioni**, chiamate anche:**linee di codice.**

Ogni linea di codice serve per fare un'azione.

Per esempio:

- accendere una luce
- leggere un sensore
- far muovere un robot

Il compilatore controlla che tutto sia scritto bene, e ogni riga di codice dice al computer cosa fare.

UN ESEMPIO SEMPLICE

Immagina di programmare un robot.

Possiamo dargli queste istruzioni:

- Vai avanti di 3 passi
- Gira a destra
- Saluta

Il robot farà **esattamente queste azioni**, nell'ordine in cui le abbiamo scritte.

⚠ Attenzione: l'ordine è importantissimo!

Se cambiamo l'ordine, cambia anche il risultato.

Il computer (o il robot) segue le istruzioni **una dopo l'altra**, senza sbagliare e senza decidere da solo.

UN ESEMPIO PRATICO

Prova a fare questo esperimento con un tuo amico.

Immagina di doverlo guidare fino all'uscita della scuola.
Devi scrivere tutte le istruzioni, una per una, per esempio:

1. Alzati dalla sedia
2. Fai un passo in avanti
3. Gira di 90 gradi a destra

Il tuo amico deve seguire **solo quello che hai scritto**, senza inventare niente!

Se riesce ad arrivare all'uscita: hai scritto un **buon programma.**

Se invece:

- si ferma
- sbatte contro un muro
- non sa cosa fare

allora c'è un errore.

Questo errore, in programmazione, si chiama: **"BUG".**

In poche parole:
se le istruzioni non sono chiare o complete, il programma non funziona bene!

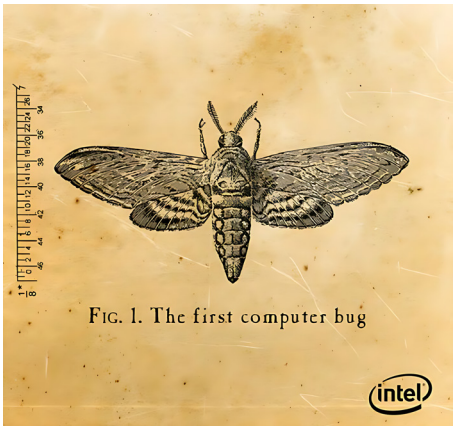
PERCHÉ SI CHIAMA BUG?

La parola **bug** ha una storia davvero curiosa!

Siamo nel **1947**.
Un gruppo di tecnici sta lavorando su un computer chiamato **Mark II**, all'Università di Harvard.

A un certo punto succede qualcosa di strano…
Il computer non funziona bene!

Dopo aver controllato, trovano il problema: una **vera falena** (un insetto) era rimasta incastrata dentro la macchina!



Questo causava il malfunzionamento.

Una scienziata molto importante, **Grace Hopper**, decide di:

- incollare la falena nel registro
- scrivere: *"Primo vero caso di bug trovato"*

Da quel momento, la parola **bug** viene usata per indicare: errori o problemi nei programmi.

Anche se la parola **bug** esisteva già prima, ed era usata per indicare problemi o difetti nelle macchine, questo episodio è diventato famosissimo!

È entrato nella **storia dell'informatica** e ha fatto conoscere la parola "bug" in tutto il mondo.

Da allora, ogni errore in un programma viene chiamato proprio così: **bug**

In poche parole: un piccolo incidente è diventato una parola usata ancora oggi da tutti i programmatori!

Un piccolo incidente è diventato una parola usata ancora oggi da tutti i programmatori.
Un piccolo insetto ha dato il nome a uno dei problemi più comuni della programmazione!

ALGORITMI: LE RICETTE DELLA TECNOLOGIA

Un **algoritmo** è semplicemente una **lista di istruzioni**, messe in un **ordine preciso**, per ottenere un risultato.

Non è qualcosa di difficile:
è solo un modo per dire **"come si fa"** qualcosa.

Per capirlo meglio, pensa a una **ricetta di cucina**:

1. Metti la farina
2. Aggiungi l'acqua
3. Mescola
4. Inforna

Se segui tutti i passaggi nel modo giusto: il dolce viene bene!

Ma se sbagli l'ordine… la torta non riuscirà.

Per esempio:

- prima inforni
- poi mescoli

Impossibile!

Lo stesso succede con i computer: se le istruzioni non sono nell'ordine giusto, il programma **non funziona**.

In poche parole: un algoritmo è come una ricetta: se segui bene i passaggi, ottieni il risultato.

L'ORDINE DELLE ISTRUZIONI È FONDAMENTALE!

Un algoritmo deve essere:

- **chiaro**
- **ordinato**
- **preciso**

La confusione non è ammessa!
Il computer farà **esattamente quello che scrivi**, né più, né meno.

Programmare = trasformare un'idea in realtà

Programmare significa prendere un'idea e farla diventare qualcosa che funziona davvero.

È come avere un **superpotere moderno.**

Se sai spiegare bene le istruzioni, puoi far fare alle macchine tantissime cose!

Cosa puoi fare con il codice

Con la programmazione puoi:

- accendere un LED
- controllare un robot (farlo camminare, girare, evitare ostacoli)
- leggere sensori (temperatura, luce, movimento)
- creare videogiochi
- costruire sistemi intelligenti (allarmi, automazioni, assistenti)

E molto altro!

Non è solo scrivere codice

Programmare significa anche:

- progettare strumenti utili
- realizzare le tue idee tecnologiche
- capire come "ragionano" le macchine
- sviluppare un modo di pensare **logico e creativo**

In poche parole: programmare è trasformare la tua immaginazione in qualcosa di reale.

NOTE

HARDWARE E SOFTWARE: QUAL È LA DIFFERENZA?

Abbiamo iniziato il nostro viaggio nella programmazione, ma prima dobbiamo capire due parole molto importanti:

HARDWARE E SOFTWARE

Hardware

L'hardware è tutto ciò che puoi **toccare con mano**.

Per esempio:

- computer
- smartphone
- tastiera
- Arduino

È la parte **fisica**.

Software

Il software è ciò che **fa funzionare l'hardware**.

Sono i programmi, come:

- app
- videogiochi
- programmi sul computer

Non si può toccare, ma è fondamentale!

La differenza

- Hardware = **parte fisica**
- Software = **programma**

Lavorano sempre insieme!

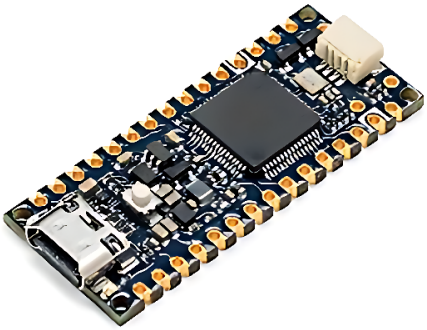
Se manca uno dei due, niente funziona.

L'hardware è ciò che tocchi, il software è ciò che gli dice cosa fare

Nella nostra avventura nel mondo della programmazione con **Arduino** useremo sempre sia l'**hardware** sia il **software**

Il nostro hardware sarà composto da:

- il **PC**
  (che useremo per scrivere i programmi)
- la **scheda Arduino**
  (nel nostro caso: **Arduino Nano R4**)
- tutti i **componenti elettronici**
  (come LED, sensori, fili… che useremo negli esercizi)



Scriveremo il programma sul computer e poi lo useremo per far funzionare Arduino e i suoi componenti.

IL SOFTWARE CHE USEREMO

Useremo due tipi di **software**:

1. L'ambiente di sviluppo Arduino

È un programma che installiamo sul PC.

Serve per:

- scrivere il codice
- controllare se è corretto
- caricarlo sulla scheda Arduino

È lo strumento che ci permette di lavorare.

2. Il programma scritto da noi

È il codice che creiamo noi.

Questo programma viene caricato sulla scheda Arduino e le dice cosa deve fare.

Il primo software serve per **scrivere e inviare il codice**, il secondo è **il codice che fa funzionare Arduino.**

FACCIAMO ANCORA UN PICCOLO PASSO AVANTI

Abbiamo usato la parola **software** per indicare due cose diverse:

- il programma che usiamo sul computer (Arduino IDE)
- il programma che carichiamo sulla scheda Arduino

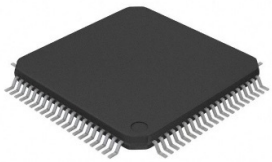
Ma in realtà non sono proprio la stessa cosa!

Il programma che carichiamo su Arduino ha un nome speciale: si chiama **firmware**.

Cos'è il firmware?
È un programma che viene caricato sulla scheda e **rimane lì**, senza cambiare, finché non decidiamo noi di sostituirlo con uno nuovo.

Il **software**, invece, è qualcosa di più "flessibile".
Può essere modificato, aggiornato e cambiato quando vogliamo.

IL MICROCONTROLLORE: IL CERVELLO DI ARDUINO



Con Arduino succede qualcosa di davvero interessante.

Il "cervello" della scheda si chiama **microcontrollore**.

È un **piccolissimo computer**, grande più o meno come un'unghia!

Dentro questo minuscolo chip ci sono diverse parti importanti:

- un **processore** → esegue le istruzioni
- una **memoria** → conserva il programma
- dei **pin** → servono per collegarsi al mondo esterno (LED, sensori, ecc.)

Quando carichiamo un programma su Arduino, stiamo scrivendo delle istruzioni dentro la memoria del microcontrollore.

Da quel momento:
Arduino esegue quelle istruzioni **milioni di volte al secondo**, finché non lo spegniamo o non carichiamo un nuovo programma.

Ed è qui che sta la magia.
Non stiamo solo costruendo oggetti… stiamo creando **comportamenti**!

Ma cosa significa davvero che è un "piccolo computer"?

Dentro al microcontrollore ci sono diversi "pezzi" che lavorano insieme:

- **CPU** → è il "cuore", esegue le istruzioni velocemente
- **Memoria Flash** → salva il programma anche quando Arduino è spento
- **RAM** → memoria temporanea usata mentre il programma funziona

- **Periferiche interne** → aiutano a fare cose specifiche (le vedremo più avanti)

Una cosa molto importante:
il microcontrollore **non ha un sistema operativo** come un PC.

Quando lo accendi:

- parte subito
- inizia dalla prima istruzione
- continua a ripeterla all'infinito

Questo significa che tutto dipende dal programma che hai scritto!

Un'altra differenza rispetto al computer è che: le risorse sono limitate.

Non possiamo:

- fare programmi troppo pesanti
- fare mille cose complicatissime insieme

Ed è proprio questo il bello di Arduino.

Ti fa capire davvero come funziona un computer, in modo semplice e concreto, collegato al mondo reale.

Pensa a questo esempio:

Se vuoi far lampeggiare un LED ogni secondo:

- senza Arduino → devi premere un pulsante ogni secondo (noioso!)
- con Arduino → scrivi il programma e lo fa da solo, in modo preciso

Programmare significa insegnare a una macchina a fare automaticamente e con precisione quello che per noi sarebbe lento e faticoso.

INTRODUCIAMO ARDUINO

Spesso si pensa che **Arduino** sia solo una scheda elettronica.
Ma in realtà non è proprio così!

Questo errore nasce perché una scheda molto famosa, la **Arduino UNO R3**, è conosciuta in tutto il mondo.

Così tante persone finiscono per dire "Arduino" pensando solo a quella scheda.

È un po' come quando:

- chiamiamo tutte le bibite alla cola **"Coca Cola"**
- oppure tutti i fazzoletti **"Scottex"**

In realtà sono solo **nomi specifici**, ma li usiamo per indicare una cosa più generale.

Allo stesso modo:
Arduino non è solo una scheda, ma un insieme di strumenti per programmare e creare progetti elettronici.

MA ARDUINO È MOLTO DI PIÙ DI UNA SEMPLICE SCHEDA!

È una **piattaforma completa** che permette a chiunque, anche ai principianti, di creare progetti elettronici e interattivi.

Con Arduino puoi trasformare le tue idee in qualcosa di reale, in pochi passi!

Arduino è formato da **tre elementi fondamentali**, che lavorano insieme come una squadra

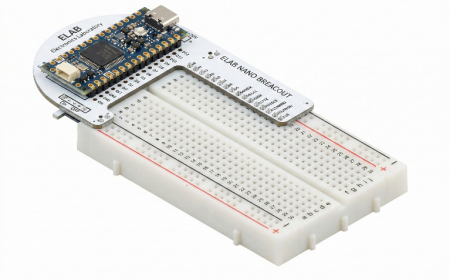
- l'**hardware** → la scheda e i componenti che userai
- l'**ambiente di sviluppo** → il programma sul PC dove scrivi il codice
- la **comunità di sviluppatori** → persone in tutto il mondo che ogni giorno condividono idee, progetti e aiuti

Grazie a questa comunità, puoi trovare tantissimo materiale per imparare e creare cose sempre nuove!

Con Arduino, ogni idea può diventare realtà.

L'HARDWARE DA PROGRAMMARE

Come abbiamo già anticipato nel capitolo precedente, all'interno di questo kit è inclusa una scheda Arduino Nano R4, che rappresenta il cuore di tutti gli esperimenti che realizzeremo. Per rendere l'utilizzo ancora più pratico e immediato, noi del team ELAB abbiamo progettato una schedina di supporto chiamata ELAB Nano Breakout. Questa piccola scheda aggiuntiva è stata pensata appositamente per facilitare il collegamento della Nano alla breadboard, semplificando notevolmente il cablaggio. In questo modo si riduce il numero di fili necessari per iniziare a lavorare, si evitano errori di collegamento e si mantiene il circuito più ordinato e leggibile — un aspetto fondamentale quando si muovono i primi passi nella sperimentazione elettronica.



La Nano è il cervello, e la schedina ELAB ti aiuta a usarla senza confusione.

L'AMBIENTE DI SVILUPPO (ARDUINO IDE)

Per poter utilizzare Arduino in modo efficace non basta avere la scheda:

è necessario anche uno strumento che ci permetta di scrivere il programma, modificarlo quando serve e trasferirlo nella memoria della scheda stessa.

In altre parole, abbiamo bisogno di un ambiente di lavoro digitale dove sviluppare il nostro codice.

Questo ambiente si chiama IDE, acronimo di Integrated Development Environment (Ambiente di Sviluppo Integrato).

L'IDE è un software che mette a disposizione, in un'unica interfaccia, tutti gli strumenti necessari per programmare: un editor di testo per scrivere il codice, funzioni per verificarne eventuali errori, e il sistema per caricarlo direttamente su Arduino tramite il collegamento USB.

Grazie all'IDE possiamo quindi passare dall'idea al programma funzionante in pochi passaggi: scriviamo il codice, lo controlliamo, lo carichiamo sulla scheda e osserviamo il risultato del nostro lavoro. È il "laboratorio virtuale" in cui prende forma il comportamento dei nostri progetti elettronici.

L'IDE è il posto dove nascono e prendono vita tutti i nostri progetti.

LA COMMUNITY: IL VERO CUORE DI ARDUINO

Arduino è una piattaforma Open Source.

Significa che:

- i progetti,
- la documentazione,
- i file delle schede,
- l'ambiente di sviluppo

sono pubblici e accessibili a tutti.

Nessun brevetto segreto, nessun blocco artificiale: chiunque può studiare, migliorare e condividere.

Questa scelta ha creato nel tempo una comunità (community) gigantesca di appassionati, studenti, insegnanti, inventori, sviluppatori professionisti.

INSTALLIAMO ARDUINO IDE

Ora che sappiamo quanto è importante Arduino IDE, è il momento di fare il primo vero passo:

Ti si aprirà una pagina del sito ufficiale di Arduino.

⚠ Attenzione: potrebbe non essere identica alle immagini del libro.

Perché?
Il sito cambia spesso:

- aggiornano le versioni
- modificano le grafiche
- mettono nuovi banner

È tutto normale!

Anche il programma Arduino IDE viene aggiornato spesso.
Questo significa che la versione che troverai potrebbe essere diversa da quella del libro.

Quando è stato scritto questo testo, l'ultima versione era la **2.3.7**, ma tu potresti trovare una versione più nuova… ed è ancora meglio!

Ora guarda bene la pagina: troverai il pulsante per scaricare il programma.

Accanto al download vedrai indicato il tuo sistema operativo:

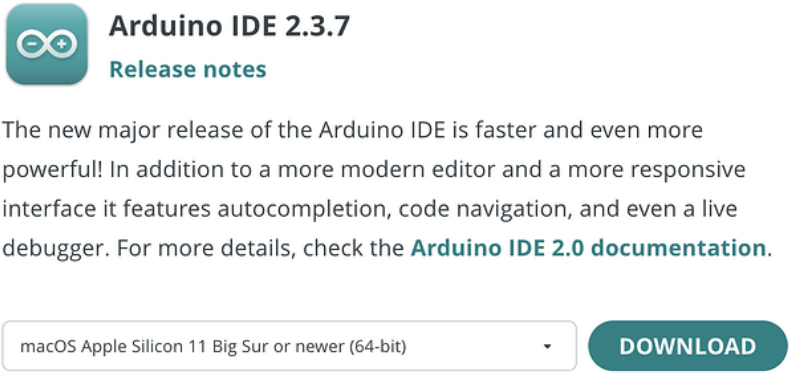
- **Windows**
- **Mac**
- **Linux**

Devi scegliere quello giusto per il tuo computer.

E a questo punto sei pronto per iniziare davvero… stai per installare lo strumento con cui darai vita ai tuoi progetti!

A questo punto siamo pronti!

Possiamo cliccare sul pulsante **DOWNLOAD** per iniziare a scaricare il programma sul nostro computer.



Una volta finito il download, troverai un file nel tuo PC.
Cliccaci sopra per avviare l'installazione.

Si aprirà una finestra con la procedura guidata.
Non preoccuparti, è molto semplice!

Ti basterà:

- cliccare su **ACCETTO**
- poi premere **Avanti**
- continuare così fino alla fine

In pochi passaggi, Arduino IDE sarà installato e pronto all'uso!

Lasciando la spunta su Esegui Arduino IDE e cliccando su chiudi, avvieremo quindi l'IDE!

Al primo avvio l'IDE installerà alcuni file necessari al suo funzionamento. Non preoccuparti è tutto normale!

INSTALLIAMO ARDUINO NANO R4

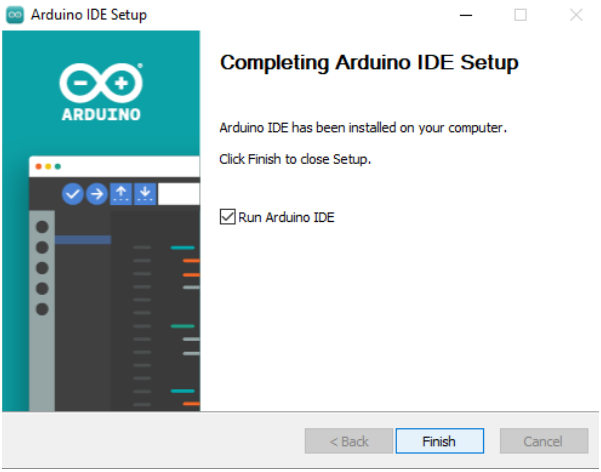
Arduino è molto potente, ma c'è una cosa importante da sapere.

Esistono **tantissimi tipi di schede Arduino**, non solo quella che stai usando tu.

Per questo motivo, l'IDE non le conosce tutte subito.

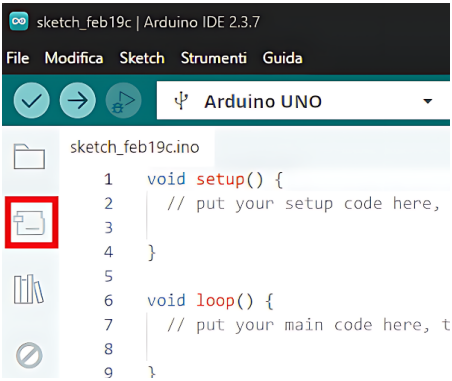
Quindi cosa dobbiamo fare?
Dobbiamo "insegnargli" a riconoscere la nostra scheda!

Per farlo si usa uno strumento speciale dentro l'IDE:
si chiama **Boards Manager**



Il Boards Manager è come un "negozio" interno al programma, dove possiamo trovare e installare le schede che ci servono.

Per aprirlo, basta cliccare sull'icona indicata nella figura.



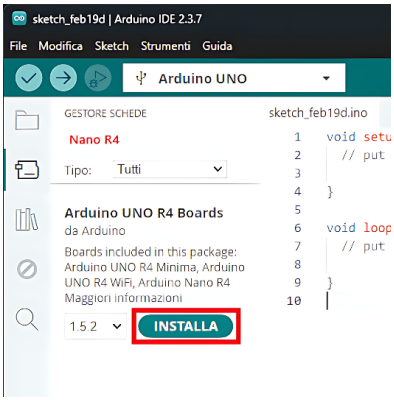
Una volta aperto, potremo cercare la nostra scheda e installarla in pochi clic.

Una volta aperto il Boards Manager, vedrai un menu con una **barra di ricerca.**

Scrivi dentro: **Nano R4**

A questo punto succede una cosa utile, la lista si riduce e dovrebbe comparire **un solo risultato**, chiamato: **Arduino UNO R4 Boards.**

Ora è semplicissimo: clicca sul pulsante **Installa**



In pochi secondi, l'IDE sarà pronto a riconoscere la tua scheda!

Da questo momento, Arduino saprà esattamente come "parlare" con la tua Nano R4.

Dopo aver cliccato su **Installa**, partirà automaticamente il processo di installazione.

Durante questo passaggio, il computer potrebbe chiederti di installare alcuni **driver**.
Non preoccuparti: è tutto normale!

I driver servono per far comunicare correttamente il computer con Arduino.

Quindi:

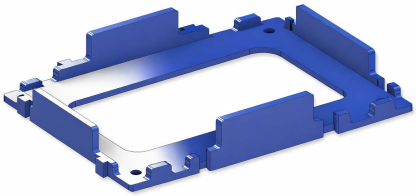
- clicca su **Accetta**
- aspetta qualche istante

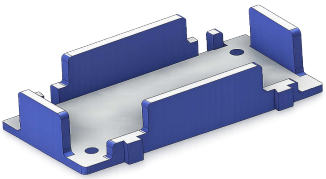
Una volta completata l'installazione…

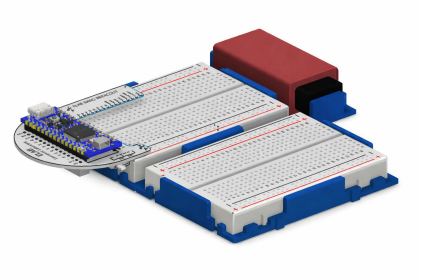
Sei davvero pronto!

Le Cover di Elab

Nel kit troverai due supporti in plastica con delle particolari “code di rondine”.
Questi supporti sono stati progettati per contenere la breadboard in modo stabile e ordinato.
Non servono solo a tenere fermo il circuito: sono stati pensati per aiutarti a gestire meglio i tuoi progetti.
Grazie a questi supporti, infatti, puoi tenere la pila sempre vicina alla breadboard, evitando cavi disordinati e rendendo il circuito più compatto.
Inoltre, utilizzando lo switch presente sulla breakout, puoi spegnere facilmente il circuito mettendolo su OFF.
In questo modo eviti di scaricare la pila senza dover ogni volta staccare i fili.
Se invece stai utilizzando il kit ELAB in laboratorio a scuola, questi supporti hanno un altro vantaggio molto interessante:
puoi collegare il tuo progetto con quello dei tuoi compagni, unendo più moduli insieme per creare progetti sempre più grandi e complessi.







NOTE

IL NOSTRO PRIMO PROGRAMMA!

Il programma più famoso: Il Blink

Siamo arrivati a un momento importante… il nostro **primo vero programma Arduino!**

Si chiama **Blink**, che in inglese significa “lampeggiare”.

È il programma più semplice e famoso, perché fa una cosa molto chiara: **accende e spegne un LED continuamente.**

Nota del capitolo

Accanto a ogni sketch che troverai nei prossimi capitoli vedrai un pulsante con l'icona di Download.

Se vai sul sito di ELAB

www.elab.school

troverai l'elenco degli sketch di ogni capitolo, organizzati per capitolo e immagine,

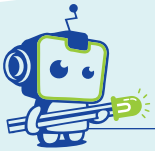
ad esempio:

Sketch_Capitolo_5_1.

Da qui potrai scaricare automaticamente tutti i programmi.

⚠ Ma attenzione!
Prima di scaricare il programma, prova sempre a scriverlo da solo.

Se proprio non riesci a trovare l'errore, allora puoi aiutarti utilizzando gli sketch già pronti.

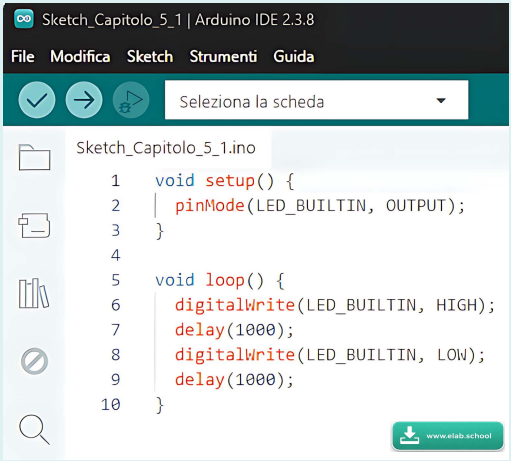


ESERCIZIO 5.1

ADESSO TOCCA A TE!

Apri **Arduino IDE** sul tuo computer e scrivi questo programma:

```
void setup() {
  pinMode(LED_BUILTIN, OUTPUT);
}

void loop() {
  digitalWrite(LED_BUILTIN, HIGH);
  delay(1000);
  digitalWrite(LED_BUILTIN, LOW);
  delay(1000);
}
```



Ma prima di caricarlo sulla scheda… fermiamoci un attimo!

Analizziamo il programma **riga per riga**, così capiamo davvero cosa stiamo scrivendo.

In questo modo non copierai solo il codice, ma saprai anche **perché funziona.**

LA STRUTTURA DI UN PROGRAMMA ARDUINO

Prima di guardare il programma Blink, dobbiamo capire una cosa molto importante.

Abbiamo detto che ogni programma è scritto in un **linguaggio di programmazione**.

E proprio come nelle lingue che parliamo, anche qui ci sono delle **regole precise**:

- di scrittura
- di struttura
- di forma

Se non le rispettiamo… il programma non funziona!

Una delle prime regole riguarda **come è organizzato il programma**.

In Arduino, ogni programma viene diviso in **due parti principali**.

Queste parti si chiamano **funzioni**.

Sono come due "sezioni" del programma, ognuna con un compito diverso.

E si chiamano:

- **setup()**
- **loop()**

Nei prossimi passaggi vedremo cosa fanno… e perché sono così importanti per far funzionare tutto!

LE DUE PARTI FONDAMENTALI DEL PROGRAMMA

Ora vediamo meglio queste due funzioni così importanti.

La prima si chiama **setup()**.
Questa parte viene eseguita **una sola volta**, quando Arduino si accende o viene riavviato.

Qui prepariamo tutto quello che serve, per esempio:

- diciamo quali pin usare
- impostiamo i componenti

È un po' come preparare gli ingredienti prima di iniziare a cucinare.

La seconda si chiama **loop()**.

Questo è il **vero cuore del programma**.

Tutto quello che scriviamo qui viene eseguito **all'infinito.**

Arduino fa così:

- esegue le istruzioni
- arriva alla fine
- e poi ricomincia da capo

Per esempio:
esegue riga 7, 8, 9, 10…
e poi torna di nuovo alla riga 7

E continua così **per sempre.**

setup() prepara tutto (una volta sola)
loop() ripete le azioni all'infinito

CAPIRE LA FUNZIONE SETUP()

Ora entriamo un po' più dentro il codice… niente paura, andiamo con calma.

Quando vedi scritto:

```
void setup() {
```

stai guardando l'inizio di una funzione.

Partiamo dalla prima parola: **void**

In inglese significa "vuoto".
Vuol dire che questa funzione **non restituisce nulla**.

Non preoccuparti troppo di questo adesso: lo capiremo meglio più avanti

Poi troviamo **setup**

È semplicemente il **nome della funzione.**

Subito dopo ci sono le parentesi: **( )**

Queste servono per passare delle informazioni alla funzione
(si chiamano parametri)

Ma in questo caso: le parentesi sono vuote perché **setup() non ha bisogno di ricevere nulla.**

Infine troviamo questa parentesi graffa: **{**

Questa è molto importante!

Serve per dire:
"Da qui inizia il codice della funzione"

È come aprire una scatola,
tutto quello che scriveremo dopo fino alla parentesi di chiusura, appartiene a **setup().**

In poche parole:

- **void** → non restituisce niente
- **setup** → nome della funzione
- **( )** → nessun parametro
- **{** → inizio del codice

E adesso siamo pronti a vedere cosa c'è dentro questa funzione!

Quando la funzione finisce, troviamo la parentesi che si chiude: **}**

Chiudere la parentesi è un po' come mettere un **punto alla fine di una frase**.
Significa: "ok, qui la funzione è finita".

Se vuoi scrivere queste parentesi sulla tastiera del PC:

- **{** → premi *AltGr + Maiusc + è*
- **}** → premi *AltGr + Maiusc + **

Devi premere i tasti insieme.

ORA VEDIAMO UNA RIGA MOLTO IMPORTANTE

pinMode(LED_BUILTIN, OUTPUT);

Questa è una delle istruzioni base di Arduino.

pinMode è una funzione, cioè un comando che dice ad Arduino come usare un pin.

Dentro le parentesi ci sono **due informazioni (parametri)**:

- **LED_BUILTIN** → indica il pin che vogliamo usare
- **OUTPUT** → indica come vogliamo usarlo

I **pin** sono i punti della scheda (ai lati) che servono per collegare e controllare i componenti.

Il pin **LED_BUILTIN** è speciale perché è già collegato a un piccolo LED presente sulla scheda.

Inoltre, invece di scrivere il numero del pin (che cambia da scheda a scheda), usiamo questo nome così il programma funziona ovunque.

Ma cosa significa OUTPUT?

Significa che quel pin **manda un segnale in uscita**.

Nel nostro caso: Arduino invia un segnale al LED per accenderlo o spegnerlo.

Quindi, questa riga sta dicendo: "Arduino, usa questo pin per controllare qualcosa e fai uscire un segnale da lì."

pinMode serve per scegliere un pin e dire ad Arduino come deve usarlo.

IL PUNTO E VIRGOLA E L'ORDINE DEL CODICE

C'è un'altra regola molto importante nella programmazione:

Alla fine di ogni istruzione bisogna mettere il **;**

Questo simbolo serve a dire: "questa istruzione è finita"

È un po' come il **punto** nelle frasi in italiano.

Nel nostro caso, abbiamo scritto:

pinMode(LED_BUILTIN, OUTPUT);

Il **;** alla fine indica che abbiamo completato l'istruzione.

Nel **setup()** abbiamo solo questa riga. Quindi:

- abbiamo impostato il pin
- abbiamo chiuso l'istruzione con il **;**

A questo punto non dobbiamo fare altro... e possiamo chiudere la funzione con la parentesi **}**

E pensa un po': abbiamo spiegato tutto questo... e siamo solo alla riga 3!

Subito dopo troviamo una riga vuota.

Possiamo lasciare tutti gli spazi che vogliamo nel codice.

Servono per:

- tenere tutto più ordinato
- rendere il codice più facile da leggere

In poche parole:

- **;** → fine istruzione
- **}** → fine funzione
- spazi → aiutano a leggere meglio il codice

ENTRIAMO NEL LOOP

Ora troviamo questa riga:

void loop() {

È molto simile a quella del setup, e infatti funziona nello stesso modo.

- **void** significa che non restituisce nulla
- **loop** è il nome della funzione
- **( )** è vuoto perché non servono parametri
- **{** apre il "contenitore" del codice

Ma c'è una differenza importantissima:

tutto quello che scriviamo nel **loop()** viene eseguito **all'infinito**!

Dentro questa funzione useremo altre due istruzioni base di Arduino.

Se guardi il programma:

- la riga **9 e 12 sono molto simili**
- la riga **10 e 13 sono identiche**

Questo non è un caso!

Serve per creare un effetto che si ripete... proprio come nel Blink, dove il LED si accende e si spegne continuamente.

Il loop() è la parte del programma che si ripete per sempre ed è qui che succede tutta la "magia".

LA FUNZIONE DELAY

La parola **delay** in inglese significa "ritardo".

In questo caso vuol dire: "aspetta un po' prima di fare la prossima cosa"

Dentro le parentesi troviamo il numero **1000**.

Questo numero indica **quanto tempo aspettare**, ma attenzione: è espresso in **millisecondi (ms)**.

1000 millisecondi = 1 secondo

Quindi questa istruzione significa: "aspetta 1 secondo prima di continuare".

delay(1000) mette in pausa Arduino per 1 secondo. Ed è proprio grazie a questa pausa che il LED del Blink riesce a lampeggiare invece di restare sempre acceso!

LA FUNZIONE DIGITALWRITE

Ora passiamo a una funzione un po' più interessante:

```
digitalWrite(LED_BUILTIN, HIGH);
```

La parola **digitalWrite** significa "scrivere in modo digitale".

Ma cosa vuol dire davvero?

- **scrivere** → impostare lo stato di un pin
- **digitale** → può avere solo due valori

Quali valori?

- **HIGH** → alto (acceso)
- **LOW** → basso (spento)

Quando scriviamo:

digitalWrite(LED_BUILTIN, HIGH);

stiamo dicendo ad Arduino:
"metti questo pin alto"

In pratica: **accendi il LED**

Se invece scriviamo:

digitalWrite(LED_BUILTIN, LOW);

stiamo dicendo:
"metti questo pin basso"

quindi: **spegni il LED**

Ora unisci tutto:

- accendi il LED
- aspetta 1 secondo
- spegni il LED
- aspetta 1 secondo

E siccome queste istruzioni sono dentro il **loop()**, Arduino le ripete **all'infinito.**

Risultato: il LED si accende e si spegne ogni secondo… cioè **lampeggia!**

IL NOSTRO PRIMO PROGRAMMA: BLINK

Abbiamo appena analizzato il nostro primo programma Arduino!

Questo programma si chiama **Blink** perché il LED sulla scheda inizia a **lampeggiare ogni secondo**.

È un po' come se Arduino ci facesse l'occhiolino.

Infatti, in inglese **"blink"** significa proprio **battere le palpebre**!

Anche se il programma sembra molto semplice, in realtà dentro c'è una logica abbastanza complessa.

Arduino sta:

- seguendo istruzioni precise
- controllando il microcontrollore
- eseguendo operazioni in modo continuo

Ma noi non vediamo tutta questa complessità!

Ed è proprio questo il bello di Arduino.

Tutta la parte difficile è **nascosta**, così possiamo concentrarci sulle idee e imparare in modo semplice e divertente.

Abbiamo già installato la Arduino Nano R4 nel nostro IDE nel capitolo precedente.Questo rende tutto molto più semplice! Innanzitutto se non lo hai già fatto lancia sul tuo computer l'IDE di Arduino.

Anche se ti consigliamo di scrivere il Blink per prendere confidenza con la punteggiatura e il linguaggio, questo programma può essere trovato sotto gli esempi. Per farlo clicca su

File->Esempi->Basics->Blink

I COMMENTI NEL CODICE

Quando aprirai il programma, noterai subito una cosa.

Il codice è un po' più lungo di quello che abbiamo visto finora.

Questo perché ci sono tante frasi (in inglese) che spiegano:

- chi ha scritto il programma
- cosa fanno le varie istruzioni

Queste frasi si chiamano **commenti.**

I commenti servono per: spiegare il codice a chi lo legge.

Sono molto importanti perché aiutano a capire meglio cosa sta succedendo, soprattutto quando i programmi diventano più lunghi e complessi.

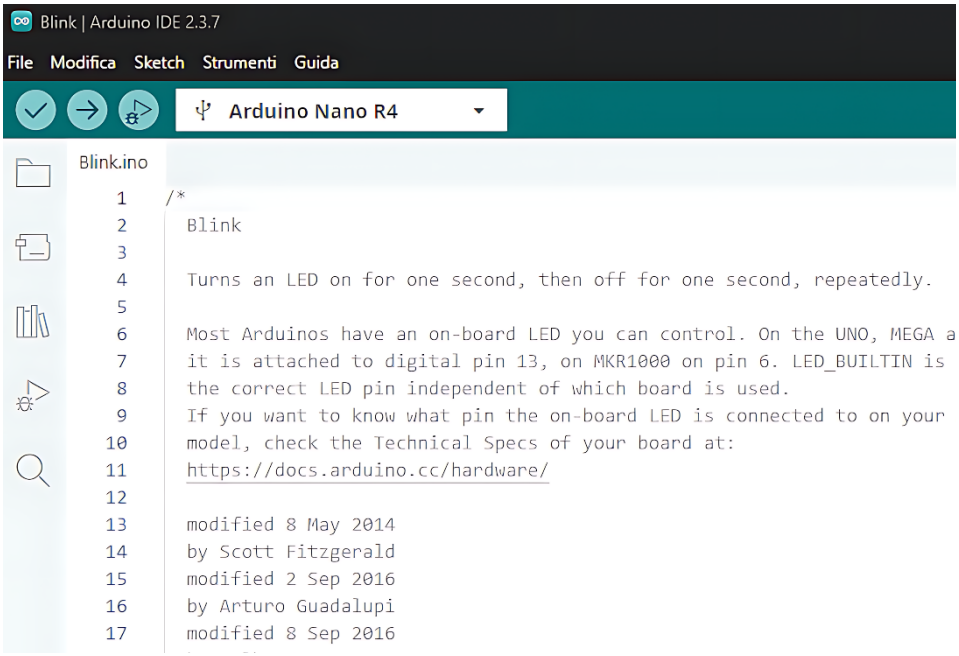
Nel linguaggio Arduino possiamo scrivere commenti in due modi.

Se vogliamo scrivere un commento su **più righe**, usiamo:

/* per iniziare
*/ per chiudere

Per esempio:

```
/*
Questo è un
commento su più righe

*/
```



A volte, mentre scriviamo un programma, abbiamo bisogno di lasciare delle **spiegazioni** per noi o per chi lo leggerà.

Quando vogliamo scrivere un commento **sulla stessa riga** di un'istruzione, usiamo:

//

Dopo questi due simboli, tutto quello che scriviamo:

non viene eseguito dal computer serve solo come spiegazione

Per esempio:

digitalWrite(LED_BUILTIN, HIGH); // accende il LED

I commenti servono per spiegare il codice, ma Arduino li **ignora completamente**

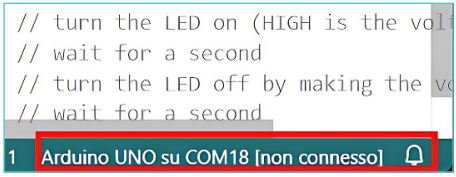
IMPOSTIAMO LA SCHEDA GIUSTA

Ora dobbiamo preparare l'IDE per caricare il programma sulla nostra Arduino.

La prima cosa da fare è scegliere la **scheda corretta**.

Se è la prima volta che usi Arduino IDE, probabilmente vedrai selezionata una scheda diversa:
Arduino UNO

Questa informazione si trova in basso a destra,
accanto al nome della porta del computer.



Ma attenzione!

Noi stiamo usando una **Arduino Nano R4**, quindi dobbiamo cambiare questa impostazione.

Per farlo segui questo percorso:
Strumenti → Scheda → Arduino UNO R4 Boards → Arduino Nano R4

Una volta fatto, guarda in basso a destra nell'IDE .

Non dovrebbe più esserci scritto "Arduino UNO".

Se tutto è corretto, vedrai il nome della scheda giusta:

Arduino Nano R4

COLLEGHIAMO ARDUINO AL COMPUTER

Ora puoi collegare la tua **Arduino Nano R4** al computer usando il cavo **USB-C**.

Appena la colleghi:

- si accenderà un **LED verde** sulla scheda (significa che è alimentata)
- il computer potrebbe fare un suono

Adesso dobbiamo fare un altro passaggio importante:
scegliere la **porta giusta.**

Ogni dispositivo collegato al computer usa una porta per comunicare.
Questa porta si chiama **porta COM**.

Per selezionarla:
vai su **Strumenti → Porta**

Vedrai una lista di porte.

Devi scegliere quella che ha scritto tra parentesi:
Arduino Nano R4

Per esempio:

- COM12 (Arduino Nano R4)

Dopo averla selezionata, guarda in basso a destra.

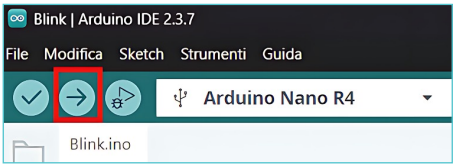
Dovresti vedere scritto qualcosa come:
Arduino Nano R4 su COM12 e la scritta **[non connesso]** scomparirà.

CARICHIAMO IL PROGRAMMA!

Adesso ci siamo davvero!

Possiamo finalmente caricare il nostro programma sulla scheda Arduino.

Per farlo basta cliccare sul pulsante **"Carica"** (quello con la freccia verso destra).



COSA SUCCEDE QUANDO CARICHIAMO IL PROGRAMMA

Quando clicchi su **Carica**, l'IDE fa due operazioni importanti:

- **Compilazione**
- **Upload**

La compilazione:

La compilazione è come un **controllo ortografico** del codice.

L'IDE controlla se hai scritto tutto correttamente.

Se c'è anche solo un piccolo errore: il programma non parte e l'IDE ti avvisa che qualcosa non va.

Proviamo a fare un errore apposta:

Togli il **;** alla fine di una riga e clicca di nuovo su **Carica**.

Cosa succede?

L'IDE ti mostra un **errore di compilazione.**

- ti dirà (in inglese) che manca un **;**
- la riga con l'errore diventerà **rossa**

Questo è molto utile, perché:
l'IDE ti aiuta a capire **dove hai sbagliato.**

L'UPLOAD E I PRIMI ESPERIMENTI

Dopo la compilazione, arriva la seconda fase:
l'**upload**

Questa è la parte in cui il programma viene **trasferito davvero sulla scheda Arduino**.

Se tutto va bene, l'IDE ti mostrerà un messaggio di successo.

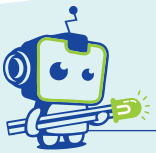
Questo significa che Arduino ha ricevuto il programma ed è pronto a eseguirlo!

Ora che sai come caricare un programma…
è il momento di divertirsi un po'!

Prova a modificare il Blink

Guarda questa riga:

```
delay(1000);
```



ESERCIZIO 5.2

Prova a cambiare il numero dentro le parentesi

Per esempio:

- **500** → il LED lampeggia più veloce
- **2000** → lampeggia più lento

Fai qualche prova:

- usa due tempi uguali (es. 500 e 500)
- poi prova tempi diversi (es. 300 e 200)

Vedrai che il lampeggio cambia!

NOTE

IL MONDO DISCRETO: I PIN DIGITALI

Nel capitolo precedente abbiamo visto il nostro primo programma Arduino: il **Blink!**

In questo programma abbiamo usato un'istruzione molto importante: **digitalWrite**

Questa parola significa, letteralmente, **"scrivere in modo digitale"**.

Lo abbiamo già accennato, ma adesso approfondiamo meglio cosa vuol dire **digitale.**

Quando qualcosa è digitale, può avere solo **due stati possibili**:

- acceso o spento
- alto o basso
- 1 oppure 0

Non esistono valori "a metà"!

È un po' come un interruttore della luce:

- o è acceso o spento
- o è vero o falso
- o è alto o basso

In poche parole:
digitale significa scegliere tra **due sole possibilità.**

Ecco perché, quando usiamo **digitalWrite**, dobbiamo scegliere tra due valori:

- **HIGH** oppure **LOW**
- **HIGH** significa **alto**
- **LOW** significa **basso**

Ma cosa vuol dire davvero?

Nel caso del LED:

- **HIGH** → il pin manda corrente il LED si **accende**
- **LOW** → il pin non manda corrente il LED si **spegne**

LOGICA DIGITALE: AND E OR

Quando si parla di programmazione, si parla spesso anche di **logica digitale**.

In pratica: come fa un computer a prendere decisioni?

Usando alcune operazioni molto semplici, tra cui le più importanti sono:

- **AND (e)**
- **OR (oppure)**

AND (insieme)

La **AND** è vera solo quando **tutte le condizioni sono vere**.

Esempio:
un LED si accende **solo se premi entrambi i pulsanti**

- premi un solo pulsante → non succede niente
- premi tutti e due → il LED si accende

OR (oppure)

La **OR** è vera quando **almeno una condizione è vera**.

Esempio:
un LED si accende se premi **uno qualsiasi dei due pulsanti**

- premi uno dei due → il LED si accende
- premi entrambi → il LED si accende comunque

Nella programmazione si scrivono così:

- **AND** → &&
- **OR** → ||

In poche parole:

- **AND** → devono essere vere **tutte** le condizioni
- **OR** → basta che sia vera **anche solo una**

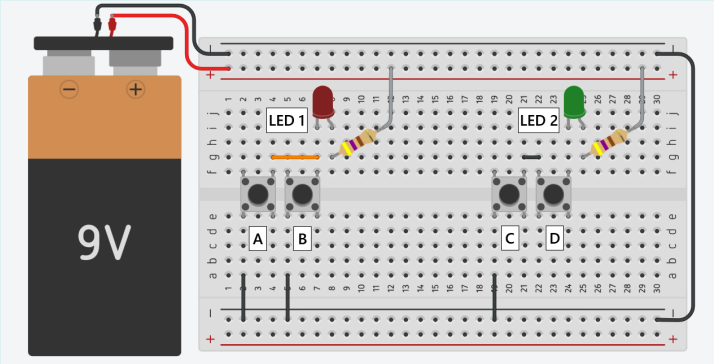
E con queste semplici regole puoi già iniziare a far "ragionare" i tuoi progetti!

ESERCIZIO 6.1

PROVIAMO IL CIRCUITO AND E OR

Adesso passiamo alla pratica!

Prova a realizzare il circuito mostrato in figura.



In realtà non è uno solo, ma sono **due circuiti diversi** costruiti sulla stessa breadboard.

A sinistra (OR)

Qui i due pulsanti sono collegati in modo che basta premere **uno qualsiasi dei due** e il LED si accende

Non importa quale premi:

- primo pulsante
- secondo pulsante

il risultato è sempre lo stesso: LED acceso

A destra (AND)

In questo caso la situazione cambia.

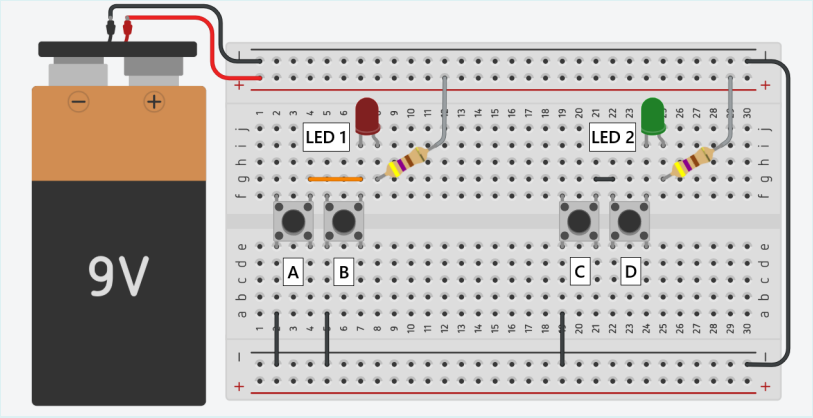
Il LED si accende **solo se premi entrambi i pulsanti insieme**

- premi uno solo → niente
- premi tutti e due → LED acceso

In poche parole:

- circuito a sinistra → **OR (uno basta)**
- circuito a destra → **AND (servono tutti)**

E così inizi a vedere come la logica della programmazione funziona anche nei circuiti reali!



Prova ora a montare il circuito per verificare quello che abbiamo appena spiegato.

Per capirlo ancora meglio, diamo un nome ai pulsanti:

- A e B per il circuito OR
- C e D per il circuito AND

A questo punto possiamo costruire quella che si chiama **tabella di verità**.

La tabella di verità serve per capire in quali casi il circuito "funziona", cioè quando il LED si accende.

In pratica, mostra tutte le possibili combinazioni degli ingressi (i pulsanti) e il risultato finale (il LED).

Abbiamo detto che:

- **VERO** significa **acceso**
- e acceso corrisponde al valore **1**

Di conseguenza:

- **FALSO** significa **spento**
- e spento corrisponde a **0**

Per questo motivo, nella tabella di verità useremo solo **0 e 1**

In questo modo possiamo descrivere il comportamento del circuito in modo chiaro e preciso, proprio come fa un computer.

Per quanto riguarda la **OR**, possiamo costruire la seguente tabella di verità:

A	B	LED (uscita)
0	0	0
0	1	1
1	0	1
1	1	1

Come vedi:
il LED si accende (1) ogni volta che **almeno uno dei due pulsanti è premuto**.

Solo nel caso in cui **entrambi i pulsanti sono rilasciati (0 e 0)** il LED rimane spento.

Questo è proprio il comportamento della logica **OR**.

Ricordiamo che:

- per i **pulsanti**, 1 significa **premuto**
- per il **LED**, 1 significa **acceso**

Per quanto riguarda la **AND**, la tabella di verità è la seguente:

C	D	LED (uscita)
0	0	0
0	1	0
1	0	0
1	1	1

Come vedi:
il LED si accende solo quando **entrambi i pulsanti sono premuti** (1 e 1).

In tutti gli altri casi, anche se uno solo è premuto, il LED rimane spento.

Questo è il comportamento della logica **AND**.

Quello che abbiamo scritto nella tabella dovrebbe corrispondere esattamente a quello che osservi nel circuito.

Nel caso della **OR**, basta premere uno dei due pulsanti per accendere il LED.

Nel caso della **AND**, invece, il LED si accende solo quando premi entrambi i pulsanti contemporaneamente.

Se provi il circuito, vedrai che il comportamento è proprio questo.

A questo punto il concetto di **digitale** dovrebbe essere più chiaro.

Ora possiamo continuare e vedere come usare queste idee anche nella programmazione.

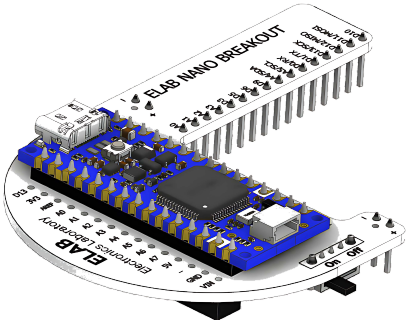
ALIMENTAZIONE E COLLEGAMENTI DI ARDUINO NANO R4

La **Arduino Nano R4** che useremo nei nostri esperimenti funziona con una tensione di **5 Volt (5V)**.

Possiamo alimentarla in due modi:

- collegandola al computer tramite il cavo **USB**
- oppure usando una **batteria da 9V**

Per realizzare i nostri circuiti, useremo anche la **ELAB Nano Breakout** di cui abbiamo parlato prima.



Questa schedina ci aiuta a:

- organizzare meglio i collegamenti
- evitare errori mentre montiamo il circuito
- lavorare in modo più sicuro

In questo modo sarà più facile costruire i nostri esperimenti e capire cosa stiamo facendo.

Grazie alla levetta con l'indicazione **ON/OFF**, possiamo controllare facilmente l'alimentazione del circuito.

- Quando la mettiamo su **OFF**, interrompiamo la corrente
- Quando la mettiamo su **ON**, il circuito torna a funzionare

Questo è molto utile perché ci permette di **non consumare la batteria inutilmente** quando non stiamo usando il circuito.

Abbiamo detto che la **Arduino Nano R4** funziona a **5V**.

Questo significa due cose molto importanti:

- sui suoi pin non uscirà mai una tensione maggiore di 5V
- non dobbiamo mai fornire ai pin una tensione superiore a 5V

Se lo facessimo, rischieremmo di **danneggiare la scheda**.

Come puoi vedere, la **ELAB Nano Breakout** si inserisce nelle strisce di alimentazione della breadboard (quelle sopra e sotto).

In questo modo porta molti pin della Arduino Nano R4 (come A0, D2, ecc.) direttamente sulla prima riga della breadboard.

Questo serve a ridurre il numero di fili e rendere tutto più semplice e ordinato.

Il fatto che si colleghi anche alle strisce di alimentazione non è casuale.

Serve per lavorare in sicurezza.

La breakout prende i **5V generati da Arduino** e li distribuisce sulla breadboard.

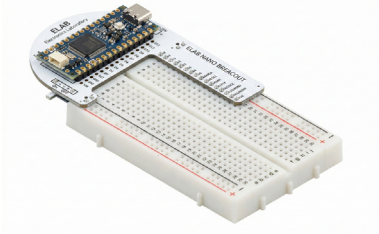
Quindi avremo sempre:

- sulla linea **nera** → 0V
- sulla linea **rossa** → 5V

In questo modo evitiamo di collegare per errore tensioni troppo alte che potrebbero danneggiare la scheda.

Puoi inserire la breakout sulla breadboard da entrambi i lati, ma fai attenzione a una cosa importante:

controlla sempre che la breadboard sia orientata correttamente, con i numeri che vanno in ordine crescente da **1 a 30**



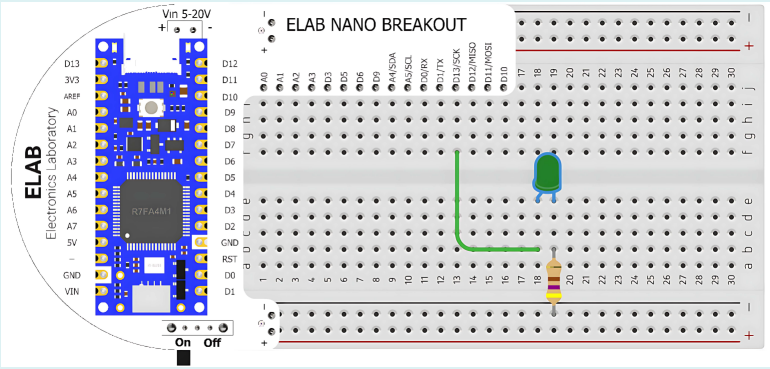
ESERCIZIO 6.2

ACCENDIAMO UN LED ESTERNO

In questo esperimento faremo un passo in più:
accenderemo un LED collegato a un pin della nostra Arduino Nano R4.

Materiale necessario:

- Arduino Nano R4
- ELAB Nano Breakout
- 1 LED verde
- 1 resistore da 470 Ohm
- 1 cavetto



Per iniziare:

inserisci la **Arduino Nano R4** nella breakout
poi inserisci tutto sulla breadboard, come mostrato in figura.

Da questo momento in poi userai sempre breadboard e breakout insieme, quindi puoi considerarle come un unico strumento.

Ora realizza i collegamenti:

collega il cavetto, il resistore e il LED seguendo la figura.

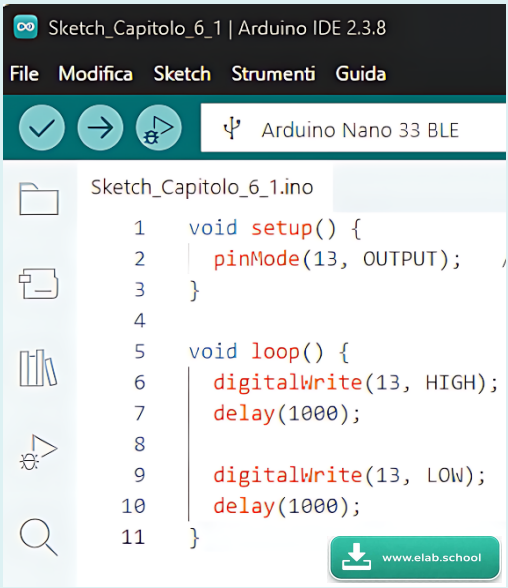
Ricorda una cosa molto importante:
il **resistore deve essere in serie al LED**.

Serve per evitare che il LED si bruci.

Puoi metterlo:

- sul lato positivo (gambetta lunga)
- oppure sul lato negativo (gambetta corta)

Non cambia: l'importante è che sia in serie al LED.

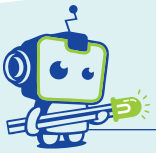


Per quanto riguarda il programma:

useremo lo stesso codice del **Blink**, ma con una piccola modifica

Al posto di:
LED_BUILTIN
scriveremo:
13
Perché?
Il LED che hai collegato è connesso al pin **D13**

stiamo usando lo stesso programma di prima, ma ora controlliamo un LED esterno invece di quello sulla scheda



ESERCIZIO 6.3

In questo esercizio, abbiamo utilizzato il pin 13 per accendere e spegnere il LED, ma avremmo potuto utilizzare qualunque altro pin disponibile sulla breakout.

Come puoi vedere, abbiamo a disposizione diverse alternative per collegare i nostri componenti.

Sulla **ELAB Nano Breakout** troviamo vari pin, divisi in:

- **pin analogici A0 A1 A2 A3 A4 A5**
- **pin digitali 3 5 6 9 10 11 12 13**

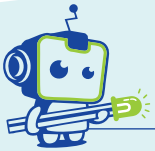
Accanto alla scheda Arduino Nano sono comunque riportati **tutti i pin completi**, così puoi sempre sapere quale stai utilizzando.

- **pin analogici A0 A1 A2 A3 A4 A5 A6 A7**
- **pin digitali 0 1 2 3 4 5 6 7 8 9 10 11 12 13**

Puoi modificare questo esercizio in modo semplice:

- scegli un altro pin tra quelli disponibili
- sposta il filo collegato all'anodo del LED su quel pin
- modifica il programma inserendo il numero del nuovo pin

In questo modo puoi sperimentare liberamente e prendere confidenza con i diversi pin della scheda.



ESERCIZIO 6.4

DUE LED CHE SI ALTERNANO

In questo esperimento faremo un passo avanti:
accenderemo **due LED**, collegati a due pin diversi della Arduino Nano R4.

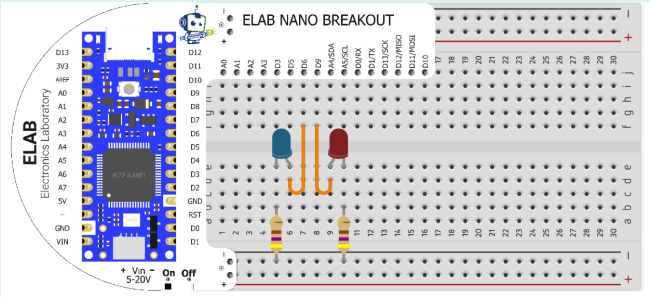
Materiale necessario:

- Arduino Nano R4
- ELAB Nano Breakout
- 1 LED blu
- 1 LED rosso
- 2 resistori da 470 Ohm
- 2 cavetti

Come nel caso precedente:
ogni LED deve avere il suo **resistore in serie**

- il **catodo** (gambetta corta) va verso il GND tramite il resistore
- l'**anodo** (gambetta lunga) va collegato a un pin di Arduino

Questa volta useremo **due pin diversi**, ad esempio:



pin **6** e pin **9**

Per quanto riguarda il programma, partiamo dal Blink e lo modifichiamo.

Sketch_Capitolo_6_2 | Arduino IDE 2.3.8

File Modifica Sketch Strumenti Guida

Arduino Nano 33 BLE

Sketch_Capitolo_6_2.ino

```
1   void setup() {
2     pinMode(6, OUTPUT); // Imposta il pin 6 come uscita (LED 1)
3     pinMode(9, OUTPUT); // Imposta il pin 9 come uscita (LED 2)
4   }
5
6   void loop() {
7     digitalWrite(6, HIGH); // Accende il LED collegato al pin 6
8     delay(200);            // Aspetta 200 millisecondi (0,2 secondi)
9
10    digitalWrite(6, LOW);  // Spegne il LED sul pin 6
11
12    digitalWrite(9, HIGH); // Accende il LED collegato al pin 9
13    delay(200);            // Aspetta 200 millisecondi
14
15    digitalWrite(9, LOW);  // Spegne il LED sul pin 9
16  }
```

www.elab.school

Nel **setup()** dobbiamo dire ad Arduino che useremo entrambi i pin come output:

```
pinMode(6, OUTPUT);
pinMode(9, OUTPUT);
```

⚠ Attenzione:
non possiamo scrivere tutto in una sola riga
(dobbiamo usare una riga per ogni pin)

Quando Arduino si avvia, i pin impostati come OUTPUT partono già spenti (LOW), quindi non dobbiamo scriverlo.

Nel **loop()** facciamo alternare i due LED:

1. accendiamo il LED sul pin 6
2. aspettiamo un po' (per esempio 200 millisecondi)
3. spegniamo il LED 6
4. accendiamo il LED sul pin 9
5. aspettiamo di nuovo
6. spegniamo il LED 9

Poi il ciclo ricomincia da capo, all'infinito.

i due LED si accendono uno dopo l'altro, creando un effetto alternato

ESERCIZIO 6.5

COSTRUIAMO UN SEMAFORO

In questo esperimento aggiungiamo un terzo LED per realizzare un semplice semaforo.

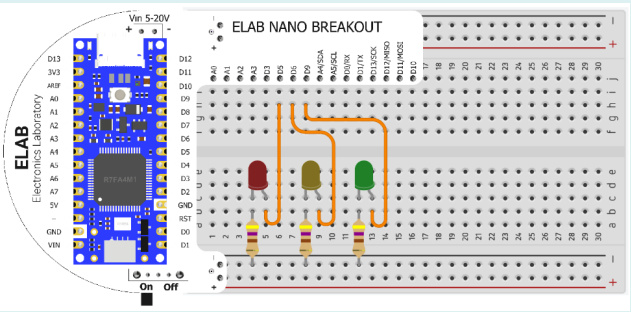
Per farlo abbiamo bisogno di:

- Arduino Nano R4
- ELAB Nano Breakout
- 1 LED rosso
- 1 LED giallo
- 1 LED verde
- 3 resistori da 470 Ohm
- 3 cavetti

Come nei casi precedenti, ogni LED deve essere collegato con un **resistore in serie**:

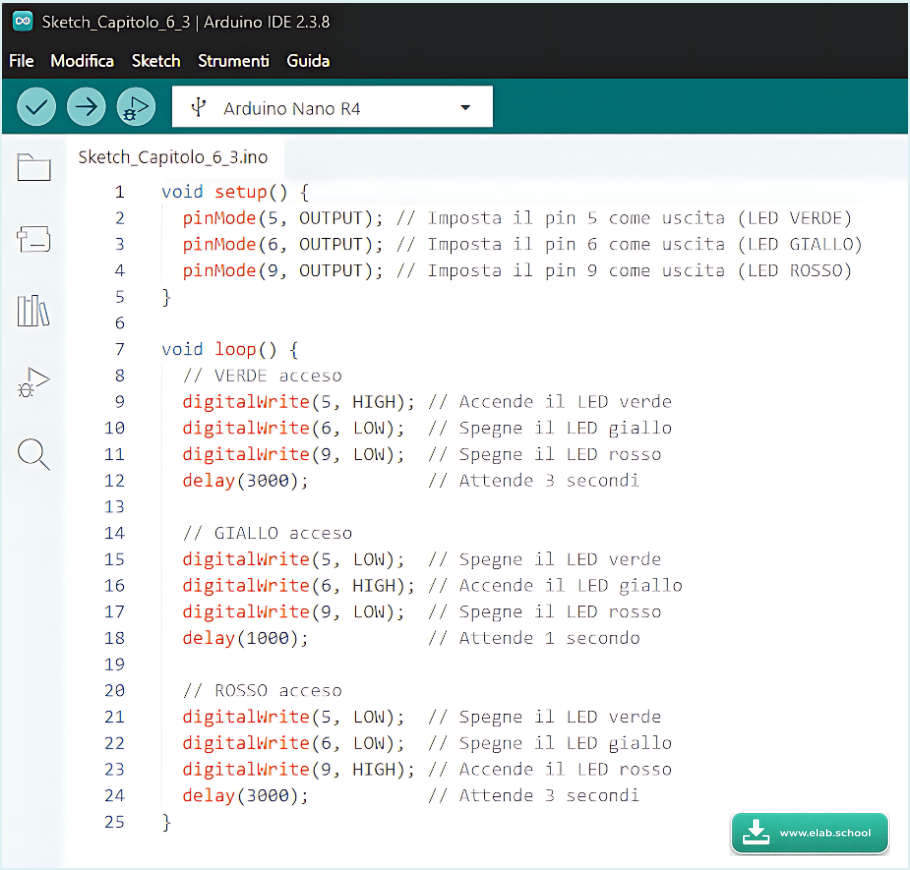
- il catodo (gambetta corta) va verso il GND tramite il resistore
- l'anodo (gambetta lunga) va a un pin di Arduino

Questa volta useremo **tre pin diversi**, ad esempio: 5, 6 e 9



Per il programma, possiamo partire dall'esercizio precedente con due LED.

Cosa cambia?




```
1  void setup() {
2    pinMode(5, OUTPUT); // Imposta il pin 5 come uscita (LED VERDE)
3    pinMode(6, OUTPUT); // Imposta il pin 6 come uscita (LED GIALLO)
4    pinMode(9, OUTPUT); // Imposta il pin 9 come uscita (LED ROSSO)
5  }
6
7  void loop() {
8    // VERDE acceso
9    digitalWrite(5, HIGH); // Accende il LED verde
10   digitalWrite(6, LOW);  // Spegne il LED giallo
11   digitalWrite(9, LOW);  // Spegne il LED rosso
12   delay(3000);           // Attende 3 secondi
13
14   // GIALLO acceso
15   digitalWrite(5, LOW);  // Spegne il LED verde
16   digitalWrite(6, HIGH); // Accende il LED giallo
17   digitalWrite(9, LOW);  // Spegne il LED rosso
18   delay(1000);           // Attende 1 secondo
19
20   // ROSSO acceso
21   digitalWrite(5, LOW);  // Spegne il LED verde
22   digitalWrite(6, LOW);  // Spegne il LED giallo
23   digitalWrite(9, HIGH); // Accende il LED rosso
24   delay(3000);           // Attende 3 secondi
25 }
```

Dobbiamo solo aggiungere il nuovo pin.

Prima usavamo:

- pin 6
- pin 9

Adesso aggiungiamo anche:

- pin 5

Quindi nel **setup()** dovremo inizializzare anche questo nuovo pin come OUTPUT.

A questo punto possiamo creare il comportamento del semaforo:

- accendiamo un LED
- aspettiamo un po'
- lo spegniamo e accendiamo il successivo
- e così via, in ciclo continuo

Prova anche a modificare i tempi del **delay()**.

Ricorda:

- il numero dentro delay indica i **millisecondi**
- 1000 ms = 1 secondo

DAGLI OUTPUT AGLI INPUT

Fino ad ora abbiamo parlato dei pin digitali usandoli come OUTPUT, cioè per controllare qualcosa, come accendere o spegnere un LED.

Ma nel mondo reale non esistono solo le uscite.

Esistono anche gli **ingressi**, chiamati **INPUT**.

Gli input servono per **ricevere informazioni dall'esterno**, mentre gli output servono per **agire**.

In altre parole:

- con gli OUTPUT Arduino "fa qualcosa"
- con gli INPUT Arduino "ascolta" quello che succede

Ed è proprio grazie agli input che Arduino può iniziare a prendere decisioni e diventare davvero "intelligente".

Nei libri precedenti abbiamo utilizzato spesso i **pulsanti**.

I pulsanti sono l'esempio più semplice e importante di **input**.

Pensaci un attimo:
quasi tutti i dispositivi elettronici che usi ogni giorno hanno almeno un pulsante.

Per esempio:

- il tasto per accendere il telefono
- il telecomando
- il computer

Con un pulsante stiamo dando un'informazione alla macchina, come dire: "fai questa cosa".

In poche parole:
il pulsante è un modo per **comunicare con Arduino**

Nei libri precedenti abbiamo acceso dei LED premendo dei pulsanti.

Ma in quel caso il pulsante non era un vero **input** nel senso della programmazione.

Perché?

Perché il pulsante agiva direttamente sul circuito:
quando lo premevi, la corrente passava e il LED si accendeva.

In programmazione, invece, un **input** è qualcosa che viene:

- letto da Arduino
- interpretato
- usato per decidere cosa fare

prima il pulsante accendeva il LED da solo, adesso vogliamo che sia Arduino a "capire" se è premuto e decidere cosa fare.

Adesso vogliamo fare un passo in più.

Non basta più premere un pulsante per accendere un LED direttamente. Vogliamo che sia **Arduino a leggere il pulsante** e decidere cosa fare.

Per fare questo usiamo una nuova funzione:

digitalRead

Questa funzione significa:
"leggi lo stato di un pin in modo digitale".

Ma perché Arduino possa leggere correttamente il pulsante, dobbiamo fare in modo che ci siano **due stati ben distinti**:

- **HIGH (alto)**
- **LOW (basso)**

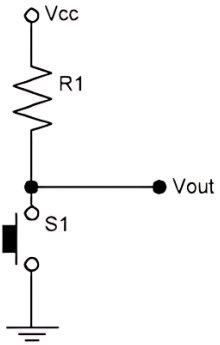
In pratica:

- pulsante non premuto ->un valore
- pulsante premuto -> un altro valore

Per ottenere questo risultato usiamo un piccolo trucco:
un **resistore di pull-up**

Cosa succede?

- quando il pulsante NON è premuto → il resistore mantiene il pin su **HIGH**
- quando premi il pulsante → il pin viene portato a **LOW**



All'inizio può sembrare complicato, ma in realtà è molto semplice:

- il resistore tiene il pin "alto"
- il pulsante lo porta "basso" quando lo premi

Quando impostiamo un pin di Arduino come **INPUT**, questo si comporta un po' come se fosse collegato a un **resistore molto grande**.

Questo significa che da solo non "decide" se essere alto o basso, ha bisogno di qualcosa che gli dica che valore assumere.

Per questo usiamo un resistore (per esempio da **10kOhm**) collegato ai 5V.

Cosa succede in questo caso?

quando il pulsante **non è premuto** → il pin viene "tirato" verso i **5V** quindi diventa **HIGH**

Questo succede perché la corrente preferisce passare dal resistore più piccolo (10k) invece che da quello interno molto grande.

Quando invece **premiamo il pulsante**: il pin viene collegato direttamente a **0V (massa)** e quindi diventa **LOW.**

- pulsante non premuto → **HIGH**
- pulsante premuto → **LOW**

Questo ci permette di leggere in modo chiaro lo stato del pulsante.

Sembra complicato… ma in perfetto stile Arduino, rendiamolo semplice.

Abbiamo visto che per leggere lo stato di un pin usiamo la funzione **digitalRead**.

Ma prima dobbiamo dire ad Arduino che quel pin sarà un **INPUT**.

Per farlo, nel **setup()** scriviamo:

```
pinMode(pin, INPUT);
```

Fin qui tutto bene… ma c'è ancora un problema.

Se usiamo solo questa istruzione, il pin non ha un valore stabile.

Questo significa che:

- a volte può leggere HIGH
- a volte LOW
- senza un motivo preciso

Per evitare questo, come abbiamo visto prima, servirebbe un **resistore esterno** per "tenere" il pin su HIGH quando il pulsante non è premuto.

INPUT_PULLUP

Per evitare di aggiungere un resistore esterno, Arduino ci offre una soluzione molto comoda.

Possiamo scrivere:

```
pinMode(pin, INPUT_PULLUP);
```

Questa istruzione attiva un **resistore di pull-up interno** al microcontrollore.

In pratica:
Arduino fa tutto da solo, senza bisogno di componenti aggiuntivi.

Cosa succede quindi?

- quando il pulsante **non è premuto** → il pin è **HIGH**
- quando premi il pulsante → il pin va a **LOW**

Questo significa che dobbiamo solo collegare il pulsante in modo che, quando viene premuto, porti il pin a massa (0V).

con **INPUT_PULLUP** evitiamo di usare un resistore esterno e semplifichiamo molto il circuito

Facile, no?

In pratica, grazie alla **ELAB Nano Breakout**, ci basta **un solo filo** per leggere lo stato di un pulsante!

Ora è il momento di mettere in pratica quello che abbiamo imparato.

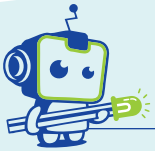
Proveremo alcuni esercizi che uniscono:

- **input** (come i pulsanti)
- **output** (come i LED)

Durante questi esercizi scopriremo anche nuove regole della programmazione, un passo alla volta.

Questo ci permetterà di scrivere programmi sempre più completi e interessanti.

E da qui inizierai davvero a costruire progetti che reagiscono a ciò che succede nel mondo reale.



ESERCIZIO 6.6

LEGGERE UN PULSANTE E CONTROLLARE UN LED

In questo esperimento impareremo a usare l'istruzione **digitalRead** per leggere lo stato di un pulsante e, in base a questo, accendere o spegnere un LED.

Materiale necessario:

- Arduino Nano R4
- ELAB Nano Breakout
- 1 LED verde
- 1 pulsante
- 1 resistore da 470 Ohm
- 2 cavetti

Come visto nella teoria, useremo la modalità **INPUT_PULLUP**.

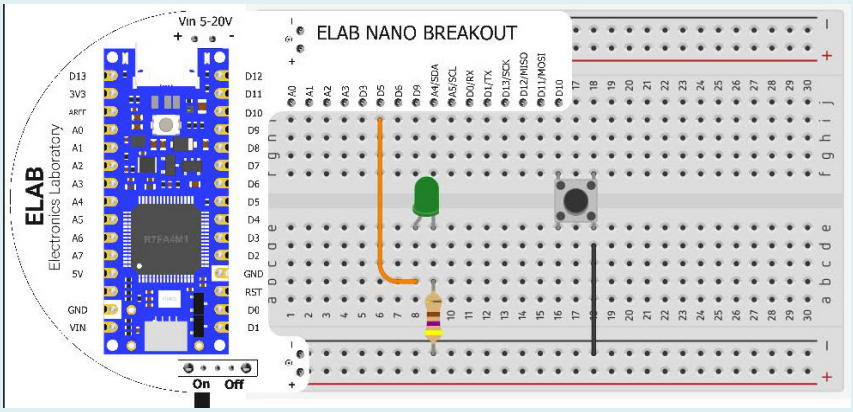
Questo ci permette di non aggiungere un resistore esterno per il pulsante, semplificando il circuito.

Per questo esempio useremo:

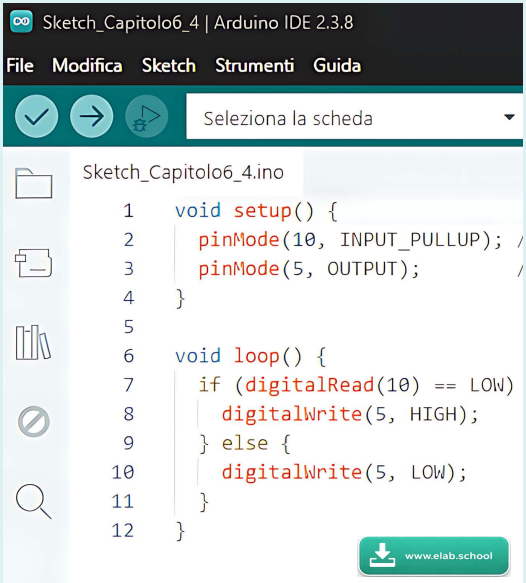
- il **pin 10** per leggere il pulsante
- un altro pin (per esempio il 6 o il 9) per controllare il LED

Il pin 10 è scelto solo per comodità:
in realtà puoi usare **qualsiasi pin disponibile**

Lo stesso vale per il LED:
puoi collegarlo a qualunque pin della breakout



Poche righe di codice... ma tante cose nuove!



Per prima cosa, dobbiamo dire ad Arduino che vogliamo usare il **pin 10 come INPUT_PULLUP**.
In questo modo attiviamo il resistore interno e possiamo leggere il pulsante senza componenti aggiuntivi.

Poi, come abbiamo già fatto prima, impostiamo il pin del LED come **OUTPUT**.

Adesso arriva la parte più interessante.

Nel **loop()** dobbiamo controllare continuamente lo stato del pulsante e decidere cosa fare:

- se il pulsante è premuto → accendiamo il LED
- altrimenti → lo spegniamo

Qui entrano in gioco due parole fondamentali della programmazione:

- **IF** (se)
- **ELSE** (altrimenti)

In pratica stiamo dicendo ad Arduino:
"**Se** succede questa cosa, fai questo…
altrimenti fai qualcos'altro"

Un'altra regola molto importante riguarda la struttura del codice.

Ogni volta che vogliamo eseguire più istruzioni insieme, dobbiamo usare le parentesi graffe:

```
if (condizione) {
  // istruzioni
}
```

La parentesi **{** apre il blocco
La parentesi **}** lo chiude

con if e else Arduino inizia a prendere decisioni
e il tuo programma diventa davvero “intelligente”

LE DECISIONI NEL CODICE: L’ISTRUZIONE IF

Quindi, se traduciamo il nostro programma in parole semplici (quello che si chiama anche **pseudocodice**), stiamo dicendo:

SE il pulsante collegato al pin 10 è premuto
→ accendi il LED collegato al pin 5

ALTRIMENTI
→ spegni il LED

Ora capiamo meglio come si scrive questa cosa nel linguaggio di Arduino.

Quando usiamo la parola **SE** (in inglese *if*), dobbiamo sempre usare le parentesi tonde ( ).
Dentro queste parentesi scriviamo la **condizione**, cioè quello che vogliamo controllare.

Nel nostro caso vogliamo sapere:
“Il pulsante è premuto oppure no?”

Per scoprirlo usiamo l’istruzione **digitalRead(10)**.

Questa funzione legge il pin 10 e restituisce solo due possibili valori:

- **HIGH** (alto)
- **LOW** (basso)

Ed è qui che bisogna fare attenzione a una cosa importante.

Nel codice troviamo questa scrittura:

```
if (digitalRead(10) == LOW)
```

IL CONFRONTO NEL CODICE: L'OPERATORE ==

Quel doppio uguale == non è un errore.

Il simbolo **=** (uno solo) serve per assegnare un valore
Il simbolo **==** (due uguali) serve per confrontare

Quindi stiamo dicendo:
"SE il valore letto è UGUALE a LOW..."

È come fare una domanda al computer:
"È vero che il pulsante è premuto?"

Se la risposta è sì, allora Arduino esegue quello che c'è dentro il blocco.
Se invece è no, passa all'**ALTRIMENTI (else)**.

Ed è proprio così che iniziamo a prendere decisioni nei programmi:
facendo domande... e dando istruzioni diverse a seconda della risposta.

Questo esercizio è molto semplice, ma è anche molto importante.
Ci permette di fare con Arduino quello che avevamo già fatto nel Volume 1, quando collegavamo direttamente un pulsante a un LED per accenderlo e spegnerlo.

In quel caso, però, il comportamento era "automatico":
il pulsante controllava direttamente il LED.

Adesso invece stiamo facendo la stessa cosa, ma usando la **programmazione**.

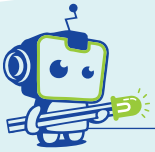
È Arduino che legge il pulsante e decide cosa fare.

E qui arriva la parte interessante.

Con la programmazione non siamo più limitati ad un solo comportamento.

Per esempio, possiamo fare qualcosa di più avanzato: accendere il LED quando premiamo il pulsante e spegnerlo quando lo premiamo una seconda volta.

prima il circuito faceva una sola cosa,
adesso possiamo decidere noi come deve comportarsi



ESERCIZIO 6.7

Il circuito è esattamente lo stesso di prima, ma ora vogliamo cambiare il comportamento.

Invece di tenere il LED acceso solo mentre premiamo il pulsante, vogliamo:

- accenderlo con una pressione
- spegnerlo con la pressione successiva

Per fare questo dobbiamo introdurre un nuovo concetto fondamentale:
le variabili

Le variabili sono come delle "scatole" dove possiamo salvare delle informazioni.

Per esempio:

- un numero
- un risultato
- oppure, come nel nostro caso, uno stato

Noi vogliamo ricordarci se il LED è:

- acceso
- spento

Quindi ci serve una variabile che possa contenere solo due valori.

Questo tipo di variabile si chiama **booleana** (o "bool").

Per creare una variabile si usa questa struttura:

tipo nomeVariabile = valore_iniziale;

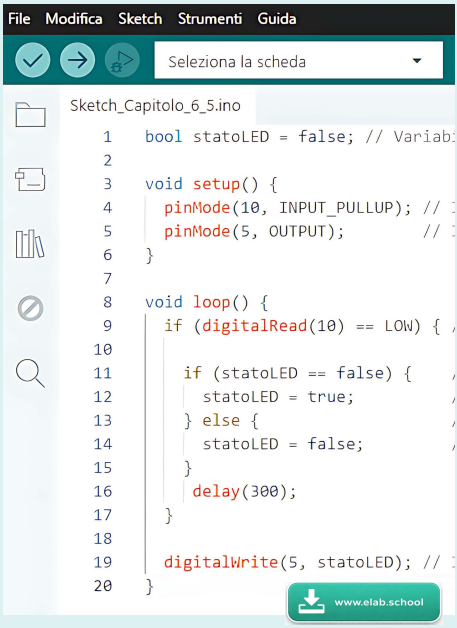
Nel nostro caso scriviamo:

bool statoLED = false;

Questo significa:

- **bool** → il tipo (solo vero o falso)
- **statoLED** → il nome della variabile
- **false** → il valore iniziale (LED spento)

Stiamo creando una "memoria" che ci dice se il LED è acceso oppure no e questa sarà la chiave per far cambiare stato al LED ogni volta che premiamo il pulsante.



Leggi lo stato del pulsante

SE il pulsante è premuto:

SE il LED è spento (false):

accendi il LED (statoLED =

true) ALTRIMENTI

(se il LED è acceso:

spegni il LED (statoLED = false)

Imposta il LED in base al valore della variabile statoLED

Ogni volta che premi il pulsante, il LED cambia stato.

Se provi a caricare questo codice sulla tua Arduino Nano R4 noterai subito una cosa un po' strana:
quando premi il pulsante, a volte il LED non si comporta come ti aspetti. Può accendersi e spegnersi più volte velocemente, oppure fare il contrario.

Ma perché succede questo?

Il motivo è molto semplice.

All'interno del pulsante c'è una piccola **molla** che serve a farlo tornare su quando lo rilasci. Quando premi il pulsante, i contatti si toccano… ma non in modo perfetto e stabile.

Per qualche istante fanno "rimbalzo" (in inglese *bounce*)

Questo significa che:

- il contatto si chiude
- si riapre
- si richiude di nuovo
  tutto in pochissimo tempo

Il problema è che Arduino è **velocissimo**.

Riesce a leggere questi piccoli cambiamenti più volte, anche se per noi sembrano impercettibili.

Quindi può succedere che:

- tu premi una sola volta
- ma Arduino pensa che tu abbia premuto più volte

E per questo il programma entra più volte nell'IF, facendo cambiare stato al LED in modo "strano".

Non è il codice sbagliato… è il pulsante che "rimbalza"
Questo fenomeno si chiama **bounce**, cioè "rimbalzo".

Per fortuna esiste un modo molto semplice per risolvere questo problema.

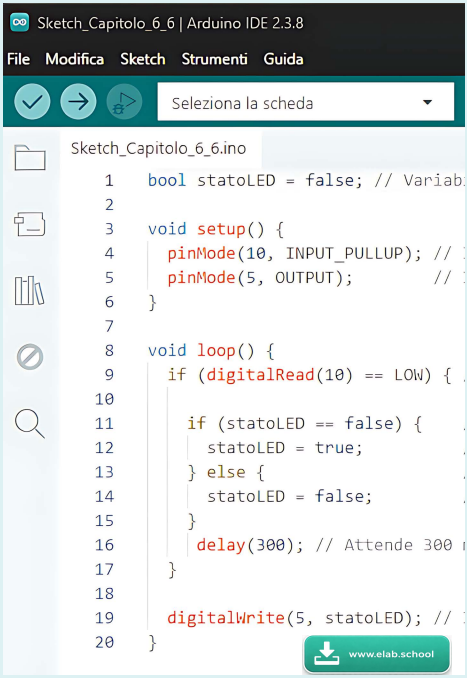
Possiamo aggiungere un piccolo ritardo subito dopo aver letto il pulsante.

In questo modo diamo il tempo alla molla interna di fermarsi e al segnale di diventare stabile.

Se non lo facciamo, Arduino potrebbe continuare a leggere quei piccoli cambiamenti velocissimi e pensare che il pulsante venga premuto più volte.

Un valore sicuro è circa **300 millisecondi**.

Questo significa che, dopo aver rilevato la pressione del pulsante, Arduino aspetta un attimo prima di controllarlo di nuovo.



Aggiungendo un piccolo delay evitiamo che Arduino venga "ingannato" dal rimbalzo del pulsante

Se provi ora questo programma noterai che:

- se premi una volta il pulsante il LED si accende
- se ripremi si spegne

Sembra tutto funzionare correttamente… ma c'è ancora un piccolo problema.

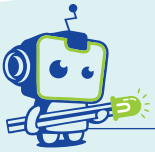
Se tieni premuto il pulsante, il LED **inizia a lampeggiare** ogni 300 millisecondi.

Questo succede perché Arduino è molto veloce e continua a leggere il pulsante come "premuto" tante volte di seguito.

Ogni volta che lo legge come premuto, entra nell'IF e cambia lo stato del LED

Quindi:

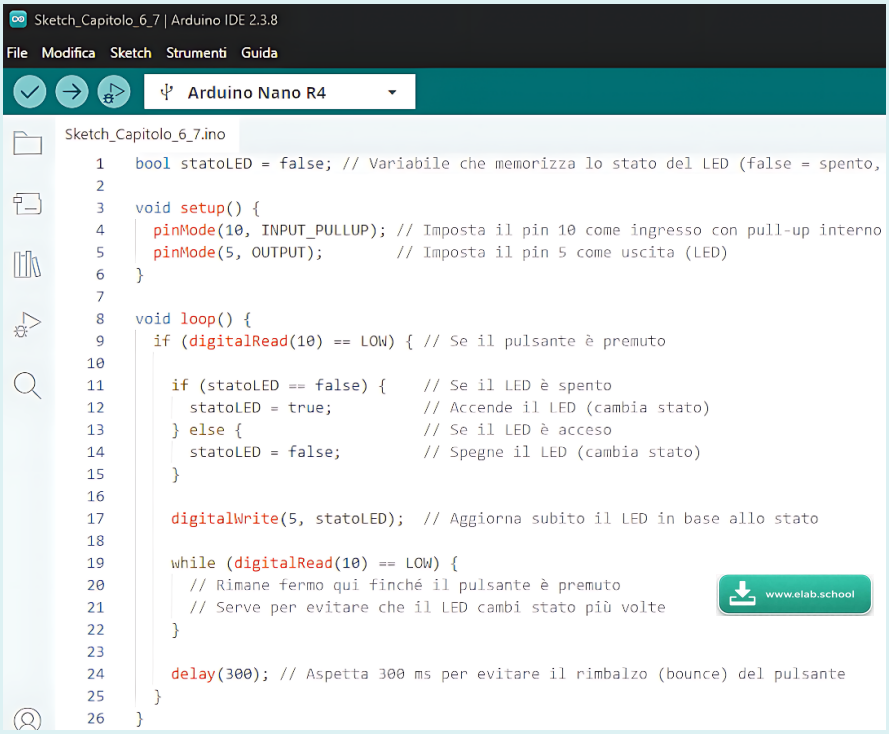
- tu fai una sola pressione lunga
- Arduino pensa che siano tante pressioni



ESERCIZIO 6.8

EVITARE IL LAMPEGGIO: GESTIRE CORRETTAMENTE IL PULSANTE

In questo esperimento capiremo come sistemare il problema del lampeggio dell'esperimento 6.7!




```
bool statoLED = false; // Variabile che memorizza lo stato del LED (false = spento,

void setup() {
  pinMode(10, INPUT_PULLUP); // Imposta il pin 10 come ingresso con pull-up interno
  pinMode(5, OUTPUT);        // Imposta il pin 5 come uscita (LED)
}

void loop() {
  if (digitalRead(10) == LOW) { // Se il pulsante è premuto

    if (statoLED == false) {    // Se il LED è spento
      statoLED = true;          // Accende il LED (cambia stato)
    } else {                    // Se il LED è acceso
      statoLED = false;         // Spegne il LED (cambia stato)
    }

    digitalWrite(5, statoLED);  // Aggiorna subito il LED in base allo stato

    while (digitalRead(10) == LOW) {
      // Rimane fermo qui finché il pulsante è premuto
      // Serve per evitare che il LED cambi stato più volte
    }

    delay(300); // Aspetta 300 ms per evitare il rimbalzo (bounce) del pulsante
  }
}
```

Come puoi osservare, abbiamo fatto una piccola ma importante modifica: abbiamo spostato la **digitalWrite** dentro l'IF principale.

In questo modo il LED si accende o si spegne subito dopo aver deciso cosa fare.
Il comportamento diventa più veloce e reattivo

L'ultima operazione resta il **delay**, proprio come avevamo previsto nello pseudocodice.

E in mezzo?

Non dobbiamo fare nulla… finché il pulsante non viene rilasciato.

Questo passaggio è fondamentale: serve a evitare che il LED cambi stato più volte quando teniamo premuto il pulsante.

Il problema è che il pulsante **non viene ancora rilasciato**.

Dobbiamo quindi fare in modo che Arduino aspetti finché il pulsante resta premuto.

Per farlo usiamo una nuova struttura chiamata **while**.

While in inglese significa "fino a quando".
È un ciclo, cioè una struttura che continua a ripetersi finché una certa condizione è vera.

La sua forma è molto semplice:

```
while (condizione) {

  // fai qualcosa
}
```

Nel nostro caso vogliamo ottenere questo comportamento:

- Finché il pulsante è premuto non fare nulla

Per controllare se il pulsante è premuto usiamo:

```
digitalRead(10) == LOW
```

Quindi scriviamo:

```
while (digitalRead(10) == LOW) {
}
```

Questo significa:

"rimani qui fermo finché il pulsante è premuto"

Il blocco è vuoto perché non vogliamo eseguire nessuna azione, stiamo semplicemente aspettando che il pulsante venga rilasciato.

Come sempre, quando apriamo un blocco usiamo le parentesi graffe { }, proprio come abbiamo fatto con gli **if**.

Ora puoi caricare il programma:
dovrebbe funzionare correttamente e il LED cambierà stato una sola volta per ogni pressione.

NOTE

IL MONDO CONTINUO: I PIN ANALOGICI

Nel capitolo precedente abbiamo parlato di un mondo fatto di **0 e 1**, cioè di valori digitali: acceso o spento, vero o falso.

Ma il mondo reale che ci circonda non funziona così.

Il mondo reale è **analogico**.

Ma cosa significa analogico?

Significa che, invece di avere solo due possibilità, possiamo avere **tantissimi valori diversi**, anche infiniti.

Facciamo un esempio semplice.

Se vogliamo misurare una tensione, possiamo usare un **multimetro**.

Quando lo facciamo, non leggiamo solo "acceso" o "spento", ma un valore preciso, per esempio:

- 1,2 Volt
- 3,7 Volt
- 4,95 Volt

E usando strumenti sempre più precisi possiamo misurare anche i millivolt (mV), cioè valori ancora più piccoli.

Questo vuol dire che possiamo ottenere misure sempre più precise.

In pratica, i valori analogici sono quasi infiniti.

I NUMERI BINARI E LA RISOLUZIONE:

Teniamo bene a mente questo esempio.

Abbiamo detto che Arduino lavora in **binario**, cioè con 0 e 1.

Allora nasce una domanda importante:

come fa a misurare una tensione?
come fa ad esempio un multimetro!

Se ci pensi, anche il multimetro ha uno **schermo** che mostra un numero.

Quel numero non appare da solo: qualcuno deve calcolarlo e mostrarlo.

All'interno del multimetro c'è un **microcontrollore**

Questo microcontrollore è programmato per:

- leggere la tensione
- trasformarla in un numero
- mostrarla sul display

A seconda dello strumento che utilizziamo, possiamo avere misure più o meno precise.

Più uno strumento è preciso, più cifre riesce a mostrarci.
Questo si chiama **risoluzione**.

Ma cos'è la risoluzione?

La risoluzione ci dice **quanto è preciso uno strumento** e quanti valori diversi può distinguere.

Nel caso dei microcontrollori, ci dice anche **quanti numeri (in binario)** possiamo usare per rappresentare una misura.

Facciamo un passo indietro per capire meglio.

I numeri possono essere scritti in modi diversi:

Nel sistema **decimale** (quello che usiamo tutti i giorni) abbiamo 10 cifre:
0 1 2 3 4 5 6 7 8 9

Nel sistema **binario** (quello dei computer) abbiamo solo 2 cifre:
0 e 1

Vediamo qualche esempio:

0 (decimale) = 0 (binario)
7 (decimale) = 111 (binario)
1 (decimale) = 1 (binario)
8 (decimale) = 1000 (binario)
2 (decimale) = 10 (binario)
9 (decimale) = 1001 (binario)
3 (decimale) = 11 (binario)
10 (decimale) = 1010 (binario)
4 (decimale) = 100 (binario)
11 (decimale) = 1011 (binario)
5 (decimale) = 101 (binario)
12 (decimale) = 1100 (binario)
6 (decimale) = 110 (binario)
13 (decimale) = 1101 (binario)

Cosa significa questo?

Per rappresentare numeri più grandi in binario, servono **più cifre**. Più cifre abbiamo a disposizione, più possiamo rappresentare valori diversi.

la **risoluzione** dipende da quanti "bit" usiamo e più bit abbiamo, più la misura sarà precisa

quando si esauriscono le combinazioni possibili, il numero cresce verso sinistra

Come puoi osservare, ogni volta che aggiungiamo **+1** a un numero binario succede qualcosa di particolare.

Siccome abbiamo solo due cifre (0 e 1), non possiamo andare oltre 1.

Quindi, quando arriviamo al massimo possibile con le cifre disponibili, dobbiamo:

- **spostare un 1 verso sinistra**
- e rimettere le altre cifre a 0

Facciamo un esempio:

0 → 0

1 → 1

1 + 1 = 2 → **10**

2 + 1 = 3 → **11**

3 + 1 = 4 → **100**

È un po' come nel sistema decimale: quando passiamo da 9 a 10, aggiungiamo una nuova cifra.

In binario succede la stessa cosa, ma molto prima, perché abbiamo solo 0 e 1.

Ora che abbiamo capito come funzionano i numeri binari e cos'è la risoluzione, possiamo affrontare un concetto molto importante:

LA CONVERSIONE ANALOGICO-DIGITALE

La nostra Arduino Nano R4 funziona a **5 Volt**.

Questo significa che può leggere tensioni da:

- 0 V (minimo)
- 5 V (massimo)

Queste tensioni sono valori **analogici**, cioè continui.

Ma Arduino capisce solo numeri digitali (binari).

Quindi deve trasformare la tensione in un numero.

La scheda lo fa grazie a un sistema chiamato **ADC (convertitore analogico-digitale)**.

Questo sistema associa a ogni tensione un numero.

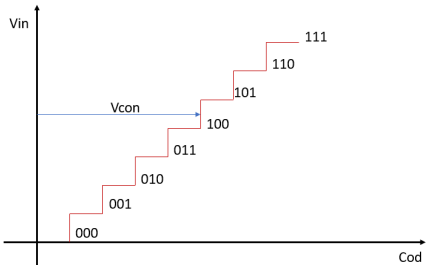
La risoluzione della scheda è di **10 bit**.

Questo significa che può rappresentare **1024 valori diversi** cioè da **0 a 1023**

Facciamo qualche esempio:

- a **0 V** corrisponde il numero **0**
- a **5 V** corrisponde il numero **1023**

Tutti gli altri valori stanno in mezzo



Per esempio:

a **2,5 V** (che è metà di 5 V) dovrebbe corrispondere circa metà di 1023

cioè:
1023 / 2 = **511,5**

Ma Arduino può usare solo numeri interi, quindi:
deve scegliere un numero vicino.

Questo problema si chiama **quantizzazione.**

In pratica, dobbiamo arrotondare (per esempio a 511 o 512).

Ora vediamo una cosa molto importante.

Quanto vale il "passo minimo", cioè quanto cambia la tensione passando da un numero al successivo?

Basta fare:

5V ÷ 1024 = **4.88mV** circa 5 mV

Questo significa che:
ogni numero rappresenta un aumento di **0,005 Volt**, cioè **5 millivolt (5 mV)**

Arduino non può leggere qualsiasi valore preciso ma lo divide in tanti piccoli "gradini" da 5 mV

Quindi tra **511 e 512** c'è una differenza di **5 mV**.

Questo significa che Arduino ha una **sensibilità di 5 mV**

In pratica:
il valore cambia solo se la tensione aumenta (o diminuisce) di almeno **5 millivolt**.

Se la variazione è più piccola, Arduino non se ne accorge.

Ora facciamo un altro passo importante.

Come facciamo a capire a quale tensione corrisponde il numero letto da Arduino?

È molto semplice: basta fare una proporzione.

5V : 1023 = V : numero

Da cui otteniamo la formula:

V = (5 * numero) / 1023

Facciamo qualche esempio.

Se Arduino legge **511**:

V = (5 * 511) / 1023 ≈ 2,497 V

Se legge **512**:

V = (5 * 512) / 1023 ≈ 2,502 V

La differenza tra i due valori è proprio **0,005 V (5 mV)**

Sulla Arduino Nano R4, i pin che possono leggere valori analogici sono quelli con la lettera **A**:

A0, A1, A2, A3, A4, A5, A6, A7

In totale abbiamo **8 pin analogici.**

Ora siamo pronti per fare qualcosa di più interessante.

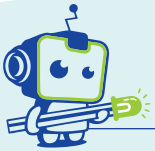
Useremo un componente che già conosci: il **potenziometro**.

Lo useremo come **partitore di tensione.**

In questo modo potremo:

- ottenere una tensione variabile
- leggerla con Arduino
- e prendere decisioni in base al valore letto

Passiamo dalla teoria… alla pratica

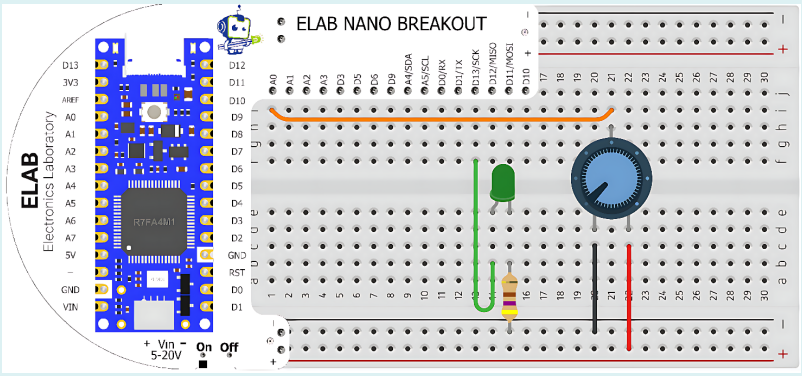


ESERCIZIO 7.1

In questo esperimento faremo una cosa molto interessante: accenderemo un LED collegato al pin 13 della Arduino Nano R4 solo quando la tensione letta sul pin A0 supera i 2,5 V.

Per realizzare il circuito abbiamo bisogno di:

- Arduino Nano R4
- ELAB Nano Breakout
- 1 LED verde
- 1 resistore da 470 Ohm
- 1 trimmer (potenziometro)
- 3 cavetti



Prova a costruire il circuito come nella figura.

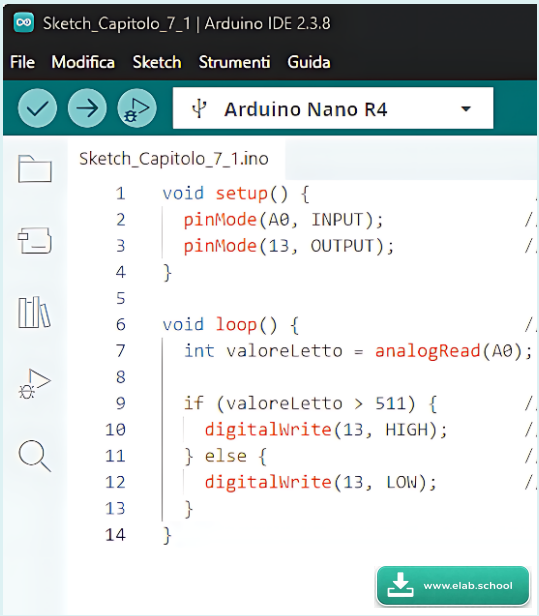
I collegamenti sono molto semplici. Se metti i componenti nello stesso modo dell'immagine, sarà più facile collegare tutto senza errori.
Ricorda che il trimmer, collegato così, funziona come un partitore di tensione.

Quando lo giri:

- verso sinistra, la tensione sul pin centrale (pin 2) è circa 0V
- verso destra, la tensione arriva fino a circa 5V
- a metà, la tensione è circa 2,5V

Questo succede perché le resistenze interne cambiano: una aumenta mentre l'altra diminuisce.

In questo esercizio non serve usare il PULLUP. Infatti, al pin A0 è collegato il trimmer, che funziona già come partitore di tensione. Aggiungere un'altra resistenza altererebbe la misura, rendendola meno precisa.



Per questo motivo impostiamo semplicemente:

pinMode(A0, INPUT);

Sul pin 13 invece abbiamo collegato un LED, che può solo essere acceso o spento.
Quindi lo impostiamo come uscita:

pinMode(13, OUTPUT);

Nel loop dobbiamo leggere continuamente il valore analogico dal pin A0.

Per farlo, creiamo una variabile intera chiamata **valoreLetto** e le assegniamo il valore letto con:

int valoreLetto = analogRead(A0);

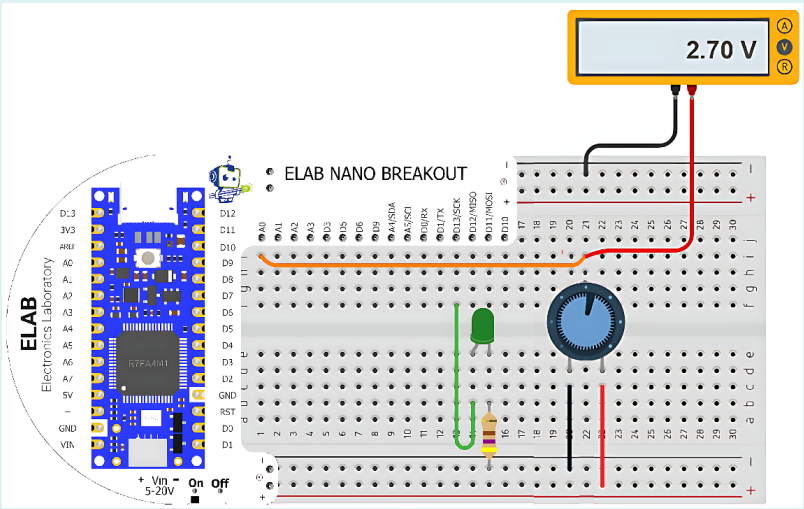
Questo significa:
dichiara una variabile di tipo intero chiamata **valoreLetto** e assegnale il valore letto dal pin A0.

Attenzione: qui usiamo un solo = perché stiamo assegnando un valore, non confrontando (come invece si fa con ==).

A questo punto controlliamo il valore letto.

Se è maggiore di 512 (circa metà, quindi circa 2.5V), accendiamo il LED. Altrimenti lo spegniamo.

Ora prova tu:
ruota il trimmer oltre la metà per accendere il LED, e riportalo indietro per spegnerlo. Se vuoi, puoi anche misurare la tensione con un multimetro per vedere cosa succede davvero



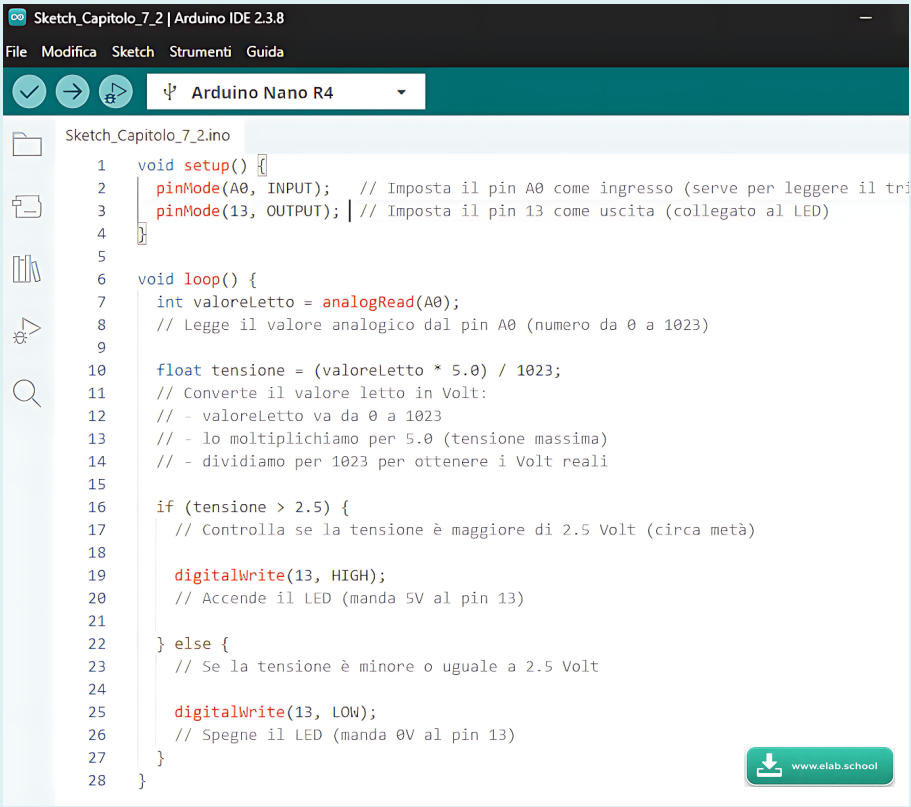
ESERCIZIO 7.2

In questo esperimento faremo una piccola modifica all'esercizio precedente, usando lo stesso circuito.

Finora abbiamo controllato il valore letto in forma digitale (da 0 a 1023). Per esempio, abbiamo usato il valore 512, che corrisponde circa a 2.5V.

Ma cosa succede se vogliamo ragionare direttamente in Volt invece che in numeri?

Come possiamo usare un valore di tensione (per esempio 2.5V) dentro l'if?




```
void setup() {
  pinMode(A0, INPUT);   // Imposta il pin A0 come ingresso (serve per leggere il tri
  pinMode(13, OUTPUT); // Imposta il pin 13 come uscita (collegato al LED)
}

void loop() {
  int valoreLetto = analogRead(A0);
  // Legge il valore analogico dal pin A0 (numero da 0 a 1023)

  float tensione = (valoreLetto * 5.0) / 1023;
  // Converte il valore letto in Volt:
  // - valoreLetto va da 0 a 1023
  // - lo moltiplichiamo per 5.0 (tensione massima)
  // - dividiamo per 1023 per ottenere i Volt reali

  if (tensione > 2.5) {
    // Controlla se la tensione è maggiore di 2.5 Volt (circa metà)

    digitalWrite(13, HIGH);
    // Accende il LED (manda 5V al pin 13)

  } else {
    // Se la tensione è minore o uguale a 2.5 Volt

    digitalWrite(13, LOW);
    // Spegne il LED (manda 0V al pin 13)
  }
}
```

Il setup() rimane uguale a prima, perché non dobbiamo cambiare i collegamenti.

Dobbiamo invece lavorare sul loop(), dove decidiamo quando accendere il LED.

Dopo aver letto il valore con valoreLetto, dobbiamo fare un calcolo per trasformarlo in tensione (Volt).

Questo calcolo ci permette di capire a quanti Volt corrisponde il valore letto dal pin A0.

Per fare questo dobbiamo:

- dichiarare una variabile che possa contenere numeri con la virgola
- prendere il valore letto dal pin analogico
- moltiplicarlo per 5 (la tensione di alimentazione della scheda)
- dividere il risultato per 1023 (il valore massimo leggibile)

Quando scriviamo la moltiplicazione usiamo **5.0** invece di 5.

Questo serve per far capire al compilatore che stiamo lavorando con numeri con la virgola.
Vedremo meglio questo concetto più avanti.

Ora abbiamo ottenuto il valore della tensione in Volt.

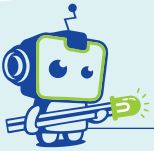
Possiamo quindi usarlo per controllare se siamo sopra o sotto i **2.5V**:

- sopra → LED acceso
- sotto → LED spento

Come puoi notare:

- prima usiamo int per salvare un numero intero
- poi usiamo float per salvare un numero con la virgola

Andando avanti impareremo altri tipi di variabili.



ESERCIZIO 7.3

In questo esercizio useremo il trimmer per accendere un LED diverso in base alla sua posizione.

Abbiamo tre LED:

- rosso
- giallo
- verde

Per questo dividiamo il valore massimo (1023) in tre parti:

1023 ÷ 3 = circa 341

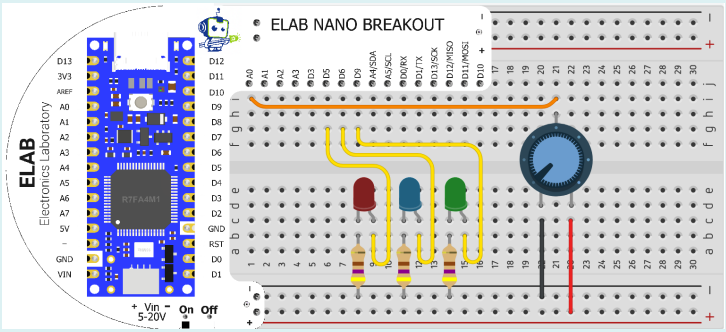
Quindi:

- da **0 a 340** -> LED rosso
- da **341 a 681** -> LED giallo
- da **682 a 1023** ->LED verde

Per controllare questi intervalli useremo l'operatore **&&** (AND).

Serve per verificare due condizioni insieme, ad esempio:
"maggiore di 341 **e** minore di 682"

In questo modo possiamo capire in quale posizione si trova il trimmer e accendere il LED giusto.



Proviamo a scrivere in pseudocodice quello che vogliamo fare:

se la lettura analogica è tra 0 e 341:

- accendi il LED rosso
- spegni il LED giallo
- spegni il LED verde

se la lettura analogica è tra 341 e 682:

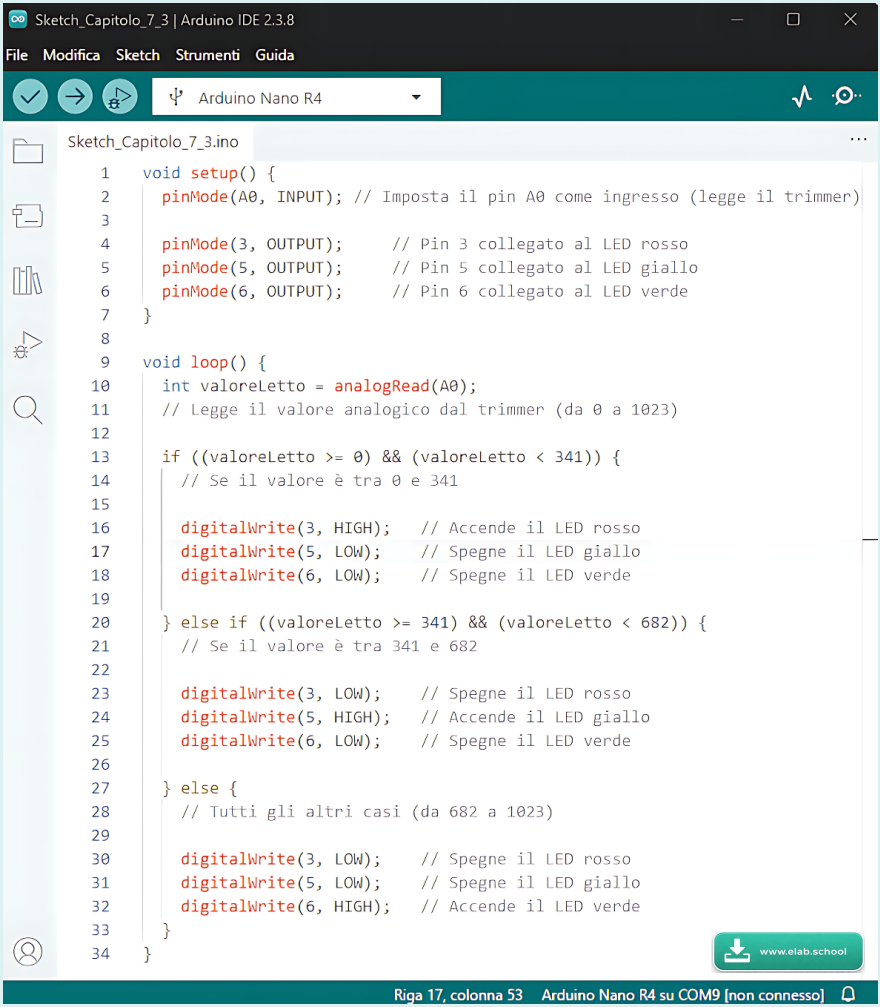
- spegni il LED rosso
- accendi il LED giallo
- spegni il LED verde

altrimenti:

- spegni il LED rosso
- spegni il LED giallo
- accendi il LED verde

Nota importante (da capire bene):

- "altrimenti" significa: tutti i casi che non sono nei primi due
- quindi da 682 a 1023



Nel setup() dobbiamo impostare:

- il pin **A0 come input**, perché leggiamo il trimmer
- i pin **3, 5 e 6 come output**, perché comandano i LED

Nel loop() leggiamo il valore del pin A0 e lo salviamo nella variabile valoreLetto.

Ora vediamo le condizioni.

Nel primo **if** controlliamo che il valore sia tra 0 e 341:

```
0 <= valoreLetto < 341
```

Per farlo usiamo:

- le parentesi
- l'operatore && (che significa "e")

Quindi verifichiamo che il valore sia:
maggiore o uguale a 0 **e** minore di 341

Poi usiamo una cosa nuova: **else if** (altrimenti se).

Finora abbiamo usato:

- if
- else

Ma in questo caso abbiamo **tre situazioni diverse**, quindi usiamo:

- if → prima condizione
- else if → seconda condizione
- else → tutti gli altri casi

In inglese:

- if (condizione A)
- else if (condizione B)
- else

Se servono più condizioni, possiamo aggiungere altri else if.

Regola importante.
È buona norma:
- controllare tutti i casi importanti con if e else if
- usare else per gestire tutto il resto

Così evitiamo comportamenti strani del programma.

Esempio reale

Immagina un cancello automatico.

Se dimentichi una condizione, potrebbe succedere qualcosa di pericoloso, ad esempio:
il cancello non si apre anche se c'è un ostacolo!

Per evitare problemi, possiamo dire: "in tutti gli altri casi → apri il cancello"

Questo è proprio il ruolo dell'**else**:
coprire tutte le situazioni non previste per rendere il sistema sicuro.

Finora abbiamo usato analogRead per leggere un segnale analogico e abbiamo parlato della **conversione analogico → digitale (ADC)**.

Ma esiste anche il contrario:
la **conversione digitale → analogica (DAC)**.

CONVERSIONE DIGITALE ANALOGICA

Quando vogliamo trasformare un numero in una tensione, usiamo un componente chiamato **DAC** (Digital to Analog Converter).

La scheda **Arduino NANO R4** ha un DAC collegato al pin **A0**,
sì, proprio lo stesso pin usato prima per leggere!

Per utilizzare il DAC in Arduino usiamo:

analogWrite()

Questa istruzione permette di generare una **tensione analogica**.

Possiamo anche scegliere la precisione (fino a 12 bit) usando:

analogWriteResolution(numeroDiBit)

Più bit ⟶ maggiore precisione della tensione.

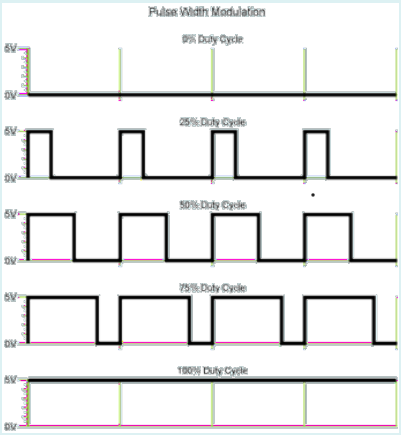
⚠ **Attenzione: esistono due analogWrite!**

In Arduino esistono **due modi diversi** di usare analogWrite:

1. **DAC reale** (solo su alcuni pin, come A0 sulla NANO R4)
2. **PWM** (su pin con simbolo ~)

COS'È IL PWM?

Il PWM (Pulse Width Modulation) è un trucco per **simulare una tensione analogica** usando un segnale digitale.



Il segnale:

- passa velocemente tra **0V e 5V**
- si ripete continuamente

Concetti importanti

- **Periodo (T)** → tempo in cui il segnale si ripete
- **TON** → tempo in cui il segnale è alto (5V)
- **TOFF** → tempo in cui è basso (0V)
  T = TON + TOFF
- **Duty Cycle (DC)** → quanto tempo resta acceso:
  DC = TON / T

Come funziona davvero

Non possiamo cambiare il periodo con Arduino,
ma possiamo cambiare **quanto tempo il segnale resta acceso (TON)**.

Quindi controlliamo il **Duty Cycle**

Valori PWM in Arduino

Il PWM usa **8 bit**, quindi:

valori da **0 a 255**

- 0 → sempre spento
- 255 → sempre acceso
- 127 → circa metà (50%)

Esempio:

```
analogWrite(3, 127);
```

segnale al 50% sul pin 3

Perché funziona?

Anche se il LED si accende e spegne velocemente, il nostro occhio

non vede lo sfarfallio ma vede una luminosità "media".

Questo succede grazie alla **media del segnale nel tempo**.

Applicazione pratica:

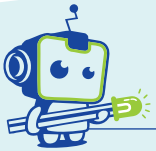
Se aumenti il valore di analogWrite:

il LED diventa più luminoso

Se lo diminuisci:
il LED diventa più debole

In breve

- DAC → tensione analogica vera
- PWM → tensione "simulata"
- analogWrite() → controlla entrambi (a seconda del pin)



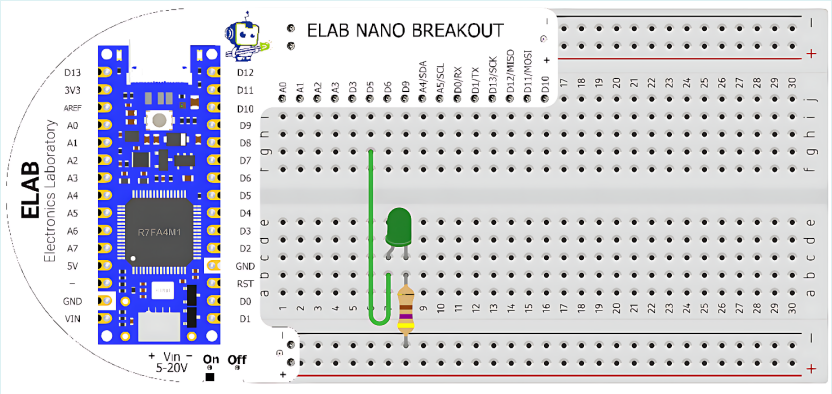
ESERCIZIO 7.4

In questo esperimento impareremo a utilizzare l'istruzione **analogWrite** per modificare la luminosità di un LED.

Materiale necessario

- Arduino NANO R4
- ELAB NANO Breakout

- 1 LED verde
- 1 resistore da 470 Ohm
- cavetti di collegamento



In questo esercizio introduciamo una nuova struttura molto utile: il **ciclo for**.

Serve per evitare di scrivere tante volte le stesse istruzioni, cioè per non fare operazioni ripetitive a mano.

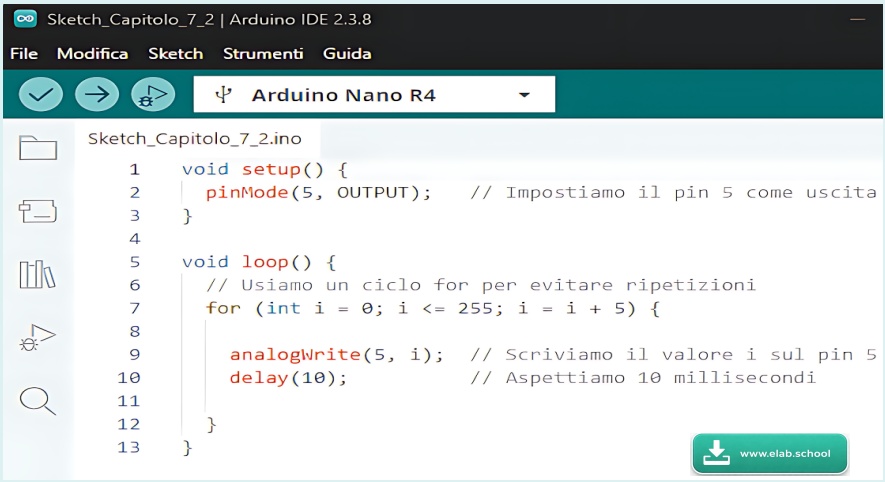
Prima di usarlo, proviamo a descrivere cosa vogliamo fare con lo **pseudocodice**. Questo ci aiuta a capire meglio il funzionamento del ciclo.

Vogliamo:

- Ripetere da 0 fino a 255
  - assegnare a una variabile il valore da scrivere sul pin PWM
  - scrivere questo valore sul pin PWM 5
  - aspettare 10 millisecondi
  - aumentare il valore di 5

Senza il ciclo for, dovremmo scrivere l'istruzione analogWrite per ogni valore da 0 a 255.

Il programma diventerebbe lunghissimo e difficile da leggere.



Grazie al **ciclo for**, invece, possiamo fare la stessa cosa con poche righe di codice, in modo molto più semplice e ordinato.

VEDIAMO COME FUNZIONA IL CICLO FOR

Per prima cosa dichiariamo una variabile intera chiamata **i**.
Potremmo chiamarla anche *Giovanni* o *valore*, ma nei cicli for si usano quasi sempre nomi brevi come **i, j, k**.

Il ciclo for è formato da tre parti, separate da punto e virgola ;:

1. **Valore iniziale**
   Qui diciamo che la variabile parte da 0:
   int i = 0

2. **Condizione**
   Il ciclo continua finché questa condizione è vera:
   i <= 255
   Questo significa che il ciclo si ripete fino a quando i è minore o uguale a 255.
   Quando i diventa 256, la condizione non è più vera e il ciclo si ferma.

3. **Incremento**
   Dopo ogni ripetizione del ciclo, aumentiamo il valore della variabile:
   i = i + 5
   (cioè aumentiamo di 5 ogni volta)

   Quindi il ciclo farà: 0, 5, 10, 15… fino a 255.

Siccome vogliamo aumentare il valore di **i** di 5 ogni volta, scriviamo:

i = i + 5

Questo significa:
prendi il valore attuale di **i** e aggiungi 5.

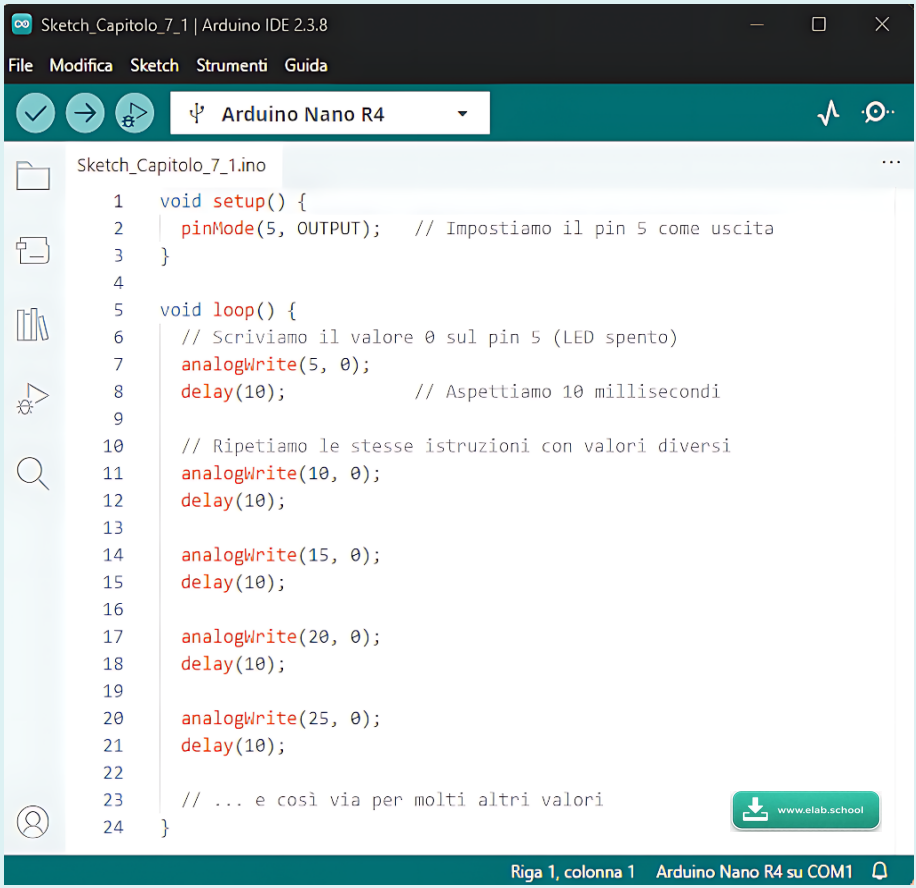
All'interno del ciclo dobbiamo poi usare questo valore. In particolare, lo scriviamo sul pin PWM che abbiamo scelto (il pin 5):

analogWrite(5, i)

Questo vuol dire:
scrivi sul pin 5 un valore proporzionale a **i**, così cambiamo la luminosità del LED.

Dopo ogni passaggio aspettiamo un pochino:

delay(10)



Questo serve a dare tempo al nostro occhio di vedere il cambiamento di luminosità del LED.
Se non aspettassimo, il cambiamento sarebbe troppo veloce!

Pensa a quante righe abbiamo risparmiato!
Senza il ciclo **for**, dovremmo scrivere una riga per ogni valore (0, 5, 10, 15… fino a 255).

Il programma diventerebbe lunghissimo e difficile da gestire.

Come puoi vedere, si ripetono sempre le stesse due istruzioni:

- analogWrite(5, qualcosa);
- delay(10);

Questo è proprio il caso tipico in cui utilizziamo un **ciclo for**.

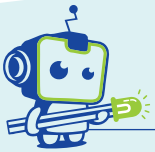
Quando dobbiamo ripetere più volte le stesse operazioni, invece di scriverle tante volte, usiamo un ciclo che le esegue automaticamente.

Queste operazioni si chiamano **iterative**, cioè che si ripetono.

Grazie al ciclo **for**, possiamo:

- scrivere meno codice
- rendere il programma più ordinato
- evitare errori

In poche parole, il ciclo for ci permette di **semplificare e ottimizzare** i nostri programmi.

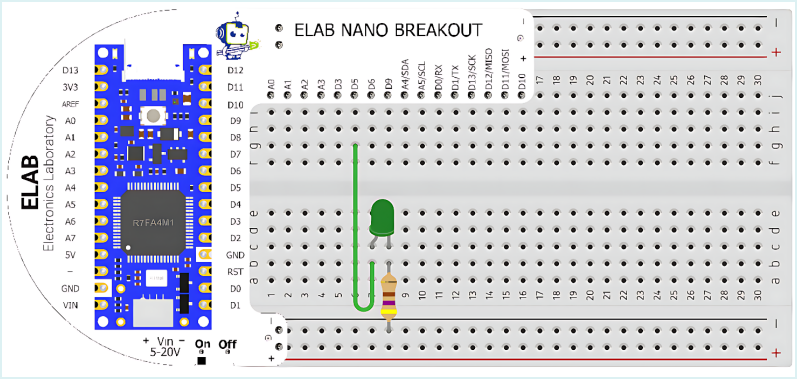


ESERCIZIO 7.5

Il circuito è lo stesso utilizzato nell'esperimento precedente.

Questa volta però modifichiamo il codice per fare una cosa diversa: accendere il LED gradualmente e poi spegnerlo allo stesso modo.

In questo modo il LED non sarà più solo acceso o spento, ma varierà la sua luminosità in modo continuo.



Nell'esperimento 4 abbiamo visto come usare il ciclo for per aumentare un valore.

Ma possiamo fare anche il contrario:
far diminuire il valore usando un ciclo for decrescente.

Se uniamo questi due cicli:

- uno crescente (accende il LED)
- uno decrescente (spegne il LED)

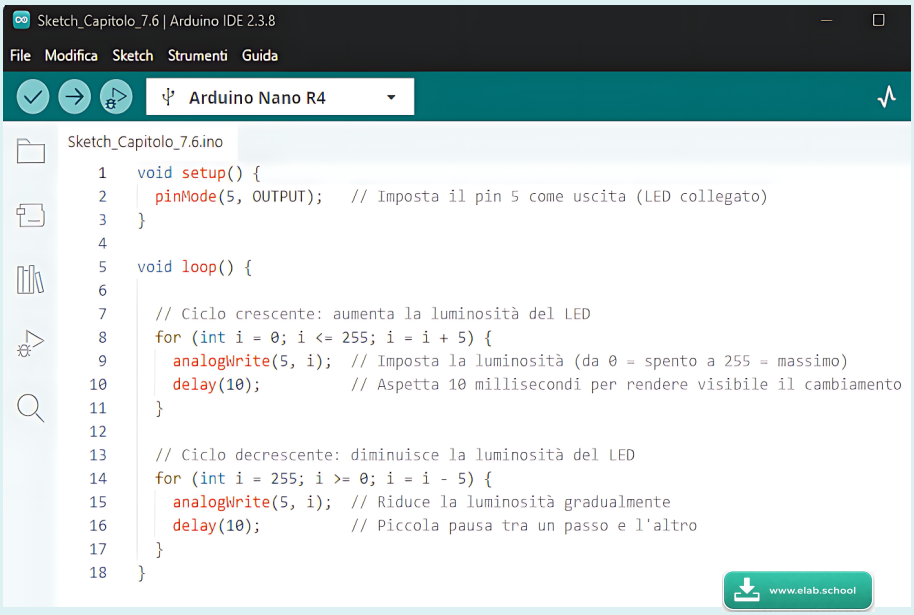
possiamo creare un effetto molto interessante chiamato:

effetto respiro

In questo effetto il LED:

- si accende lentamente
- raggiunge la massima luminosità
- poi si spegne lentamente

Vediamo ora come modificare il programma dell'esperimento 4 per aggiungere un secondo ciclo for, questa volta decrescente.



Il secondo ciclo for è molto simile al primo, ma funziona al contrario:

partiamo da **255**
arriviamo fino a **0**
e diminuiamo di **5** ogni volta

Prova a caricare il programma e osserva cosa succede al LED.

Vedrai che si accende e si spegne lentamente, creando un effetto molto fluido.

Puoi anche sperimentare:

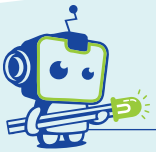
cambia il valore del delay() per rendere l'effetto più veloce o più lento

Piccola nota utile

Esiste un modo più veloce per scrivere alcune operazioni:

- invece di
  i = i + 5
  puoi scrivere
  i += 5
- invece di
  i = i - 5
  puoi scrivere
  i -= 5

Funzionano esattamente allo stesso modo, ma sono più comode da scrivere.

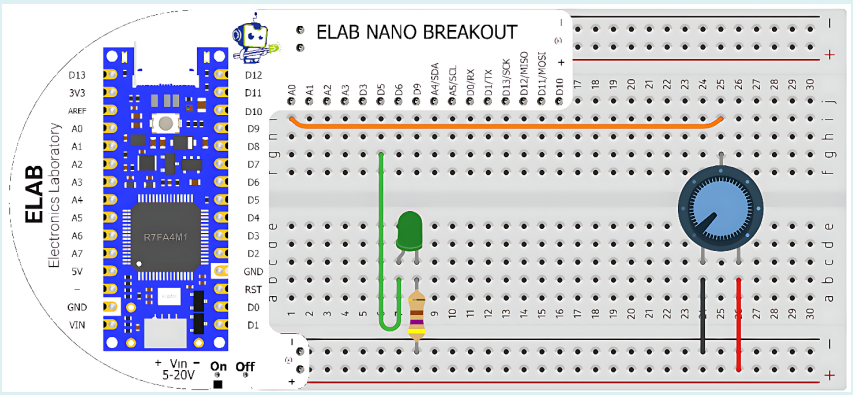


ESERCIZIO 7.6

In questo esperimento aggiungeremo un trimmer per controllare in tempo reale la luminosità del LED collegato al pin PWM 5.

Partiamo dal circuito dell'esperimento 4 e aggiungiamo semplicemente un trimmer collegato al pin **A0**.

In questo modo potremo regolare la luminosità del LED ruotando il trimmer.



Abbiamo visto che la funzione analogRead() restituisce un valore a **10 bit**, quindi compreso tra **0 e 1023**.

Abbiamo anche visto che la funzione analogWrite() utilizza valori tra **0 e 255**.

Questo significa che non possiamo usare direttamente il valore letto da analogRead().

Una soluzione semplice è dividere il valore per 4:

1023 ÷ 4 ≈ 255

Però questa soluzione funziona solo nei casi più semplici.

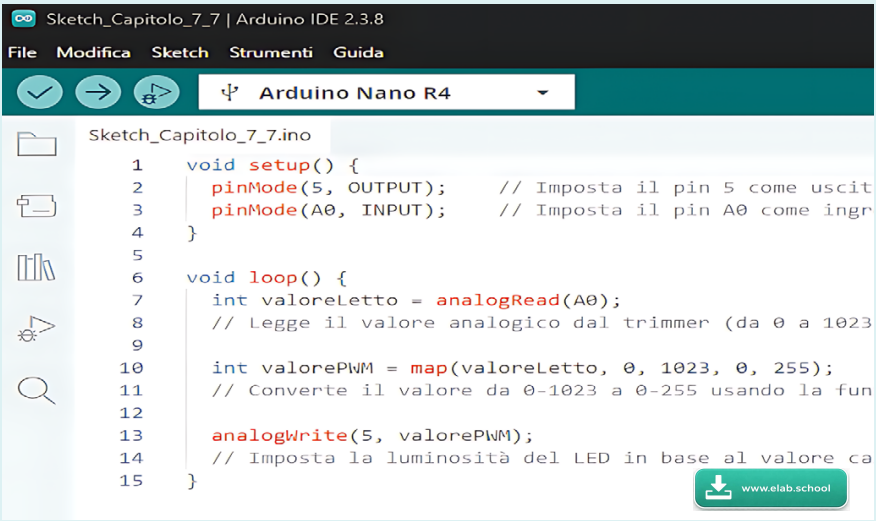
In situazioni più complesse, il calcolo non è così immediato.

Per questo motivo possiamo usare una funzione molto utile:

map()

Questa funzione permette di **trasformare un valore da un intervallo a un altro** in modo automatico.

Nel prossimo passo vedremo come usare map() per realizzare questo esercizio



Come prima, dichiariamo una variabile intera chiamata valoreLetto per memorizzare il valore restituito da analogRead().

A questo punto dobbiamo adattare questo valore.

Infatti:

- valoreLetto varia tra **0 e 1023**
- analogWrite() invece accetta valori tra **0 e 255**

Per fare questa conversione utilizziamo la funzione:

map()

Scriviamo:

int valorePWM = map(valoreLetto, 0, 1023, 0, 255);

Cosa significa questa riga?

Vuol dire:

- prendi valoreLetto (che va da 0 a 1023) e trasformalo in un valore tra 0 e 255

Il risultato viene salvato nella variabile valorePWM.

Come funziona map?

La funzione map() calcola automaticamente il rapporto tra i due intervalli.

Nel nostro caso:

- intervallo di partenza 0–1023
- intervallo di arrivo 0–255
- quindi il valore viene **ridimensionato**

In pratica (semplificando):
è come dividere per circa 4

Risultato

- giri il trimmer cambia valoreLetto
- map() lo trasforma
- analogWrite() usa il valore per regolare la luminosità del LED

Così il LED si accende e si spegne in modo proporzionale alla posizione del trimmer.

ESERCIZIO 7.7

In questo esperimento riprendiamo quanto fatto in quello precedente, ma con una differenza importante.

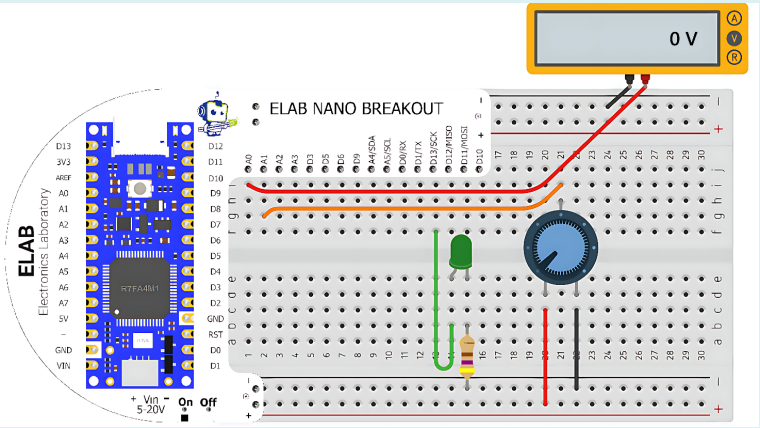
Useremo la funzione analogWrite() sul pin **A0**, che può generare una **vera tensione analogica**.

Per controllare questa tensione:
useremo un trimmer collegato al pin **A1**

Questa volta non useremo il LED.

Al suo posto collegheremo un **multimetro**, così potremo misurare direttamente la tensione in uscita dal pin A0.

Ruotando il trimmer vedremo cambiare il valore di tensione sul multimetro



In questo esperimento non abbiamo bisogno di usare map().

Questo perché nel setup() abbiamo utilizzato l'istruzione:

analogWriteResolution(10);

Questa funzione imposta la risoluzione della scrittura analogica a **10 bit**.

Ma cosa significa?

- analogRead() restituisce valori da **0 a 1023** (10 bit)
- anche analogWrite() ora lavora da **0 a 1023**

Quindi possiamo usare direttamente valoreLetto senza fare conversioni.

Attenzione importante:

Non tutte le schede Arduino funzionano così!

Alcune schede (come Arduino UNO):
non hanno un vero DAC (convertitore digitale-analogico.)
non permettono di cambiare la risoluzione

Per questo motivo spesso si usa map() per adattare i valori.

Consiglio utile:

Quando cerchi esempi online o usi l'intelligenza artificiale:

controlla sempre quale scheda Arduino stai usando
verifica se supporta analogWriteResolution()

Approfondimento

Alcune schede Arduino permettono risoluzioni anche più alte, ad esempio **12 o 14 bit**.

Più bit significa:

- più valori disponibili
- maggiore precisione
- variazioni di tensione più piccole e precise

In pratica: più bit = controllo più preciso

NOTE

VISUALIZZARE DATI: IL MONITOR SERIALE

OBIETTIVO DEL CAPITOLO

Fino ad ora Arduino ha lavorato "in silenzio". Leggeva dati, faceva calcoli e prendeva decisioni… ma noi non vedevamo nulla.

In questo capitolo imparerai a:

- vedere i dati letti da Arduino
- inviare messaggi dal PC alla scheda
- usare strumenti fondamentali per il debug

COME POSSIAMO VEDERE DAVVERO QUESTI DATI?

Quando colleghiamo Arduino al computer, succede una cosa molto interessante.

Di solito pensiamo che il PC serva solo per **inviare istruzioni ad Arduino**, cioè per caricare il programma nella scheda.

Ed è vero… ma non è tutto!

La comunicazione funziona anche al contrario.

Arduino può **mandare informazioni al computer**.
In pratica, non solo riceve ordini, ma può anche "parlare" e dirci cosa sta succedendo.

Per esempio, può inviare:

- i valori che legge dai sensori
- i risultati dei calcoli
- informazioni utili per capire se il programma funziona bene

Questi dati vengono poi **visualizzati sul PC**, così possiamo leggerli e capire meglio cosa sta facendo il nostro progetto.

Grazie a questa comunicazione possiamo fare due cose molto utili:

- **inviare comandi dal PC ad Arduino**
- **ricevere i dati elaborati da Arduino**

Questi dati possiamo:

- usarli in altri programmi
- oppure semplicemente leggerli per capire cosa sta succedendo nel nostro progetto

Questo metodo è usato molto spesso negli sketch Arduino per fare una cosa importantissima chiamata **debug**.

MA COS'È IL DEBUG?

Il debug è il processo che ci permette di:

- controllare se il programma funziona correttamente
- trovare eventuali errori
- capire dove qualcosa non va

In pratica, è come fare il "detective del codice", Arduino ci manda informazioni, noi le osserviamo… e scopriamo cosa sistemare!

Grazie al debug possiamo quindi **correggere gli errori e migliorare i nostri programmi**.

Nei programmi Arduino, questa comunicazione avviene grazie a uno strumento molto utile:
il **Serial Monitor** (Controllore Seriale).

La comunicazione che usiamo si chiama **seriale**.

È un modo molto comune per far parlare tra loro diversi dispositivi elettronici.

Non lo usa solo Arduino!
Un esempio molto semplice è la **stampante**.

Quando stampi qualcosa:

- il computer invia dati alla stampante
- questi dati possono essere **testo o immagini**
- la stampante li riceve e li trasforma in qualcosa di reale su carta

Tutto questo avviene grazie alla comunicazione seriale (spesso tramite il cavo USB).

Anche con le stampanti wireless il concetto è lo stesso:
i dati viaggiano comunque dal computer alla stampante, solo senza filo.

Allo stesso modo, Arduino usa la comunicazione seriale per:

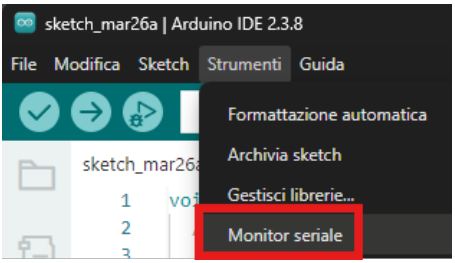
- inviare dati al computer
- ricevere comandi
- aiutarci a capire cosa sta succedendo nel programma

È come aprire una finestra dentro Arduino e vedere cosa sta facendo in tempo reale.

Prima di vedere come usare le istruzioni negli sketch Arduino per inviare o leggere dati, facciamo un passo indietro.

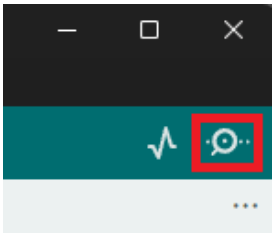
Vediamo **come aprire il Serial Monitor** dall'IDE di Arduino.

Il primo metodo consiste nel cliccare su **Strumenti** e poi su **Monitor Seriale**.

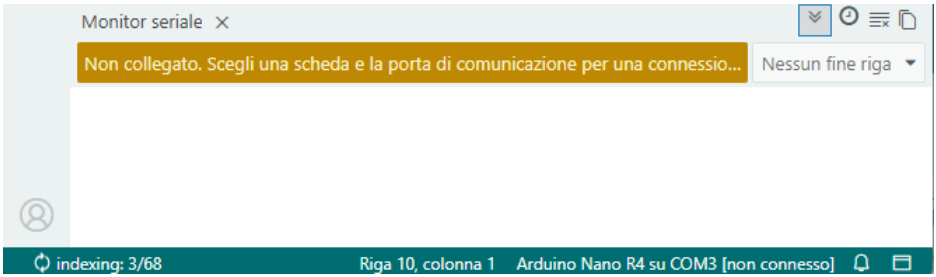


Il secondo metodo è ancora più semplice:
basta cliccare sulla **lente di ingrandimento** che si trova in alto a destra nella finestra dello sketch, proprio sotto la **X** per chiudere la finestra.

I due metodi sono **identici**:



Una volta effettuato il click con uno dei due metodi, in basso si aprirà una finestra con sfondo bianco.

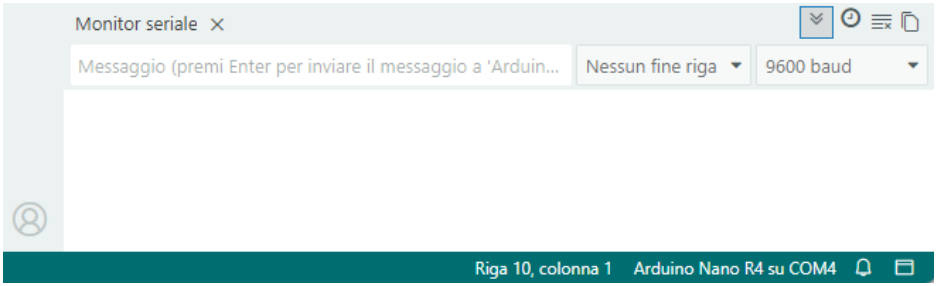


Se apri il Serial Monitor quando la scheda **non è collegata al PC** (oppure hai selezionato la **porta sbagliata**), comparirà un **messaggio di avviso**.

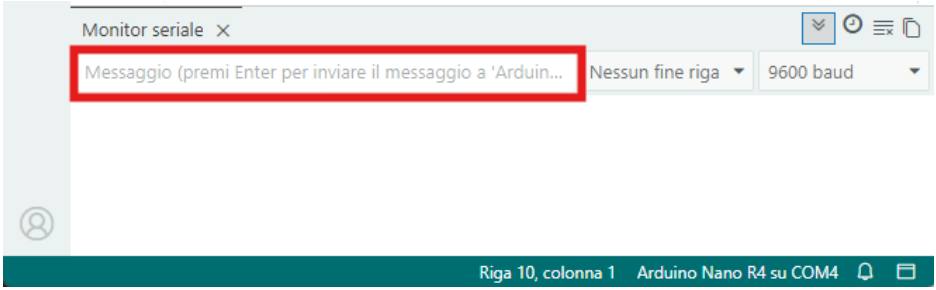
Vedrai un popup giallo che ti dice che la scheda non è connessa.

Non è un errore grave: significa solo che Arduino non riesce a comunicare con il computer.

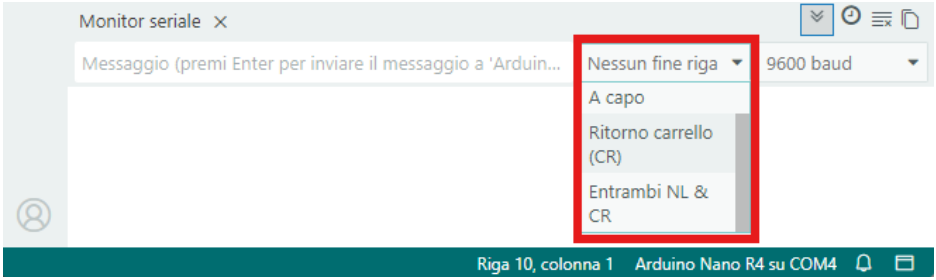
Quando la scheda è collegata, nel Serial Monitor compaiono alcune informazioni in più molto utili:



- una riga in cui puoi **scrivere e inviare un messaggio ad Arduino**
- un **menù** dove puoi scegliere se aggiungere qualcosa alla fine del messaggio (lo useremo più avanti)



- un menù in cui puoi scegliere quale carattere di fine riga aggiungere al messaggio inviato dal Monitor seriale alla scheda Arduino



- un **menù** in cui puoi scegliere la **velocità di comunicazione** tra il PC e la scheda Arduino.

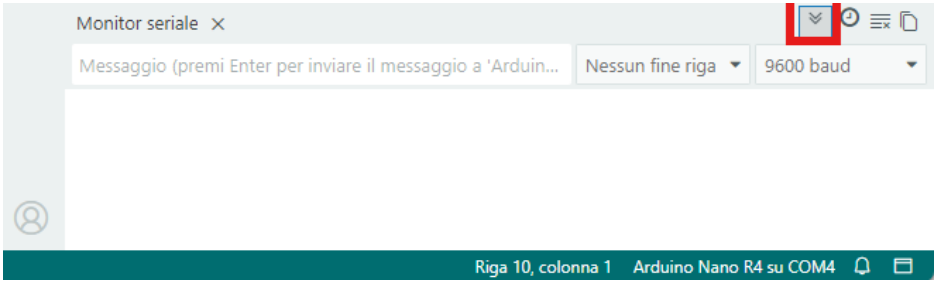
Questa velocità si misura in **baud**.

Il valore più usato è **9600**, mentre un altro molto comune è **115200**.

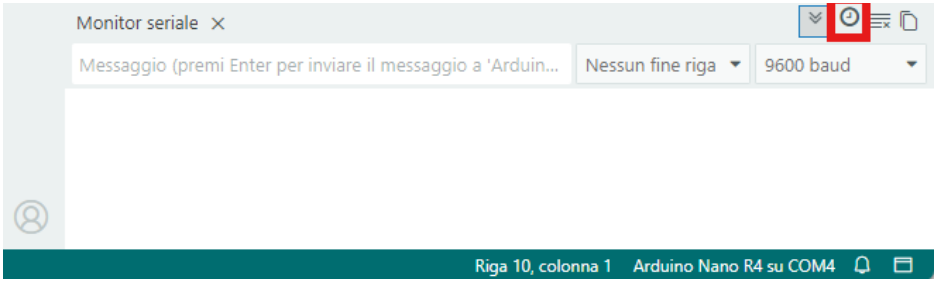
Per semplificare le cose, nei nostri esercizi useremo sempre **9600.** Così evitiamo errori e tutto funzionerà senza problemi!

In alto a destra nel Serial Monitor ci sono anche **quattro pulsanti** molto utili per gestire i dati che arrivano da Arduino:

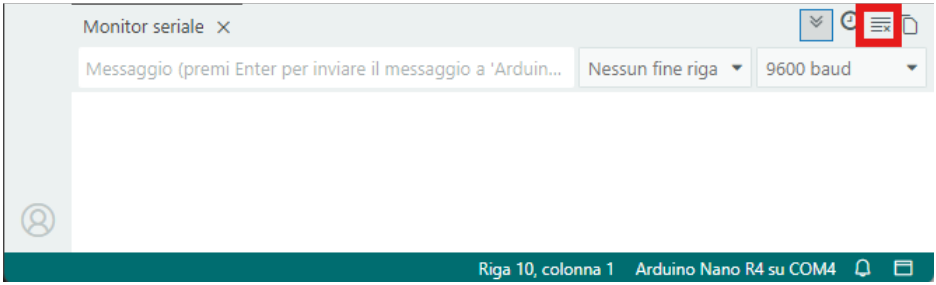
- il **primo pulsante** attiva l'**autoscorrimento**, così vedi sempre gli ultimi messaggi senza dover scorrere manualmente



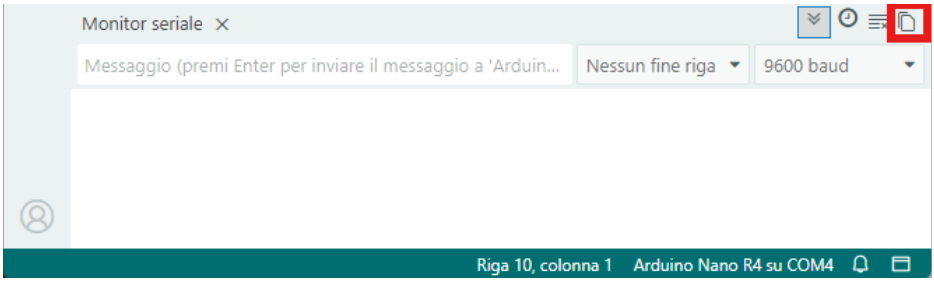
- il **secondo pulsante** permette di **mostrare o nascondere il tempo** accanto ai messaggi ricevuti.



- il **terzo pulsante** serve per **cancellare tutti i messaggi** presenti nella finestra.



- il **quarto pulsante** ti permette di **copiare tutti i messaggi** ricevuti



Questi strumenti ti aiutano a leggere meglio le informazioni e a lavorare in modo più ordinato.

Ora che abbiamo capito come funziona il Serial Monitor, possiamo iniziare a fare qualche esperimento per **inviare e ricevere dati** con Arduino!

ESERCIZIO 8.1

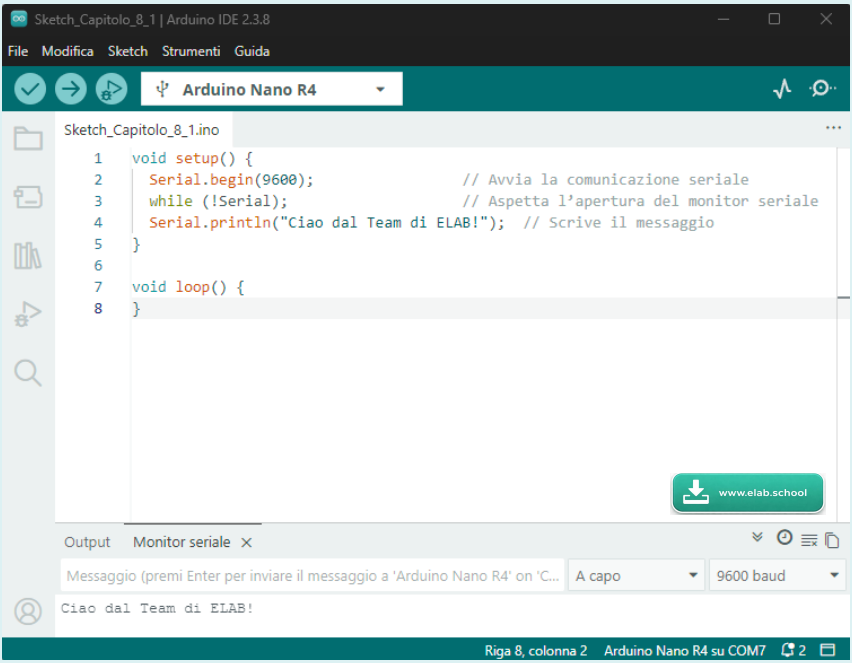
In questo esperimento faremo una cosa molto semplice ma importante: Arduino invierà un messaggio di saluto al computer!

Per farlo non serve costruire nessun circuito complicato.

Ti basta:

- collegare la scheda Arduino al PC con il **cavo USB** .

Una volta collegata la scheda, saremo pronti per vedere Arduino “parlare” con il computer!



Abbiamo scritto solo **tre righe di codice**, ma in realtà stanno succedendo molte cose nuove!

Non farti ingannare dalla semplicità: qui stiamo facendo un passo importante.

Adesso andiamo con calma e vediamo **passo passo**:

PER PRIMA COSA DOBBIAMO ATTIVARE LA COMUNICAZIONE SERIALE

```
Serial.begin(9600);
```

Questa istruzione significa:

"inizia la comunicazione tra Arduino e il computer"

- **Serial** è un "oggetto" (puoi immaginarlo come uno strumento pronto all'uso)
- **begin()** è un comando che serve per avviarlo
- **9600** è la velocità di comunicazione (baud)

Nei nostri esempi useremo sempre **9600**, anche se esistono altri valori selezionabili dal menù.

Poi troviamo questa istruzione

```
while(!Serial);
```

Significa semplicemente:
"aspetta finché la comunicazione seriale non è pronta"

In pratica:

- Arduino resta in attesa
- quando apri il Serial Monitor, allora continua

Questo serve per non perdere il messaggio iniziale!

Infine troviamo questa istruzione

```
Serial.println("Ciao dal Team di ELAB!");
```

Questa riga dice ad Arduino:
"scrivi questo messaggio nel Serial Monitor"

- **println** stampa il testo **e va a capo**
- il messaggio è scritto tra **virgolette " "**

Attenzione alle virgolette!

Le virgolette sono fondamentali perché indicano che quello che c'è dentro è **testo**.

Questo tipo di dato si chiama:
stringa (String)

- una **stringa** è una sequenza di caratteri (lettere, numeri, simboli…)
- se invece abbiamo bisogno di **un solo carattere**, usiamo il tipo **char**

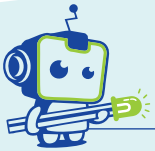
In poche parole:

- **char** = un solo carattere
- **String** = tanti caratteri messi insieme

Riassumendo:

- avviamo la comunicazione → Serial.begin(9600)
- aspettiamo che sia pronta → while(!Serial);
- inviamo un messaggio → Serial.println("...")

E così Arduino inizia a "parlare" con noi!



ESERCIZIO 8.2

Nell'esperimento precedente abbiamo scritto il messaggio dentro **setup()**.

Questo significa che:

- il messaggio viene inviato **una sola volta**
- subito dopo aver caricato il programma e aperto il Serial Monitor

Ma cosa succede se vogliamo vedere il messaggio **continuamente**?

Semplice!

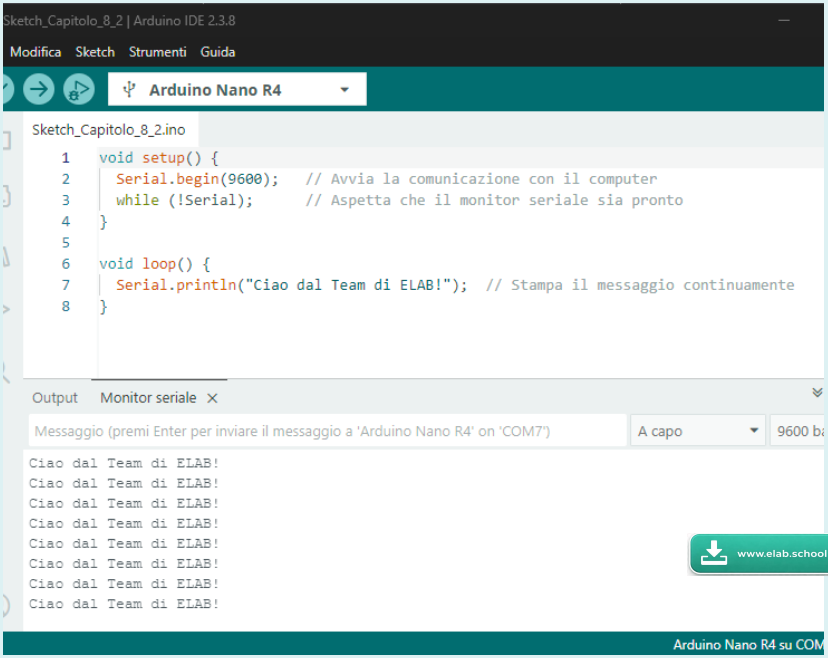
Ci basta spostare il comando dentro **loop()**.

Ricorda:

- **setup()** → si esegue una sola volta
- **loop()** → si ripete all'infinito

Quindi, mettendo il messaggio nel loop, Arduino lo invierà **senza fermarsi**, ancora e ancora.

È come dire ad Arduino:
"Continua a ripetere questo messaggio all'infinito!"



Proviamo ora a usare il pulsante che mostra l'orario accanto ai messaggi ricevuti.

Come puoi osservare facilmente, ora ogni volta che Arduino invia un messaggio vengono mostrati anche:

ore, minuti, secondi e millisecondi in cui il messaggio è stato ricevuto.

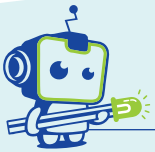
Questo è molto utile perché ti permette di:

- capire **quando arrivano i dati**
- vedere **quanto velocemente vengono inviati**

Ora prova anche tu!

- clicca sul pulsante per **disattivare l'autoscorrimento**
  → così puoi fermarti e leggere con calma i messaggi
- clicca sul pulsante per **cancellare i messaggi**
  → per ripulire la schermata e ripartire da zero

Fare pratica con questi strumenti ti aiuta a prendere confidenza con il Serial Monitor e a usarlo meglio nei prossimi esperimenti.



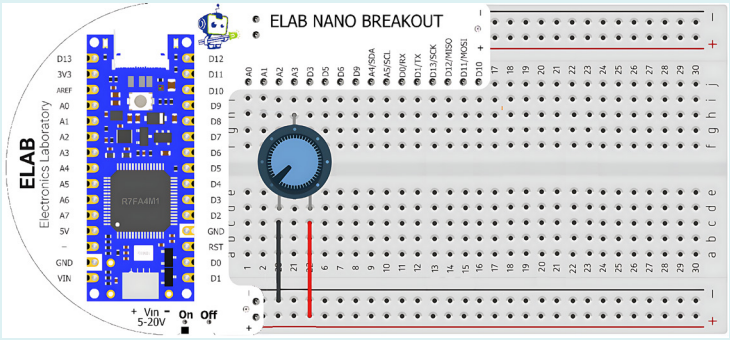
ESERCIZIO 8.3

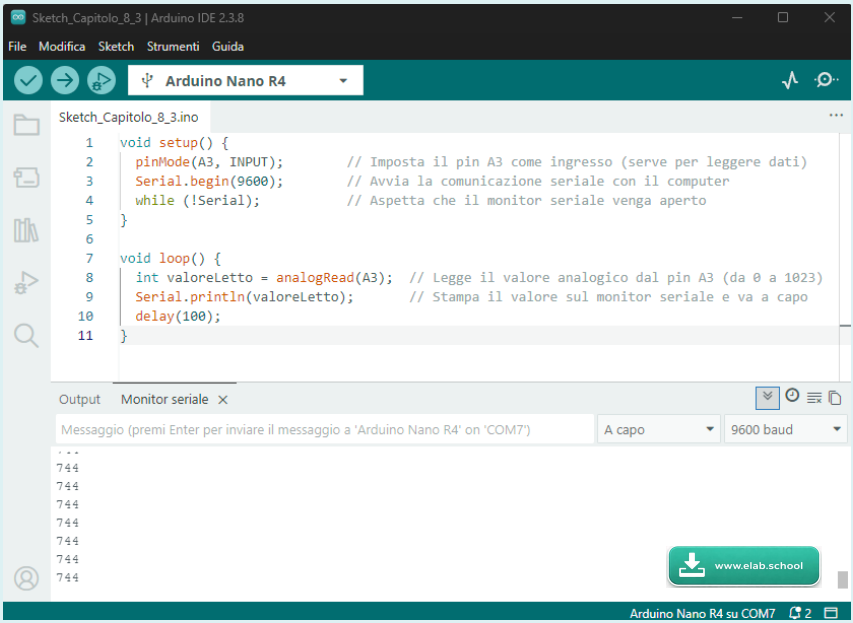
In questo esperimento vedremo come **stampare sul monitor seriale** il valore che Arduino legge da un ingresso analogico (per esempio un sensore o un potenziometro).

Per farlo abbiamo bisogno di:

- **Arduino** (qualsiasi modello, tipo UNO)
- **La breakout di Elab**

- **Cavo USB** per collegarlo al computer
- **Un potenziometro**
- **Cavetti jumper**
- **Breadboard** (facoltativa ma utile)
- **Arduino IDE** (il programma sul computer)





Per visualizzarli meglio, abbiamo aggiunto un piccolo delay dopo la stampa del valore.

Apri il **monitor seriale** e prova a ruotare il **trimmer**!
Vedrai che i valori cambiano da **0 a 1023**, proprio come abbiamo visto nella teoria.

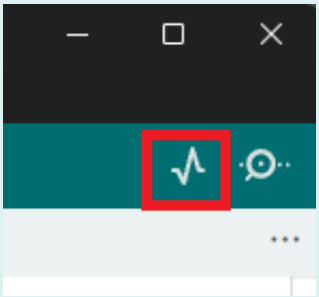
Abbiamo sempre usato il comando:

```
Serial.println()
```

Questo comando **stampa il valore e va automaticamente a capo**.

SERIAL PLOTTER (VISUALIZZAZIONE GRAFICA)

Ora vediamo un modo ancora più interessante per visualizzare i dati.



Accanto al pulsante del **Monitor Seriale**, trovi il pulsante:

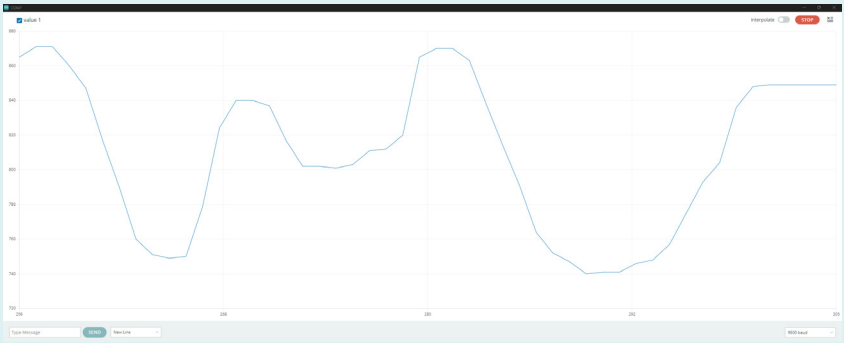
Serial Plotter

Il Serial Plotter serve per **disegnare un grafico in tempo reale** dei dati inviati da Arduino.

Per usarlo correttamente, puoi scrivere:

```
Serial.print(valoreLetto);
Serial.println();
```

In questo modo ogni valore viene mandato su una nuova riga e il grafico verrà disegnato automaticamente.



In pratica

- Monitor seriale → vedi numeri
- Serial Plotter → vedi un grafico che si muove

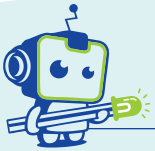
Come hai visto, nel grafico il valore visualizzato viene chiamato automaticamente **“value 1”**.
Se vogliamo, però, possiamo assegnare un nome ai grafici!
Questo è molto utile perché possiamo visualizzare **più valori contemporaneamente**.

Vedremo nel prossimo esperimento come fare.

Un’ultima cosa importante: se non l’hai già notato, per caricare un nuovo programma sulla scheda devi prima **chiudere il Serial Plotter**.
Questo succede perché la scheda sta usando la comunicazione USB e l’IDE non riesce a chiuderlo da solo.

Quindi ricordati sempre: **prima chiudi il Serial Plotter, poi carica il nuovo sketch!**

“Il Serial Plotter è utilissimo per capire l’andamento dei sensori nel tempo”

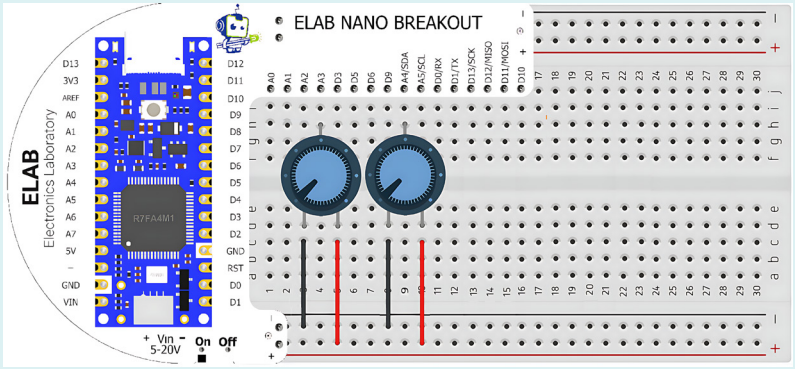


ESERCIZIO 8.4

In questo esperimento vedremo come stampare sul **Serial Plotter** il risultato della conversione analogica letta con analogRead() su due pin analogici.

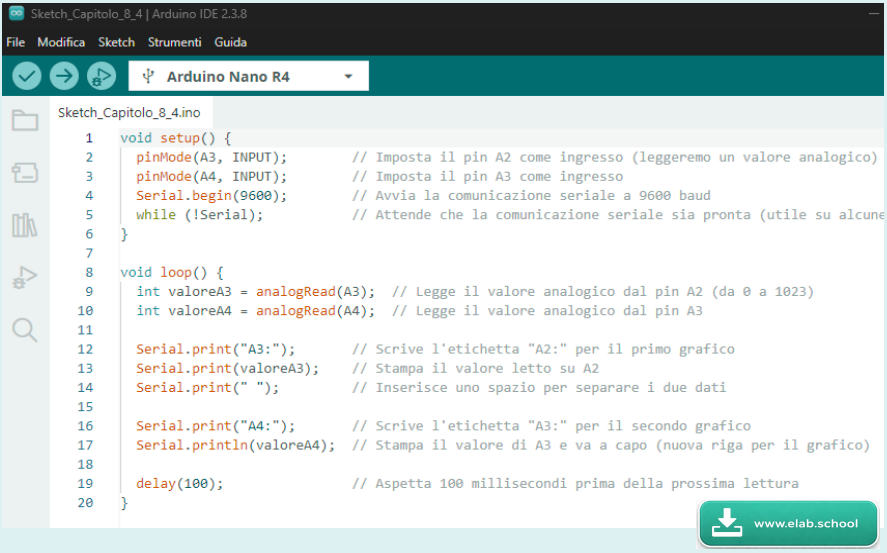
Per farlo abbiamo bisogno di:

- Arduino NANO R4
- ELAB NANO Breakout
- 4 fili colorati
- 2 trimmer



Per separare tra di loro i due grafici e assegnare delle etichette, è necessario aggiungere alcune istruzioni `Serial.print()`.

Queste servono per far capire al **Serial Plotter** che vogliamo visualizzare **due diverse serie di dati** e per associare un nome (etichetta) a ciascuna di esse.




```ino
void setup() {
  pinMode(A3, INPUT);        // Imposta il pin A2 come ingresso (leggeremo un valore analogico)
  pinMode(A4, INPUT);        // Imposta il pin A3 come ingresso
  Serial.begin(9600);        // Avvia la comunicazione seriale a 9600 baud
  while (!Serial);           // Attende che la comunicazione seriale sia pronta (utile su alcune

}

void loop() {
  int valoreA3 = analogRead(A3);  // Legge il valore analogico dal pin A2 (da 0 a 1023)
  int valoreA4 = analogRead(A4);  // Legge il valore analogico dal pin A3

  Serial.print("A3:");       // Scrive l'etichetta "A2:" per il primo grafico
  Serial.print(valoreA3);    // Stampa il valore letto su A2
  Serial.print(" ");         // Inserisce uno spazio per separare i due dati

  Serial.print("A4:");       // Scrive l'etichetta "A3:" per il secondo grafico
  Serial.println(valoreA4);  // Stampa il valore di A3 e va a capo (nuova riga per il grafico)

  delay(100);                // Aspetta 100 millisecondi prima della prossima lettura
}
```

Come puoi vedere, adesso vengono visualizzati i nomi **A2** e **A3** sulle due serie di dati, identificate dai colori **blu** e **arancione**.

Se clicchi sulla spunta accanto al nome, puoi **mostrare o nascondere** una serie di dati nel grafico.
Se invece clicchi su **STOP**, puoi **bloccare l'acquisizione** dei dati.

Prova ora a sperimentare con il **Serial Plotter**, aggiungendo più dati e assegnando nomi diversi ai grafici!

Attenzione: i nomi dei grafici **non possono contenere spazi**.

Puoi scrivere, ad esempio:
- valoreA2
- valore_A2

ma **non** valore A2.

Se inserisci uno spazio, verrà considerata solo l'ultima parte: quindi "valore A2" diventerà semplicemente **"A2"**.

Piccolo bonus: per ridurre il numero di righe di codice, puoi scrivere direttamente:

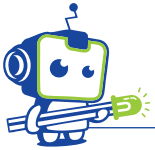
```
Serial.println(analogRead(A2));
```

senza salvare prima il valore in una variabile.

Le funzioni possono essere anche **concatenate** tra loro.
Ad esempio, puoi scrivere:

```
if (analogRead(A2) > 512)
```

senza dover salvare prima il valore in una variabile.

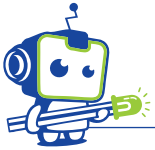


ESERCIZIO 8.5

In questo esercizio useremo un **potenziometro** per controllare dei **LED** e contemporaneamente vedremo i valori sia nel **Monitor Seriale** che nel **Serial Plotter**.

- Ruoti il potenziometro
- Il valore cambia (0 a 1023)
- In base al valore: si accendono i LED e vedi il valore nel **Monitor Seriale**
- vedi il grafico nel **Serial Plotter**

```ino
void setup() {
  pinMode(A3, INPUT);        // Imposta il pin A3 come ingresso (potenziometro)

  pinMode(12, OUTPUT);       // Imposta il pin 12 come uscita (LED 1)
  pinMode(11, OUTPUT);       // Imposta il pin 11 come uscita (LED 2)
  pinMode(10, OUTPUT);       // Imposta il pin 10 come uscita (LED 3)

  Serial.begin(9600);        // Avvia la comunicazione seriale
}

void loop() {
  int valore = analogRead(A3);  // Legge il valore del potenziometro su A3

  // Stampa per Monitor Seriale
  Serial.print("Valore: ");   // Scrive il testo "Valore:"
  Serial.println(valore);     // Scrive il valore e va a capo

  // Stampa per Serial Plotter
  Serial.print("pot:");       // Etichetta del grafico
  Serial.println(valore);     // Invia il valore al plotter

  // Controllo LED
  if (valore < 300) {
    digitalWrite(12, HIGH);  // Accende LED su pin 12
    digitalWrite(11, LOW);   // Spegne LED su pin 11
    digitalWrite(10, LOW);   // Spegne LED su pin 10
  }
  else if (valore < 700) {
    digitalWrite(12, LOW);
    digitalWrite(11, HIGH);  // Accende LED su pin 11
    digitalWrite(10, LOW);
  }
  else {
    digitalWrite(12, LOW);
    digitalWrite(11, LOW);
    digitalWrite(10, HIGH);  // Accende LED su pin 10
  }

  delay(100);                // Piccola pausa
}
```



ESERCIZIO 8.6

Qui non accendiamo più **un solo LED**, ma una **barra progressiva** (tipo volume).

Sketch_Capitolo_8_6 | Arduino IDE 2.3.8

File Modifica Sketch Strumenti Guida

Arduino Nano R4

Sketch_Capitolo_8_6.ino

```
void setup() {
  pinMode(A3, INPUT);        // Imposta il pin A3 come ingresso (potenziometro)

  pinMode(12, OUTPUT);       // LED 1
  pinMode(11, OUTPUT);       // LED 2
  pinMode(10, OUTPUT);       // LED 3

  Serial.begin(9600);        // Avvia la comunicazione seriale
}

void loop() {
  int valore = analogRead(A3);  // Legge il potenziometro

  Serial.print("Valore: ");
  Serial.println(valore);

  Serial.print("pot:");
  Serial.println(valore);

  // VALORE BASSO → 1 LED
  if (valore < 300) {
    digitalWrite(12, HIGH);  // acceso
    digitalWrite(11, LOW);   // spento
    digitalWrite(10, LOW);   // spento
  }

  // VALORE MEDIO → 2 LED
  else if (valore < 700) {
    digitalWrite(12, HIGH);  // acceso
    digitalWrite(11, HIGH);  // acceso
    digitalWrite(10, LOW);   // spento
  }

  //  VALORE ALTO → 3 LED
  else {
    digitalWrite(12, HIGH);  // acceso
    digitalWrite(11, HIGH);  // acceso
    digitalWrite(10, HIGH);  // acceso
  }

  delay(100);
}
```

www.elab.school

IL PIEZO: FACCIAMO MUSICA!

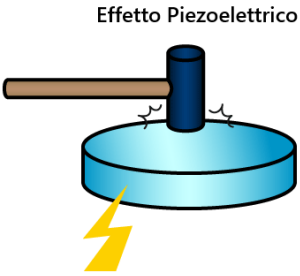
Utilizzando un componente elettronico chiamato **trasduttore piezoelettrico** (o più semplicemente **piezo**) è possibile far generare dei suoni alla nostra scheda Arduino.

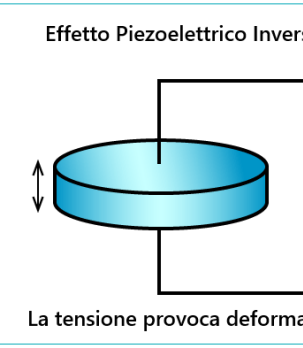
Questo componente funziona grazie a una proprietà fisica chiamata **effetto piezoelettrico**, da cui prende il nome.

L'effetto piezoelettrico si divide in due tipi:

- **effetto piezoelettrico diretto**: una deformazione meccanica genera una tensione
- **effetto piezoelettrico indiretto**: una tensione genera una deformazione meccanica

Ma cosa significa in pratica?





Con l'**effetto diretto**, se colpiamo un materiale piezoelettrico, si genera una tensione proporzionale alla forza del colpo. Questo principio viene usato, ad esempio:

- nelle chitarre acustiche per amplificare il suono
- nei pad delle batterie elettroniche per capire quale superficie è stata colpita

Con l'**effetto indiretto**, invece, se applichiamo una tensione (ad esempio usando il PWM di Arduino), il materiale vibra.
Questa vibrazione produce suono: è proprio questo effetto che useremo per far suonare il piezo.

Lo stesso principio viene utilizzato anche negli **orologi al quarzo** per scandire il tempo.

Ovviamente, usando questo sistema non possiamo riprodurre musica complessa come quella di uno strumento vero, ma possiamo creare **melodie semplici**, come quelle dei videogiochi degli anni '80 (ad esempio *Super Mario Bros!*).

Prima di iniziare con gli esperimenti, è importante capire un concetto fondamentale legato ai suoni.

Ogni suono è caratterizzato da un parametro chiamato **frequenza**.
La frequenza si misura in **Hertz (Hz)** e si indica con la lettera **f**.

Per esempio, la nota **LA centrale** del pianoforte ha una frequenza di **440 Hz**.
Possiamo quindi scrivere:
f = 440 Hz

Le frequenze delle note sono fisse, quindi possiamo usarle come valori già pronti nei nostri programmi.

Questo ci permette di introdurre un nuovo concetto: la direttiva **#define**.

Le direttive **#define**, a differenza delle variabili, servono per creare delle **costanti**, cioè valori che non cambiano.
Di solito vengono scritte all'inizio del programma, così sono facili da trovare e modificare.

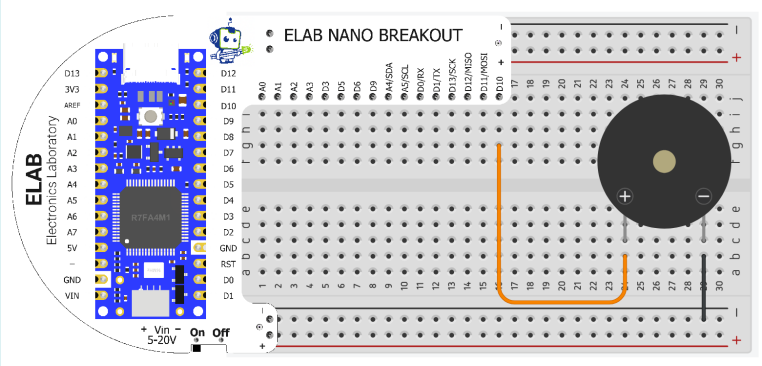
Usare le **define** è molto utile: invece di scrivere lo stesso numero tante volte nel codice, possiamo scriverlo una sola volta.

ESERCIZIO 9.1

In questo esperimento faremo suonare alla nostra scheda **Arduino NANO R4** la nota **LA centrale**, cioè a **440 Hz**.

Per farlo abbiamo bisogno di:

- Arduino NANO R4
- ELAB NANO Breakout
- Un piezo
- La breadboard
- 2 cavetti colorati



Attenzione alla polarità del piezo!
Sul componente sono indicati il **+** e il **-**:

- il **+** va collegato al pin di Arduino che useremo per controllarlo
- il **-** va collegato al negativo (GND) della breadboard

Per far suonare il piezo useremo l'istruzione **tone()** di Arduino.
Questa funzione permette di generare un suono con la **frequenza desiderata** su un determinato pin.

Se non specifichiamo la durata, il suono continuerà a essere emesso finché non useremo l'istruzione **noTone()**, che lo spegne.

QUESTO È PRATICAMENTE IL BLINK... MA CON L'AUDIO!

Proprio come abbiamo fatto per accendere e spegnere un LED, qui facciamo la stessa cosa con il piezo: lo accendiamo e lo spegniamo per produrre un suono.

Nel **setup**, come al solito, indichiamo quale pin vogliamo usare come uscita.
In questo caso abbiamo scelto il **pin 10**, ma puoi usare anche altri pin senza problemi.

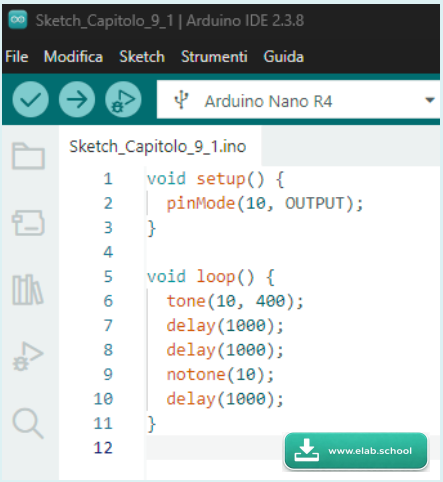
Nel **loop** succede questo:

- usiamo l'istruzione tone(10, 440) per generare un suono di **440 Hz** sul pin 10
- aspettiamo **1 secondo**
- spegniamo il suono con noTone(10)
- aspettiamo ancora **1 secondo**

E poi il ciclo ricomincia!

Prova tu:

- cambia la **frequenza** per ottenere suoni diversi
- modifica i **tempi di attesa** per cambiare il ritmo
- oppure prova a usare il **terzo parametro** della funzione tone, che serve per indicare la **durata del suono**

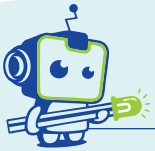


Utilizzando il parametro della **durata**, possiamo evitare di usare noTone().
Infatti, quando il tempo impostato finisce, il suono si spegne automaticamente.

Questo modo di usare la funzione è molto utile, perché ci permette di creare delle vere e proprie **suonerie** con il nostro piezo!

Prova tu:
sperimenta con le istruzioni che hai appena imparato e prova a creare una tua **suoneria personalizzata**.

Dai spazio alla tua creatività e inventa qualcosa di originale!

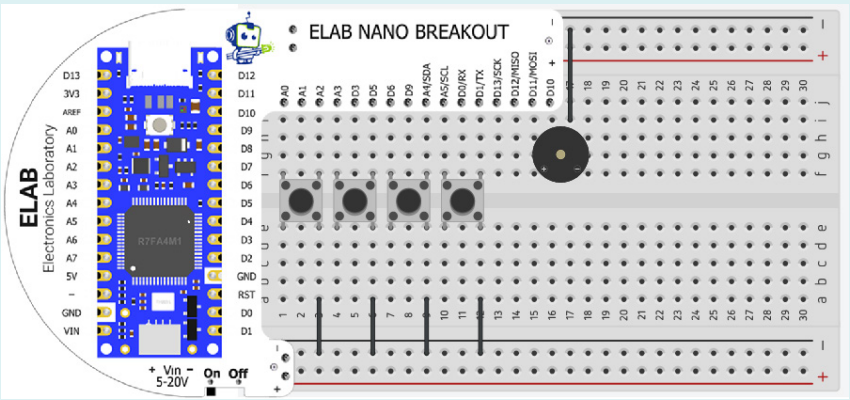


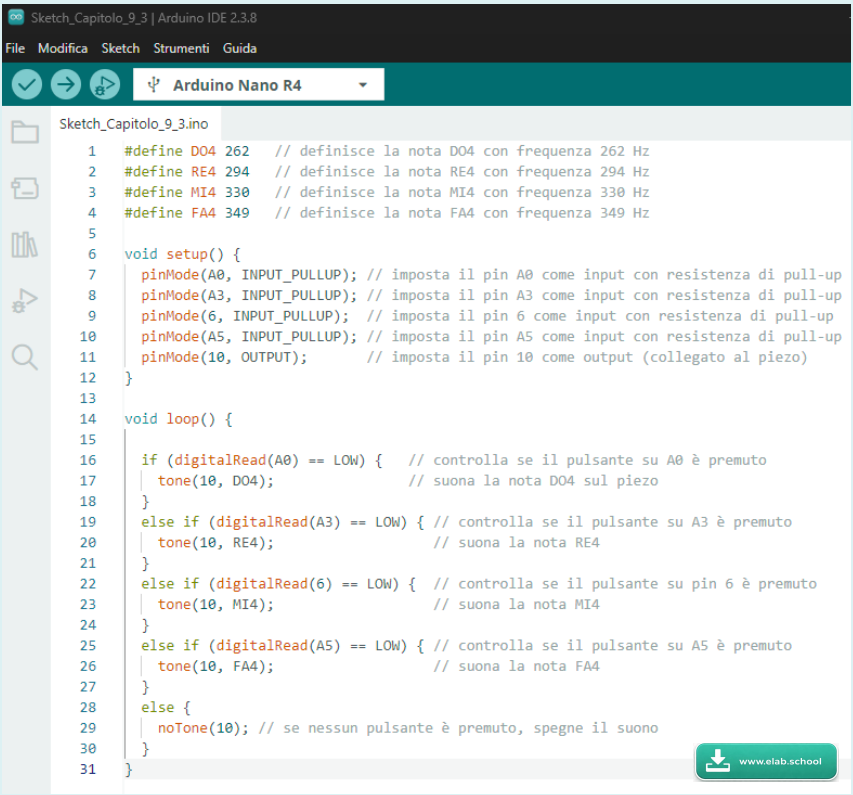
ESERCIZIO 9.2

In questo esperimento costruiremo un piccolo **pianoforte** utilizzando la funzione tone() e alcuni pulsanti

Per realizzarlo abbiamo bisogno di:

- Arduino NANO R4
- ELAB NANO Breakout
- un piezo
- 4 pulsanti
- una breadboard
- 5 cavetti colorati






```
#define DO4 262   // definisce la nota DO4 con frequenza 262 Hz
#define RE4 294   // definisce la nota RE4 con frequenza 294 Hz
#define MI4 330   // definisce la nota MI4 con frequenza 330 Hz
#define FA4 349   // definisce la nota FA4 con frequenza 349 Hz

void setup() {
  pinMode(A0, INPUT_PULLUP); // imposta il pin A0 come input con resistenza di pull-up
  pinMode(A3, INPUT_PULLUP); // imposta il pin A3 come input con resistenza di pull-up
  pinMode(6, INPUT_PULLUP);  // imposta il pin 6 come input con resistenza di pull-up
  pinMode(A5, INPUT_PULLUP); // imposta il pin A5 come input con resistenza di pull-up
  pinMode(10, OUTPUT);       // imposta il pin 10 come output (collegato al piezo)
}

void loop() {

  if (digitalRead(A0) == LOW) {   // controlla se il pulsante su A0 è premuto
    tone(10, DO4);                // suona la nota DO4 sul piezo
  }
  else if (digitalRead(A3) == LOW) { // controlla se il pulsante su A3 è premuto
    tone(10, RE4);                   // suona la nota RE4
  }
  else if (digitalRead(6) == LOW) {  // controlla se il pulsante su pin 6 è premuto
    tone(10, MI4);                   // suona la nota MI4
  }
  else if (digitalRead(A5) == LOW) { // controlla se il pulsante su A5 è premuto
    tone(10, FA4);                   // suona la nota FA4
  }
  else {
    noTone(10); // se nessun pulsante è premuto, spegne il suono
  }
}
```

Come puoi vedere, all'inizio del programma abbiamo definito le **note musicali** utilizzando il loro valore di frequenza.
Se vuoi aggiungere altre note, puoi trovare facilmente i loro valori con una semplice ricerca su internet.

Attenzione: non è possibile scrivere più #define sulla stessa riga.
Devi scrivere **una direttiva #define per ogni costante** che vuoi creare.

Il nostro "pianoforte" ha **quattro tasti**, quindi utilizziamo quattro pin impostati come INPUT_PULLUP.
Questi pin servono per capire quale pulsante viene premuto.

L'unico **OUTPUT** del programma è il piezo, che abbiamo collegato al **pin 10**.

Nel loop, il programma controlla continuamente quale dei quattro pulsanti viene premuto.
In base al pulsante premuto, viene riprodotta la **nota corrispondente**.

In questo esempio:

- il tasto più a sinistra suona il **DO4**
- il tasto più a destra suona il **FA4**

Questo pianoforte è **monofonico**, cioè può suonare **una sola nota alla volta**.

Se provi a premere più tasti insieme, noterai che viene suonata sempre la nota del **tasto più a sinistra**.
Questo succede perché il programma controlla gli if in ordine: appena trova una condizione vera, esegue quella e ignora le altre.

Prova tu:
aggiungi altri tasti e associa nuove note!

Partendo da questo esempio, sarà molto semplice espandere il tuo pianoforte.

IL PROGETTO FINALE: IL GIOCO SIMON

Siamo arrivati all'ultimo capitolo… e ora è il momento di fare qualcosa di davvero speciale!

Fino ad ora abbiamo imparato a:

- accendere LED
- leggere pulsanti
- usare il monitor seriale
- controllare la luminosità
- generare suoni con il cicalino

Adesso useremo **tutte queste conoscenze insieme** per costruire un vero gioco elettronico: **il gioco Simon**

COS'È IL GIOCO SIMON?

Il Simon è un gioco di memoria molto famoso.

Funziona in modo semplice:

- il dispositivo mostra una sequenza di luci e suoni
- il giocatore deve ripeterla premendo i pulsanti giusti
- ogni volta la sequenza diventa più lunga

Più vai avanti… più diventa difficile!

Se sbagli anche solo una volta…
Game Over!

Realizzeremo un sistema che sarà in grado di:

- generare una sequenza casuale
- accendere LED in ordine
- emettere suoni diversi
- leggere i pulsanti premuti
- controllare se la sequenza è corretta

COMPONENTI NECESSARI

- Arduino
- 4 LED
- 4 resistori
- 4 pulsanti
- 1 cicalino (buzzer)
- Breadboard
- Cavetti

ASSEMBLAGGIO DEL CIRCUITO

Iniziamo ad assemblare il nostro progetto.

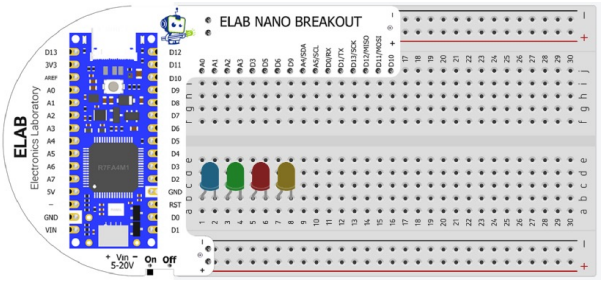
Prendiamo la breadboard e inseriamo la nostra breakout board con Arduino Nano R4 già montata.

Ora iniziamo a posizionare 4 LED colorati sulla breadboard.

Attenzione alla polarità:
tutti gli anodi (la gamba lunga) devono essere orientati dalla stessa parte.

Inseriamo quindi gli anodi dei LED nelle seguenti colonne:

ANODO	CATODO
b1	b2
b3	b4
b5	b6
b7	b8

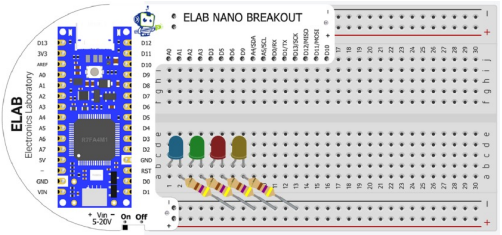


COLLEGAMENTO DELLE RESISTENZE

Ogni LED DEVE avere una resistenza, altrimenti rischi di bruciarlo.

Per ogni LED:

- Collega una resistenza (220Ω o 330Ω o 470Ω) alla gamba corta (catodo)
- Porta l'altra estremità della resistenza alla massa (GND) della breadboard

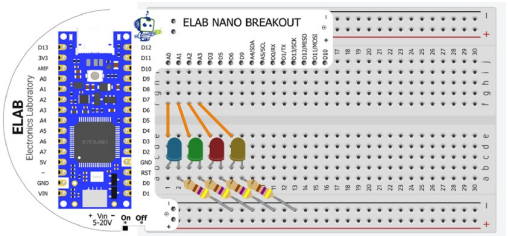


COLLEGAMENTO AI PIN DI ARDUINO

Ora colleghiamo gli anodi (gamba lunga) ai pin di Arduino.

Usiamo questi pin:

- LED in e1 → pin A0
- LED in e3 → pin A1
- LED in e5 → pin A2
- LED in e7 → pin A3



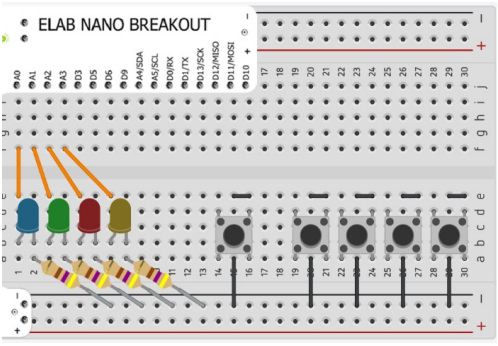
Collega con dei cavetti:

- dalla riga del LED → al pin Arduino corrispondente

INSERIMENTO DEI PULSANTI

Ora inseriamo i 5 pulsanti sulla breadboard.

Ti faccio vedere un modo semplice per collegarli, senza metterli a cavallo della fessura centrale della breadboard.



Posizioniamo ogni pulsante così:

- i due pin inferiori nella riga A
- i due pin superiori nella riga D

In questo modo il pulsante occupa una sola metà della breadboard ed è più semplice da gestire.

Per mantenere la breadboard più pulita possibile, facciamo così:

- per ogni pulsante, colleghiamo con un ponticello uno dei pin alla colonna centrale del pulsante (la stessa linea dove è inserito)

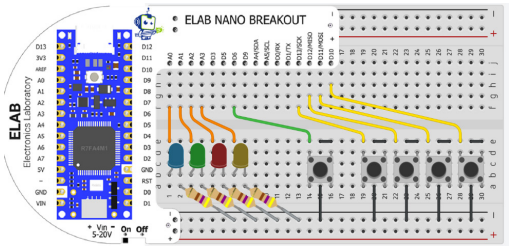
In questo modo:
portiamo il segnale del pulsante su una linea comoda da usare, senza incrociare troppi fili.

COLLEGAMENTO DEI PULSANTI AD ARDUINO

Ora colleghiamo i pulsanti ai pin digitali di Arduino.

Usiamo questi collegamenti:

- Pulsante 1 → **D13**
- Pulsante 2 → **D12**



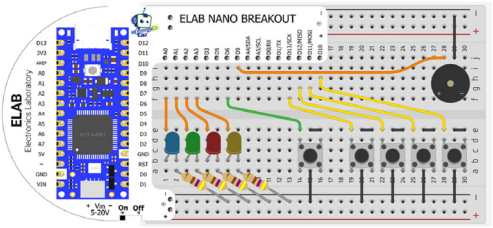
- Pulsante 3 → **D11**
- Pulsante 4 → **D10**

Colleghiamo con dei cavetti:

COLLEGAMENTO DEL BUZZER

Colleghiamolo così:

- **pin positivo (+)** → **D9** di Arduino
- **pin negativo (–)** → **GND**



PROGRAMMIAMO UN GIOCO! SIMON DICE

In questo capitolo metteremo insieme tutto quello che abbiamo imparato nei capitoli precedenti per creare un piccolo gioco interattivo: **Simon Dice**.

Il gioco consiste nel riprodurre una sequenza di luci e suoni che diventa, turno dopo turno, sempre più lunga e veloce!
È un gioco divertente che aiuta ad allenare la **memoria** e i **riflessi**.

Per realizzarlo utilizzeremo:

- dei **pulsanti**
- dei **LED**
- un **piezo** per riprodurre le note

Durante questo progetto introdurremo anche un nuovo concetto di programmazione che ci permette di scrivere codice più ordinato e con meno ripetizioni: il **vettore**.

Abbiamo già utilizzato il ciclo **for** per semplificare operazioni ripetitive.
Ma c'è una cosa che, fino ad ora, non abbiamo mai ottimizzato: l'impostazione dei pin come **INPUT** o **OUTPUT**.

Infatti, abbiamo sempre scritto tante righe come:

```
pinMode(pin, INPUT);
pinMode(pin, OUTPUT);
```

Una riga per ogni pin… ma è un po' uno spreco!

Non sarebbe meglio usare un ciclo **for** per impostare tutti i pin automaticamente?
È proprio qui che entrano in gioco i **vettori**.

Un vettore è un insieme di variabili dello stesso tipo, raccolte sotto un unico nome.
Ogni elemento del vettore può essere identificato tramite un numero chiamato **indice**.

Come si dichiara un vettore?

Nel nostro gioco utilizzeremo **4 pulsanti**, ciascuno collegato a un pin di Arduino.

I pin sono numeri interi, quindi useremo un vettore di tipo **int**:

```
const int pulsanti[] = {13, 12, 11, 10};
```

Vediamo cosa significa:

- **const** → indica che è una **costante** (non cambia mai)
- **int** → tipo di dato (numeri interi)
- **pulsanti[ ]** → nome del vettore
- **{13, 12, 11, 10}** → valori contenuti nel vettore

In pratica abbiamo creato una **lista di pin**.

Come funziona l'indice?

Immaginiamo il vettore come una fila di caselle:

- pulsanti[0] → 13

- pulsanti[1] → 12
- pulsanti[2] → 11
- pulsanti[3] → 10

Attenzione: in programmazione gli indici partono da **0**, non da 1!

Quindi un vettore di 4 elementi ha indici da **0 a 3**.

Usare il vettore con il ciclo for:

Ora arriva la parte interessante.

Possiamo usare il vettore dentro un ciclo **for** per impostare tutti i pin:

```
for (int i = 0; i < 4; i++) {
pinMode(pulsanti[i], INPUT_PULLUP);
}
```

Questo codice fa esattamente la stessa cosa di:

```
pinMode(13, INPUT_PULLUP);
pinMode(12, INPUT_PULLUP);
pinMode(11, INPUT_PULLUP);
pinMode(10, INPUT_PULLUP);
```

Ma è molto più **ordinato**, **veloce da scrivere** e **facile da modificare**.

Bello, no? Stiamo iniziando a programmare in modo più intelligente!

Variabili globali e locali

Prima di iniziare a costruire il gioco, ci serve ancora un ultimo concetto.

Le variabili (o costanti) possono essere:

- **globali** → utilizzabili in tutto il programma
- **locali** → utilizzabili solo dentro una funzione

Se dichiariamo una variabile **prima del setup()**, sarà globale.
Se invece la dichiariamo **dentro una funzione**, sarà locale.

Nel nostro caso, i vettori saranno **globali**, perché ci serviranno in tutto il programma.

Ora siamo pronti!
Buttiamoci nella realizzazione del nostro gioco **Simon Dice**.

Il codice sarà un po' più lungo rispetto agli esempi precedenti, ma lo analizzeremo passo dopo passo.

SIMON DICE

Passiamo alla realizzazione del nostro gioco interattivo!

Per costruirlo abbiamo bisogno dei seguenti componenti:

- Arduino NANO R4
- ELAB NANO Breakout
- Un piezo (cicalino)
- 5 pulsanti
- 4 LED
- Una breadboard
- Alcuni cavetti colorati

SCRIVIAMO IL NOSTRO PROGRAMMA:

Iniziamo dichiarando alcune **costanti** utilizzando la direttiva #define.

```
#define NOTA_C4 262
#define NOTA_D4 294
#define NOTA_E4 330
#define NOTA_F4 349
#define NUMERO_PULSANTI 4
#define LIVELLO_MASSIMO 100
```

In particolare:

- definiamo le note musicali (C4, D4, E4, F4) con la loro frequenza
- definiamo il numero di pulsanti utilizzati nel gioco
- definiamo il livello massimo raggiungibile

La costante NUMERO_PULSANTI è molto importante perché verrà utilizzata nei cicli for.
In questo modo evitiamo di scrivere ogni volta < 4 nel codice.

Questo ha un grande vantaggio:
se in futuro vogliamo aggiungere un altro pulsante, ci basta:

- collegarlo fisicamente
- modificare il valore della #define

Il resto del codice continuerà a funzionare senza modifiche!

Attenzione:
nelle #define non si possono usare spazi, per questo motivo utilizziamo il carattere _ (underscore).

Lo stesso vale per LIVELLO_MASSIMO.

```
7.
8.  const int piezo = 9;
9.  const int pulsanteDiAvvio = 6;
10. const int pulsanti[ ] = {13, 12, 11, 10};
11. const int luci[ ] = {A0, A1, A2, A3};
12. const int note[ ] = {NOTA_C4, NOTA_D4, NOTA_E4, NOTA_F4};
```

Successivamente dichiariamo tutte le **costanti** che utilizzeremo nel programma.

Per prima cosa assegniamo un nome ai pin:

- il **piezo**, collegato al pin 9
- il **pulsante di avvio**, collegato al pin 6

Questo è molto utile perché evita di scrivere direttamente i numeri nel codice (ad esempio dentro pinMode o digitalRead), rendendo il programma più chiaro e leggibile.

Dichiariamo poi alcuni **vettori**:

- pulsanti[ ] → contiene i pin dei 4 pulsanti (13, 12, 11, 10)
- luci[ ] → contiene i pin dei LED (A0, A1, A2, A3)

I LED sono associati ai pulsanti nello **stesso ordine**:

- pulsante[0] → luce[0]
- pulsante[1] → luce[1]
- e così via

Questo è fondamentale per far funzionare correttamente il gioco!

Infine, dichiariamo il vettore note[ ], che contiene le frequenze delle note da riprodurre.

Ogni pulsante (e quindi ogni LED) è associato a una nota:

- quando premi un pulsante → si accende il LED → viene riprodotto un suono

Nota sulle note musicali:
Abbiamo utilizzato la **notazione americana**, perché è la più diffusa su internet:

- DO = C
- RE = D
- MI = E
- FA = F
- SOL = G
- LA = A
- SI = B

```
13. int sequenza[LIVELLO_MASSIMO];
14. int livello = 1;
15. int velocita = 900;
16. bool giocoInCorso = false;
```

Successivamente dichiariamo le **variabili globali** che useremo nel programma.

Sono chiamate *globali* perché possono essere utilizzate da **tutte le funzioni** del codice.

Il vettore sequenza

```
int sequenza[LIVELLO_MASSIMO];
```

Questo è un altro modo di dichiarare un **vettore**.

In questo caso:

LIVELLO_MASSIMO vale 100

quindi il vettore può contenere **100 elementi**

A differenza di prima, qui **non assegniamo subito dei valori**.
Gli elementi del vettore verranno riempiti più avanti nel programma.

La variabile livello

```
int livello = 1;
```

Questa variabile indica il **livello di gioco**.

Partiamo dal livello 1 e aumenterà ogni volta che il giocatore completa correttamente la sequenza.

La variabile velocita

```
int velocita = 900;
```

Questa variabile rappresenta il **tempo di attesa (delay)** tra un segnale e l'altro.

Più il valore è alto → più il gioco è lento
Più il valore diminuisce → più il gioco diventa veloce

Infatti, andando avanti nei livelli, la velocità aumenterà!

Nota: nel codice non possiamo usare lettere accentate, quindi scriviamo velocita invece di velocità.

La variabile giocoInCorso

```
bool giocoInCorso = false;
```

Questa è una variabile di tipo **booleano**, cioè può assumere solo due valori:

- true (vero)
- false (falso)

Serve per capire se il gioco è attivo oppure no.

All'inizio è false perché il gioco non è ancora iniziato.
Diventerà true quando il giocatore preme il pulsante di avvio

```
17.
18. void setup() {
19.   for (int i = 0; i < NUMERO_PULSANTI; i++) {
20.     pinMode(pulsanti[i], INPUT_PULLUP);
21.     pinMode(luci[i], OUTPUT);
22.   }
23.
24.     pinMode(pulsanteDiAvvio, INPUT_PULLUP);
25.     pinMode(piezo, OUTPUT);
26.
27.     randomSeed (analogRead(A7));
28.
29.     spegniTutteLeLuci();
30. }
```

Nel **setup()** scriviamo tutte le istruzioni che definiscono lo **stato iniziale** del programma.

Impostazione dei pin con il ciclo for

Per prima cosa utilizziamo un ciclo **for** per impostare:

- i pulsanti come INPUT_PULLUP
- i LED come OUTPUT

Abbiamo detto che i vettori partono da **0** e arrivano fino a **numero elementi - 1**.

In questo caso:

- NUMERO_PULSANTI = 4
- quindi i valori di " i "saranno: **0, 1, 2, 3**

Questo ci permette di configurare tutti i pin in poche righe, senza ripetere il codice.

Pulsante di avvio

```
pinMode(pulsanteDiAvvio, INPUT_PULLUP);
```

Il pulsante di avvio viene gestito separatamente perché **non è incluso nel vettore pulsanti[ ]**.

Piezo

```
pinMode(piezo, OUTPUT);
```

Impostiamo il piezo come **OUTPUT**, perché deve generare suoni.

Casualità del gioco

```
randomSeed(analogRead(A7));
```

Questa istruzione serve a rendere il gioco **sempre diverso**.

- randomSeed() → imposta il punto di partenza della casualità
- analogRead(A7) → legge un valore "casuale" da un pin non collegato

Senza questa riga, Arduino genererebbe sempre la stessa sequenza!

Nota importante:
In precedenza si usa spesso:

```
randomSeed(millis());
```

ma usare un pin analogico scollegato (analogRead) è **più casuale e quindi migliore**.

Spegnere tutte le luci

```
spegniTutteLeLuci();
```

Questa è una funzione creata da noi.

Serve per spegnere tutti i LED all'inizio del programma.

Creare funzioni è molto importante perché:

- evita di ripetere codice
- rende il programma più ordinato
- facilita le modifiche

Come puoi immaginare, nel gioco dovremo spegnere tutte le luci più volte, quindi conviene avere una funzione dedicata.

```
31.
32. void loop() {
33.   if (!giocoInCorso && digitalRead(pulsanteDiAvvio) == LOW) {
34.     delay(50);
35.     creaSequenza();
36.     giocoInCorso = true;
37.   }
38.
39.   if (giocoInCorso) {
40.     mostraSequenza();
41.     turnoDelGiocatore();
42.   }
43. }
```

Il **loop()** è il cuore del programma: viene eseguito continuamente da Arduino.

In questo caso è molto semplice, perché utilizziamo delle funzioni create da noi per rendere il codice più ordinato e leggibile.

Avvio del gioco

Per prima cosa controlliamo se il gioco **non è ancora iniziato** e se viene premuto il pulsante di avvio:

```
if (!giocoInCorso && digitalRead(pulsanteDiAvvio) == LOW)
```

Questa condizione significa:

- !giocoInCorso → il gioco **non è in corso**
- digitalRead(...) == LOW → il pulsante è premuto

L'operatore && significa **AND (E)**: entrambe le condizioni devono essere vere.

Se queste condizioni sono soddisfatte:

- creaSequenza();
  giocoInCorso = true;
- creaSequenza() → genera la sequenza del gioco
- giocoInCorso = true → segna che il gioco è iniziato

In questo modo, al giro successivo del loop, non rientreremo più in questo if.

Svolgimento del gioco

Successivamente controlliamo se il gioco è attivo:

```
if (giocoInCorso)
```

Se è così, eseguiamo:

```
mostraSequenza();
turnoDelGiocatore();
```

- mostraSequenza() → Arduino mostra la sequenza di luci e suoni
- turnoDelGiocatore() → il giocatore deve ripeterla

Queste due funzioni rappresentano il ciclo principale del gioco.

Perché usare funzioni?

Abbiamo creato delle funzioni per:

- rendere il codice più **ordinato**
- evitare ripetizioni
- migliorare la leggibilità

Infatti, leggendo solo setup() e loop(), possiamo già capire come funziona il programma.

Dove si scrivono le funzioni?

Le funzioni personalizzate possono essere scritte in qualsiasi punto dello sketch, ma è buona pratica metterle **dopo il loop()**.

Così prima vediamo la logica generale del programma, poi i dettagli.

```
44.
45. void spegniTutteLeLuci() {
46.   for (int i = 0; i < NUMERO_PULSANTI; i++) {
47.     digitalWrite(luci[i], LOW);
48.   }
49. }
```

Come suggerisce il nome, questa funzione serve a **spegnere tutte le luci**.

Per farlo utilizziamo un ciclo **for** che scorre tutti gli elementi del vettore luci[ ].

Il ciclo parte da 0 e arriva fino a NUMERO_PULSANTI - 1, cioè attraversa tutti i LED.

Ad ogni iterazione eseguiamo:

```
digitalWrite(luci[i], LOW);
```

- luci[i] → seleziona il LED corrente
- LOW → lo spegne

Perché funziona?

Questo è possibile perché:

- abbiamo inserito tutti i pin dei LED nel vettore luci[ ]
- il numero di LED è uguale al numero di pulsanti (NUMERO_PULSANTI)

In questo modo possiamo gestire tutti i LED con un unico ciclo, senza ripetere codice.

```
50.
51. void accendiTutteLeLuci() {
52.   for (int i = 0; i < NUMERO_PULSANTI; i++) {
53.     digitalWrite(luci[i], HIGH);
54.   }
55. }
```

Questa funzione è molto simile alla precedente, ma invece di spegnere i LED li **accende tutti**.

Anche in questo caso utilizziamo un ciclo **for** per scorrere tutti gli elementi del vettore luci[ ].

Ad ogni iterazione eseguiamo:

```
digitalWrite(luci[i], HIGH);
```

- luci[i] → seleziona il LED corrente
- HIGH → lo accende

Grazie al vettore e al ciclo **for**, possiamo accendere tutti i LED con poche righe di codice, senza dover scrivere una riga per ogni pin.

```
56.
57. void illumina(int indice) {
58.    digitalWrite (luci[indice], HIGH);
59.    tone(piezo, note[indice]);
60. }
```

Questa funzione serve per **accendere un singolo LED e riprodurre il suono associato**.

La funzione illumina() è di tipo void, quindi **non restituisce nessun valore**, ma accetta un parametro in ingresso:

- indice → indica quale LED (e quale nota) vogliamo utilizzare

Come funziona?

All'interno della funzione utilizziamo l'indice per accedere ai vettori:

```
digitalWrite(luci[indice], HIGH);
```

- accendiamo il LED corrispondente

```
tone(piezo, note[indice]);
```

- riproduciamo la nota associata

Perché funziona così bene?

Perché abbiamo organizzato i dati in modo intelligente:

- luci[ ] → contiene i pin dei LED
- note[ ] → contiene le frequenze delle note

Entrambi i vettori hanno lo **stesso ordine**, quindi:

- indice 0 → LED 0 + nota 0
- indice 1 → LED 1 + nota 1
- e così via

Questo significa che con un solo numero (**indice**) controlliamo **luce e suono insieme**!

```
61.
62. void spegni(int indice) {
63.    digitalWrite (luci[indice], LOW);
64.    noTone(piezo);
65. }
```

Questa funzione fa l'esatto contrario della funzione illumina():

- spegne il LED
- interrompe il suono

Come funziona?

```
digitalWrite(luci[indice], LOW);
```

- luci[indice] → seleziona il LED
- LOW → lo spegne
  noTone(piezo);
- interrompe qualsiasi suono in uscita dal piezo

```
66.
67. void creaSequenza() {
68.   for (int i = 0; i < LIVELLO_MASSIMO; i++) {
69.     sequenza[i] = random(0, NUMERO_PULSANTI);
70.   }
71. }
```

Entriamo nel vivo del gioco!

Questa funzione serve a **generare la sequenza di luci e suoni** che il giocatore dovrà ripetere.

Come funziona?

Utilizziamo un ciclo **for** per riempire il vettore sequenza[ ].

Il ciclo va da 0 fino a LIVELLO_MASSIMO - 1, quindi genera tutta la sequenza una sola volta.

Generazione dei numeri casuali

All'interno del ciclo troviamo:

```
random(0, NUMERO_PULSANTI);
```

Questa funzione genera un numero casuale.

Attenzione importante:

- il valore minimo (**0**) è **incluso**
- il valore massimo (**NUMERO_PULSANTI**) è **escluso**

Quindi, se NUMERO_PULSANTI = 4, i valori possibili saranno:

- 0, 1, 2, 3

Riempimento della sequenza

```
sequenza[i] = random(0, NUMERO_PULSANTI);
```

Ad ogni ciclo:

- viene generato un numero casuale
- il numero viene salvato nella posizione i del vettore

In questo modo otteniamo una sequenza come, ad esempio:

2, 0, 3, 1, 2, ...

Ogni numero rappresenta:

- un LED
- un pulsante
- una nota

Perché generiamo tutta la sequenza subito?

Per semplicità!

Creiamo tutta la sequenza una sola volta, fino al livello massimo.

Poi, durante il gioco:

- mostreremo solo una parte della sequenza
- questa parte crescerà man mano che il livello aumenta

```
72.
73. void mostraSequenza() {
74.    spegniTutteLeLuci();
75.
76.  for (int i = 0; i < livello; i++) {
77.    illumina(sequenza[i]);
78.    delay(velocita);
79.    spegni(sequenza[i]);
80.    delay(velocita);
81.  }
82. }
```

Questa funzione serve a **mostrare la sequenza che il giocatore deve riprodurre**.

Come funziona?

Per prima cosa spegniamo tutte le luci:

```
spegniTutteLeLuci();
```

In questo modo partiamo da una situazione "pulita".

Mostrare la sequenza

Successivamente utilizziamo un ciclo **for**:

```
for (int i = 0; i < livello; i++)
```

Questo è molto importante:

- non mostriamo tutta la sequenza
- ma solo fino al **livello attuale**

Se il livello è 3, mostreremo solo i primi 3 elementi.

Quando il giocatore vince, il livello aumenta, quando perde si riparte da 1.

Cosa succede ad ogni iterazione?

Per ogni elemento della sequenza:

```
illumina(sequenza[i]);
```

- accendiamo il LED corrispondente
- riproduciamo il suono associato
  delay(velocita);
- manteniamo acceso il LED per un certo tempo
  spegni(sequenza[i]);
- spegniamo il LED
- fermiamo il suono
  delay(velocita);
- aspettiamo prima di passare al prossimo elemento

```
83.
84. int aspettaPressione() {
85.  while (true) {
86.    for (int i = 0; i < NUMERO_PULSANTI; i++) {
87.     if (digitalRead (pulsanti[i]) == LOW) {
88.       delay(50); // ✅ debounce
89.       return i;
90.      }
91.    }
92.   }
93. }
```

Eccola qui! Finalmente una funzione che **restituisce un valore**.

Infatti, questa funzione è di tipo int, quindi restituisce un numero intero.

Cosa fa questa funzione?

Questa funzione **aspetta finché non viene premuto un pulsante**.

Appena un pulsante viene premuto:
restituisce l'indice del pulsante premuto

Il ciclo infinito

```
while (true)
```

Questo è un **loop infinito**.

Significa: "continua per sempre"

Infatti:

- true vuol dire **vero**
- e vero… è sempre vero!

Quindi Arduino continua a controllare i pulsanti senza fermarsi.

Controllo dei pulsanti

All'interno del while utilizziamo un ciclo **for**:

```
for (int i = 0; i < NUMERO_PULSANTI; i++)
```

Questo ciclo controlla tutti i pulsanti uno per uno.

Rilevamento della pressione

```
if (digitalRead(pulsanti[i]) == LOW)
```

- LOW significa che il pulsante è premuto (ricorda: usiamo INPUT_PULLUP)

Quando troviamo un pulsante premuto:

- return i;
- usciamo subito dalla funzione
- restituiamo l'indice del pulsante premuto

Il debounce:

```
delay(50);
```

Serve a evitare il problema del **rimbalzo del pulsante**.

Quando premi un pulsante, il segnale può "oscillare" per pochi millisecondi.
Questo piccolo ritardo rende la lettura più stabile.

```
94.
95. void turnoDelGiocatore() {
96.   for (int i = 0; i < livello; i++) {
97.     int j = aspettaPressione();
98.     illumina(j);
99.
100.      while (digitalRead(pulsanti[j]) == LOW) {
101.      }
102.
103.      delay(250);
104.      spegni(j);
105.
106.      if (j != sequenza[i]) {
107.       haiPerso();
108.       return;
109.      }
110.    }
111.    haiVinto();
112.
113. }
```

Questa è la funzione più importante del gioco
Qui controlliamo se il giocatore riesce a **ripetere correttamente la sequenza**.

Come funziona?

Per prima cosa utilizziamo un ciclo **for**:

```
for (int i = 0; i < livello; i++)
```

Il ciclo va avanti fino al livello attuale.
Se siamo al livello 3, il giocatore dovrà premere **3 pulsanti corretti**.

Cosa succede ad ogni turno?

Aspettiamo che il giocatore prema un pulsante:

```
int j = aspettaPressione();
```

j rappresenta l'indice del pulsante premuto

Accendiamo il LED e il suono corrispondente:

```
illumina(j);
```

Aspettiamo che il pulsante venga rilasciato:

```
while (digitalRead(pulsanti[j]) == LOW) {
}
```

Questo serve per evitare letture multiple dello stesso pulsante

Piccola pausa e spegnimento:

```
delay(250);
spegni(j);
```

Il delay aiuta anche a rendere il comportamento più stabile (debounce)

Controllo della risposta:

Ora arriva il punto chiave:

```
if (j != sequenza[i])
```

Confrontiamo:

- j → quello che ha premuto il giocatore
- sequenza[i] → quello corretto

Se sbaglia

```
haiPerso();
return;
```

- mostriamo la sequenza di sconfitta
- usciamo subito dalla funzione return e serve proprio per interrompere tutto e non continuare il controllo.

Se non sbaglia mai

Se il giocatore completa tutto il ciclo senza errori:

esce dal for automaticamente
significa che ha **indovinato tutta la sequenza**

A questo punto, nel programma principale, verrà chiamata la funzione di vittoria

```
114. void haiPerso() {
115.   noTone(piezo);
116.
117.   // 🔊 SUONO DI SCONFITTA
118.   tone(piezo, 200);
119.   delay(500);
120.   noTone(piezo);
121.
122.   for (int i = 0; i < 3; i++) {
123.     accendiTutteLeLuci();
124.     delay(250);
125.     spegniTutteLeLuci();
126.     delay(250);
127.   }
128.
129.   livello = 1;
130.   velocita = 900;
131.   giocoInCorso = false;
132. }
```

Questa funzione viene chiamata quando il giocatore **sbaglia la sequenza**.

Serve a:

- segnalare la sconfitta
- resettare il gioco

Interruzione del suono

```
noTone(piezo);
```

Per prima cosa fermiamo qualsiasi suono in corso.

Suono di sconfitta

```
tone(piezo, 200);
delay(500);
noTone(piezo);
```

- riproduciamo un suono grave (200 Hz)
- lo manteniamo per mezzo secondo
- poi lo interrompiamo

Questo serve a far capire subito al giocatore che ha perso.

Effetto luminoso

```
for (int i = 0; i < 3; i++) {
accendiTutteLeLuci();
delay(250);
spegniTutteLeLuci();
delay(250);
}
```

Facciamo lampeggiare tutte le luci per **3 volte**.

Questo crea un effetto visivo chiaro e immediato di “errore”.

Reset del gioco

```
livello = 1;
velocita = 900;
giocoInCorso = false;
```

Qui riportiamo tutto allo stato iniziale:

- livello = 1 → si ricomincia dall'inizio
- velocita = 900 → il gioco torna lento
- giocoInCorso = false → il gioco si ferma

A questo punto il giocatore dovrà premere di nuovo il pulsante di avvio.

```
133.
134.  void haiVinto() {
135.    // 🔊 SUONO DI VITTORIA
136.    tone(piezo, 523);
137.    delay(150);
138.    tone(piezo, 659);
139.    delay(150);
140.    tone(piezo, 784);
141.    delay(200);
142.    noTone(piezo);
143.
144.    spegniTutteLeLuci();
145.    delay(250);
146.    accendiTutteLeLuci();
147.    delay(500);
148.    spegniTutteLeLuci();
149.    delay(500);
150.
151.      if (livello < LIVELLO_MASSIMO) {
152.  livello++;
153.      if (velocita > 250) {
154.      velocita -= 50;
155.      }
156.    }
157.  }
```

Questa funzione viene chiamata quando il giocatore **completa correttamente la sequenza**

Serve a:

- mostrare un feedback di vittoria
- aumentare la difficoltà del gioco

Suono di vittoria

```
tone(piezo, 523);
delay(150);
tone(piezo, 659);
delay(150);
tone(piezo, 784);
delay(200);
noTone(piezo);
```

Riproduciamo una breve sequenza di note crescenti.

Questo crea un effetto sonoro tipico di "successo".

Effetto luminoso

```
spegniTutteLeLuci();
delay(250);
accendiTutteLeLuci();
delay(500);
spegniTutteLeLuci();
delay(500);
```

Accendiamo e spegniamo i LED con un ritmo diverso rispetto alla sconfitta.

Serve a dare un feedback visivo positivo e soddisfacente.

Aumento della difficoltà

```
if (livello < LIVELLO_MASSIMO)
```

Controlliamo di non aver raggiunto il livello massimo.

Se non lo abbiamo ancora raggiunto:

livello++;

aumentiamo il livello → la sequenza sarà più lunga

Aumento della velocità

```
if (velocita > 250) {
velocita -= 50;
}
```

riduciamo il tempo di attesa tra i segnali

Questo rende il gioco:

- più veloce
- più difficile

Mettiamo un limite minimo (250 ms) per evitare che diventi impossibile.

Prova a modificare il codice!

Puoi:

- aggiungere altri pulsanti
- aumentare il numero di livelli
- rendere il gioco più veloce… o ancora più difficile

Non aver paura di sbagliare:
è proprio sperimentando che si impara davvero!

NON È FINITA QUI!

Speriamo davvero che ti sia divertito con questi esperimenti per avvicinarti al mondo di Arduino e della programmazione!

Nei prossimi volumi approfondiremo i concetti che hai imparato, entrando ancora più nel dettaglio.

Utilizzeremo nuovi componenti e, soprattutto, inizieremo a usare i sensori per costruire veri sistemi sempre più complessi.

Grazie per averci accompagnato in questa prima avventura!

Speriamo di ritrovarti anche nelle prossime tappe di questo viaggio verso il fantastico mondo dell'elettronica!